

VECTOR

J and APL Conference Reports

Plus ...

- | | |
|-----------------------------------|-----|
| • Vector on the Web | 16 |
| • Scholes on Dynamic Functions | 88 |
| • Donnelly on OLE Automation | 99 |
| • Smillie on Calendars in J | 117 |
| • Burke on Indexing in APL2 and J | 137 |



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.13 No.2 October 1996

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1996-97

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

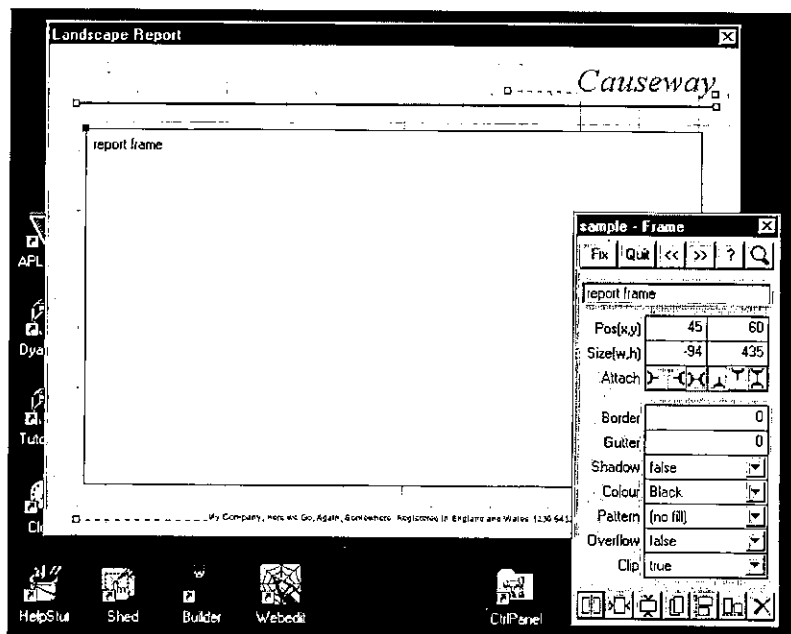
Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 01439-788385 CompuServe: 100331,644

Contents

		Page
Editorial	Duncan Pearson	3
APL NEWS		
Quick Reference Diary and Call for Nominations		5
Correspondence		7
Vector on the Web	Ray Cannon	16
News from Sustaining Members	Gill Smith	20
The Education Vector	Ian Clark	25
APL Product Guide - Updates	Gill Smith	41
RECENT MEETINGS		
The First International J User Conference (Toronto, June 24th — 25th 1996)	Duncan Pearson	48
APL96 at Lancaster (July 28th - August 1st 1996)		51
A Conference Diary	Ray Cannon	
TCP/IP with Timo Laurmaa	Jon Sandles	60
ODBC Client-Server with Morten Kromberg	Bronwen Tuffen	65
Nested Arrays with Phil Benkard	Jon Sandles	68
Gui Programming and Multi-dimensional DBs	Marc Griffiths	70
A Consultant's View	Richard Procter	77
Conference Roundup	Adrian Smith	83
GENERAL ARTICLES		
Dynamic Functions in Dyalog APL	John Scholes	88
OLE Automation with Dyalog APL	Peter Donnelly	99
Perfect Shuffles in J	Howard Peelle	108
Calendar Calculations in J	Keith Smillie	117
Swinging with APL: the Simple Pendulum	Joseph De Kerf	128
TECHNICAL SECTION		
Hacker's Corner: APLPLUS/PC & Windows 95	Adam Weber	134
A Word (or Two) from our Spenser	Norman Thomson	135
APL2 and J (2) - Indexing	Chris Burke	137
At Play with J: Volutes	Gene McDonnell	144
Extended Integers in J	Roger Hui	154
Index to Advertisers		159

NewLeaf



A professional page-layout tool for
the APL developer

- complex layouts with text, images and graphics
- fast high-volume reporting from tabular data

Available for Dyalog APL/W and APL+Win

Price £400 (\$600) includes on-line help, printed
User's Manual, and one year's telephone support.

Causeway Graphical Systems Ltd.
The Maltings, Castlegate
MALTON, North Yorkshire
YO17 0DP

Tel: +44 (0) 1653 696760
Fax: +44 (0) 1653 697719
Video: +44 (0) 1653 693006

100265.1564@compuserve.com

Editorial

by Duncan Pearson

APL96 at Lancaster was seen by many of those that attended as one of the better and more useful annual APL conferences. From the people to whom I spoke I derived a very consistent idea of why this was: APL96 was geared to learning. The emphasis of the conference was on enabling the attendee to learn new skills and techniques that would seriously enhance his ability to do his job. Furthermore the things that were being taught — and I do mean taught — were about the integration of APL into the increasingly complex web of modern computing. Many papers at earlier conferences were about the use of an APL approach to solve certain computational problems. As the various conference reports in Vector will show, the emphasis is now on making those solutions available to other users, on interfacing with TCP/IP, ODBC, EDI &c.

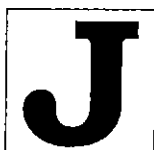
Nowhere was this more apparent than in the brilliant way that Dyadic systems demonstrated for creating OLE servers from APL workspaces. An OLE server is a "black box" that is generally designed to do a specific thing. It provides "methods" (functions) to be run and "properties" (variables) to be queried or set by any Windows tool that can be an OLE client. The inner workings of the server are invisible and immaterial to the user and the data interface is standard, and remarkably like a (possibly nested) APL array. It is the ideal vehicle for the APL developers' skill, which is with the problems of analysis. The article by Peter Donnelly in this issue explains the process of generating an OLE server from a Dyalog APL/W workspace and should be read by all APL developers who are anywhere near the Windows marketplace.

APL96 was not however the only significant conference this year. June saw a very successful inaugural J conference in Toronto. It lasted two days, was very well attended and was a solid indication of the importance of the language. It was clear that for J, unlike APL, the interest was in the use of the language. Two talks which were repeated in amended form in Lancaster typify this. Chris Burke's tutorial and Donald McIntyre's talk on forks were about the language pure and simple. The extra richness of the J language has obviously not yet been fully digested by the community.

Finally, and unfortunately too late for the inclusion of a report in this issue, we have had the APL2000 user conference in Orlando. The programme again seems to have a bias toward the teaching of techniques for interfacing with the outside world. We will have a full report in the January Vector.

Great Things Come in Three's...

Announcing:



Release 3

*Now
Available
via
OLE...*

J Release 3 for Windows 95/NT from Iverson Software makes access to J a breeze for users of Visual Basic, Delphi or any OLE-compliant software. Select the J EXE local server for development with full access to the J system, or the J DLL in-process server for optimal run-time performance.

Prefer to develop you applications only in J? You get a sophisticated Windows integrated development environment, with built-in script and forms editors, a session manager with customizable menus, support for multiple script and execution sessions, a simple yet effective Windows interface, access to virtually any database, and much more.

OLE support is provided with the Professional Edition, which also includes royalty-free runtime use. J Release 3 is also available in Standard and Educational Editions, and for other systems, including PC DOS, Windows 3.1, Linux, Mac , Sun and RS/6000.

Discover how J can work for you. Check out our web site: jsoftware.com

Proceedings from the J User Conference are now available. For more information, contact Anne Faust, Strand Software, 19235 Covington Court, Shorewood, MN 55331, Tel (612) 470-7345, Fax (612) 470-9202, email: info@jsoftware.com. Also available through dealers worldwide.

Quick Reference Diary 1996-7

Date	Venue	Event
October 23 to 25th	Orlando Florida	APL2000 User Conference
June 22-23 1997	Toronto	J User Conference

Nominations for the British APL Association Committee

The next AGM must be held in May 1997 and, seen from this side of Christmas, that seems a long way off. However, there are just two issues of Vector between now and then. Here, therefore, is an appeal for people to serve on the Committee for the year beginning next May.

Please persuade and nominate your friends or colleagues and we shall be happy to publish details of their candidature in the January issue of Vector. The election will be held in April by postal ballot. Anyone who would like to be given a feel for what is expected of the various Committee members should contact someone on the current Committee who would be delighted to receive any expression of interest.

Please send your nominations to the Secretary.

Dates for Future Issues of VECTOR

	Vol.13 No.3	Vol.13 No.4	Vol.14 No.1
Copy date	6th Dec 96	8th March 97	6th June
Ad booking	13th Dec 96	15th March 97	13th June
Ad Copy	20th Dec 96	22nd March 97	20th June
Distribution	January 97	April 97	July 97



Experienced APL Developers York / Croydon

Nestlé SA is the world's biggest food manufacturer - with Nescafé, Lyons Maid, Buitoni, Crosse & Blackwell and Kit Kat amongst its major brands. For Nestlé to achieve its mission to be the UK's best branded food company, we depend on powerful Supply Chain decision support systems.

APL has been at the heart of our corporate planning and forecasting systems for the last 17 years. In addition to a sizeable portfolio of existing core applications running on mainframe APL*PLUS and in DOS under APL*PLUS II, our new developments are now Windows 95 based, using Dyalog APL version 8.0.

We are now looking to recruit two experienced APL developers to join our specialist Operational Research Systems group. OR provides decision support systems throughout the business including Logistics, Sales, Distribution, Personnel, and Manufacturing. The work we are involved in is high profile and, given success, would provide an excellent basis for career progression within the Nestlé group.



If you have what it takes to be the best and can offer either

- at least 2 years mainframe APL*PLUS or Dyalog/W in a modelling environment
- or at least 4 years APL knowledge across multiple platforms, and recent experience of building GUI objects and working in a mixed language environment (exposure to C++, Oracle, NT, Win 95, VBA preferred)



..... we'd like to hear from you.

In return we can offer you a competitive salary and benefits package including flexible hours.

Contact: Jackie Johnson (Personnel Manager)
Nestlé UK, York, YO1 1XY
tel: (01904) 603338
email: jackie.johnson@nestlegb.nestle.com



Nestlé

CORRESPONDENCE

The APL Skills Database

From: Richard Procter (rjp@interlog.com)

17th August 1996

A joint project of:

The Toronto APL Special Interest Group
The ACM Special Interest Group on APL

The Toronto APL Special Interest Group, and the Association for Computing Machinery Special Interest Group on APL, are pleased to provide the free employment information service known as the APL Skills Database.

Recognizing the niche-market nature of the use of APL in data processing, the APL Skills Database project was launched in 1994 to aid companies and employers looking for APL-skilled personnel, and to help APL-skilled professionals seeking employment opportunities with companies needing APL expertise. Now, with the implementation of its Internet resources, the APL Skills Database is even better. To date, at least three participants in the project have been successfully placed in APL-related employment positions by using this service!

How does it work?

The APL Skills Database consists of an Internet Website and Email facility, which allows both employers and employees to contribute and browse employment information. Employers are encouraged to submit job details to the database, which will be posted at our website, and will also be broadcast by email to participating APL-skilled consultants and potential employees. All incoming information and questions about the project can be directed to our central email address, at: apl_jobs@acm.org

The Toronto APL SIG website is at:
<http://www.sigapl.mtnlake.com/sigapl/welcome.html>

The ACM SIGAPL website is at:
<http://www.acm.org/sigapl>

The Skills Database project also offers to post résumé information of APL-skilled consultants, contractors and employees who may be in the job market. Just send your details to apl_jobs@acm.org, and we'll post it on our website. Content of this information is up to you, be as brief or as detailed as you wish. If you already have your own site on Internet, we'll link to it from our site.

How can I get involved?

The APL Skills Database is a free service offered by the Toronto APL SIG and ACM SIGAPL. If you're an employer looking for APL skills, just send us the details of the position being offered and we'll post them on our website right away. APL-skilled persons who so wish will be sent these incoming job notices via email as they arrive. Candidates for employment requiring APL skills are encouraged to contact employers and apply for the job if interested. Employers are also encouraged to browse our résumé database for APL-skilled personnel. Please send all questions, job notices, or your desire to participate in the project and subscribe to our email job posting service, to our address at: apl_jobs@acm.org

The APL Skills Database can only work with the enthusiastic participation of employers seeking APL skills, and employees seeking APL employment. Please join us in this unique project.

Richard J. Procter
Chair — Toronto APL SIG

Why Isn't APL Used More Widely

From: R.G. Brown (MMSI)

28th June 1996

At some of the APL conferences and in Vector this question is a favorite topic. I can suggest an answer from observation, but first a few words of background. I first encountered APL when Ken Iverson did a seminar from Harvard. Later at IBM one of the programmers showed me how he solved a simple problem quickly in a few keystrokes. On inquiry I learned that he was using an early implementation of APL. I then took Al Rose's video tape course, and learned enough of the language in the first morning that I could use the script, without looking at the monitor, and got to the point of solving some of my own programs by the end of the three day course.

As I began to use APL more seriously I discovered that I was stating new problems, because subconsciously I felt that I had the tools to solve them. When I had been sure I couldn't have solved a problem, I never stated it. The logical and mathematical power of APL suited my mental processes to a T. Much later on someone invented $\square FMT$ and $\square$ which simplified further the way that I wanted to handle problems that were important to me.

In more recent years someone has invented $\square ELX$ and strand notation. I encountered both of these in a "by the way" comment in some talks at APL conferences. The manuals are extremely cryptic, stated in a compact way that might be an *aide memoire* for someone who had learned the techniques once. This was equally true for Dyalog, IBM and STSC. The manuals are no help at all to a programmer who wanted to get an insight into what these techniques do, how they work, and why you would want to do that. The $10 \bullet X$ logarithm and $\downarrow$ for sorting were intuitive when first presented. There is nothing intuitive about nested arrays, especially when they don't work in a consistent and predictable way.

Now I am forced by my market to convert good old DOS APL applications into Windows. Not only are the manuals obscure and opaque, I suspect they are counter-productive.

I suggest that one reason APL is not more widely used is that the only way one can learn the language is from being a member of an in group which uses its own priestly language to discriminate against outsiders and establish the supremacy of the *cognoscenti*. If a group wants wider use of the program product explain it in clear English with complete sentences, starting from providing an insight into what the language, or any part of it does, why it works, and what it might be used for.

re Vector Vol.12 No.3, January 1996, pp 133 - 142

From: Eddie Clough

19 February 1996

I reproduce a J session which I hope is self-explanatory:

NB. E. McDonnell, Vector Vol 12, No 3, pp 133-142.

NB. I find I cannot reproduce these results.

NB. Is it because I am still using an old version of J?

```
gf =. 1 _2&p. % 1 _4 1&p.
gf t. i.12
1 0.5 0 0 0 0 0 0 0 0 0 0
```

NB. I think the division is element by element, as in:

```
2 4 6 % 1 2 3
2 2 2
```

```
] 'ab'=. , > }. p. 1 _4 1
|nonce error
| ] 'ab'=. , > }. p. 1 _4 1
```

NB. Whatever that might mean!

Conifers, Leighton Beck Road
Slack Head, Beetham
Milnthorpe
Cumbria LA7 7AZ

This is indeed a limitation on your version of J. The answer is to obtain the latest version of J (freeware) from the JSoftware web site, or a UK source such as I-APL Ltd. [ed]

While in J ...

From: Nick Cox (N.J.Cox@durham.ac.uk)

8 June 1996

The so-called 'antecedent precipitation index' is a measure of the wetness of a catchment. Precipitation (rainfall, snow, etc.) makes a place wetter but such water is lost from a site over time by evaporation and drainage.

```
api[i] = precip[i-1] * k * api[i-1], 0 <= k <= 1
```

is one way of trying to capture this balance fairly simply.

The idea is pretty crude engineering hydrology and the API is naturally inferior to any number based on real measurements at a site. In practice, however, daily precipitation measurements for somewhere similar are far more readily available than are comparable soil moisture measurements, especially over long historical periods. I don't know much folklore on estimating k , but one rough physical guide is that at most places wetness fluctuates about an average, especially on time scales of the order of a year. This allows you to bracket plausible values of k .

That is just context. Like the typical reader of Vector, when reading such simple formulae, I often write down the corresponding simple programs. To me, with a brain warped by too many years of Fortran and Basic, and improved slightly by exposure to various C-like languages, what springs to mind first is something like `api1`. To any J or APL enthusiast, `api1` looks incredibly long-winded for such a mathematically trivial problem. And indeed in the best and worst traditions of APL a program can certainly be produced in one line, as for example `api2`.

Yet experiments clearly show that `api2` is much slower and more demanding of space, as examination of the programs suggests. Can a quicker yet concise J script be produced?

As yet more context, any one who wanted to do this could easily have 1000 daily rainfalls, and in fact I will want to do something similar for more like 40,000 values. (I'll probably do it with totally different software, but largely for reasons irrelevant to this journal.)

```
api1 =. 3 : 0
NB. antecedent precipitation index NJC 15 April 1996
NB. k api p
:
k =. x.
p =. y.
api =. 0
i =. 0
while. i < #y. do.
api =. api , (i { p) + k * {: api
i =. i + 1
end.
). api
)
```

```
api2 =. 3 : 0
NB. antecedent precipitation index NJC 15 April 1996
NB. k api p
:
+/"1(* x.&^@i.@-@#)\ y.
)
```

Sylvester's Algorithm (Again)

From: Howard A. Peelle

July 1996

In [1] Norman Thomson resurrects Sylvester's algorithm for converting a "vulgar fraction" (decimal between 0 and 1) into a list of "unit fractions" (or integer reciprocals). Here, Sylvester's algorithm is recast again in J for perspicuity.

First, compute the largest unit fraction less than the input:

```
unit =. >. &. %
```

For example:

```
unit Fraction =. 0.67296
0.5
```

Use this to decompose the input into the largest unit fraction and the "error" term:

```
decompose =. unit . (- unit)
```

For example:

```
decompose Fraction
0.5 0.17296
```

Use this to extend any such list by decomposing the last term:

```
extend =. }: , decompose@{:
```

For example:

```
extend 0.5 0.17296
0.5 0.1666667 0.006293333
```

And, finally, repeat it:

```
extend^:3 Fraction
0.5 0.1666667 0.006289308 4.02516e_6
```

The integer reciprocals are obtained easily. For example:

```
% extend^:3 Fraction
2 6 159 248437
```

Also, note the identity:

```

1      Fraction = +/ extend^:3 Fraction

```

Further, this applies to a vulgar mixed fraction as well:

```

      extend^:6 Mixed =. 3.67296
1 1 1 0.5 0.1666667 0.006289308 4.02516e_6

```

And to negative vulgar fractions (between _1 and 0):

```

      extend^:3 -Fraction
_1 0.25 0.07692308 0.0001169231

```

And to real, non-empty arrays:

```

      extend^:3"0 Fraction,-Fraction
0.5 0.1666667 0.006289308 4.02516e_6
_1      0.25 0.07692308 0.0001169231

```

All this may be embodied in a program to return the leading integers (without the error term) whose unit fractions approximate the vulgar fraction input:

```

sylvester =. [: ]:"1 [: % extend@]^:"0

```

For example:

```

3 sylvester (+,-) Fraction
2 6 159
_1 4 13

```

Also in [1] Norman Thomson codes a similar algorithm to represent a fraction (decimal) in a given number base. Here it is expressed simply:

```

f =. * 1&|

```

For example:

```

<. 5 f^:(i.6) Fraction
0 3 1 4 0 3
NB. +/0 3 1 4 0 3 * 5 ^ -i.6

```

Reference

[1] Thomson, N. *J-ottings* 9, Vector, Vol. 12, No. 4, April 1996 pp. 28-29

"Arrays, Arrays, wherefor art thou Arrays?"

From: Richard J. Procter (rjp@interlog.com)

24/8/96

Lancaster and APL96 were certainly a highlight on the APL calendar this year for those of us fortunate enough to attend. Indeed, with its many fine pubs and charming old-world appeal, the hamlet of Lancaster itself proved an agreeable selection as conference venue.

Not so well recognized however, by most attendees, is the fact that England is indeed the place where APL and array-oriented languages have been upper most in the minds of some individuals for centuries. In fact, just to prove this, while rummaging around in one of the curio shops in downtown Lancaster, I couldn't believe my luck when I stumbled upon this extremely obscure snippet of genuine poetry from none other than the immortal bard himself, William Shakespeare.

This work, incredibly written sometime around 423 B.I. (Before Iverson), is indeed a truly remarkable piece for it's time. Something, I would dare say, that the APL community should proudly point to as its "Magna Carta" of documents, entirely paving the way to the array approach to problem solving in everyday life.

Here's the text from the dusty manuscript, in its entirety:

*To C++, or not to C++: that is the question:
Whether 'tis better for the job security to suffer
The loops and gotos of outrageous scalar programming,
Or to take arrays against a sea of if/else statements,
And by boolean compression end them? To)off: to $\square$ DL;
No more; and by a $\rightarrow 0$ to say we end
The heart-ache and the thousand variable declarations
That compiled code is heir to, 'tis a link-edit
Devoutly to be abend'd. To)off, to $\square$ DL;
To $\square$ DL: perchance to multi-task; ay, there's the rub;
For in that delay mode what dreams may come
When we have shuffled off this if-then-else coil,
Must give us interactive systems: there's the modularity
That makes calamity of so long programs;
For who would bear the compile and links of time,*

*The compiler's wrong output, the proud programmer's frustration,
The pangs of despised loops, the rewrite's delay,
The insolence of array-unfriendly management and the rewards
That patient merit of the worthy takes and drops,
When he himself might his report format
With a bare bottom-line? who would 'for-while' bear,
To grunt and sweat under a weary do-loop,
But that the dread of something after installation,
The undiscover'd bug from whose general protection failure
No programmer returns, puzzles the support person
And makes us rather bear those workarounds we have
Than kludge others that we know not of?
Thus fear of bombing does make cowards of us all;
And thus the hue of screen resolution
Is dot-matrix'd o'er with the pale cast of pixels,
And enterprises of great input/output
With this regard their proc(main) turns awry,
And lose the prompt of abort-retry-fail. Software you now!
The fair APL! Nymph, in thy implementation
Be all my array solutions remember'd.*

Vector on the Web

by Ray Cannon (100430.740@compuserve.com)

At long last, *Vector* now has its own web site at <http://www.vector.org.uk>. This site is provided to supplement *Vector*, and is not to meant to replace it. For the rest of this article I will call the web site "[vector.org](http://www.vector.org.uk)" and the journal "*Vector*".

Currently [vector.org](http://www.vector.org.uk) contains a full back index to *Vector* vols. 1-12, a revamped product guide (see later), BAA membership details, details of the current BAA committee and sustaining members (in fact almost the whole contents of *Vector's* inside and outside front and back covers), links to other APL-related sites, information about the latest volume (including contents pages) and some articles from recent editions. Additionally, [vector.org](http://www.vector.org.uk) also has some APL workspaces which contain material published in *Vector* ready for downloading.

APL Characters on the Web

There is, at present, no standard method of specifying the symbol set of characters being used in web pages except in respect of language. (Note that language here explicitly EXCLUDES computer languages.) By default, the Latin1 symbol set is assumed.

I have tried several approaches to the problem of putting APL characters up on web pages. (Note that HTML uses 8-bit characters not the 7-bit characters that email and news groups support. So the problem is not so much sending the characters, but selecting the correct symbol set to display them.)

The idea of putting up "gifs" (images) of individual APL characters is not practicable, although a gif of a function can work quite well. However, trying to cut APL characters from a gif and pasting them into an APL workspace might not have the desired effect! It is also quite expensive in space, slow to download, and will not work on a "text only browser".

Use of Jim Weigang's APL/ASCII transliteration could work very well, but would not win any prizes for visual elegance, and might well put off the APL novice. It is however very transportable across machines, interpreters and web browsers, and with the appropriate (as yet unwritten) browser "applet", could be converted and displayed in the correct font.

Use of third party software (such as Acrobat PDF) works quite well, but only for those with the correct software installed. (Adrian Smith has put up APL articles in the past in this form.)

The best we have come up with is to put in any APL characters tagged as requiring a "fixed pitch font" and setting the mono-spaced font in the browser to an appropriate APL font. Note that because of the way Netscape detects mono-spaced fonts, the currently available Truetype fonts from Dyalog and *Vector* cannot be used by versions 2 or 3 of Netscape browsers. The APL fonts from APL*PLUS and APL2000 are OK. Adrian hopes to make his APL2741 available soon as a proper mono-spaced Truetype font.

Not only can the text displayed in this way be viewed as APL (with just a minor modification to the set-up of most (PC) web browsers), but the text may be cut from the browser and then pasted into the appropriate APL WS. This can work successfully even if the browser is not using the correct font.

Perhaps a 3-part combined solution might be the best compromise: put the APL text up (tagged as requiring a mono-spaced font), with links to a second page which has the APL/ASCII transliteration, and a third page with a gif of the APL text.

Feedback and suggestions in this area are welcome.

Product Guide

I am attempting to put the information currently available from the Vector Product Guide up into `vector.org` in a useful (and maintainable) manner. I am transferring the contents of the current product guide from a Word document into a series of Windows INI files from whence I am automatically generating the on-line `vector.org` Product Guide.

Each company or organisation will end up with a `vector.org` web page of its own. Each page will contain the contact details of the company (complete with working email and `www` links to the company, if available), followed by a list of each product the company offers in *Vector*. Each page is supplemented with its own "hit" counter, and date of last update.

Soon I hope to offer a "Vector Product Guide Input Form" from `vector.org` which will allow existing (and future) product guide contributors a simple method of supplying the Vector production team with their latest "copy".

The BAA hopes to be able to offer its product guide contributors extra web space, where we can put up any additional web pages the contributors wish to supply for a nominal fee. Final details have not yet been decided, but prices will be about an order of magnitude less than the *Vector* Advertising rates. Current suggestions are for a flat fee of say £10 every time the page(s) is(are) updated, plus a £10 fee per page per Vector edition (3 months). Thus a page being updated once a year would cost £50, while 3 pages updated 6 times in a year would cost £180. Monies raised by the sale of extra web space would be used to help cover the costs incurred in providing `vector.org`.

Note: the standard product guide entry (both in *Vector* and `vector.org`) will continue to be free of any charge.

Vector Back Numbers

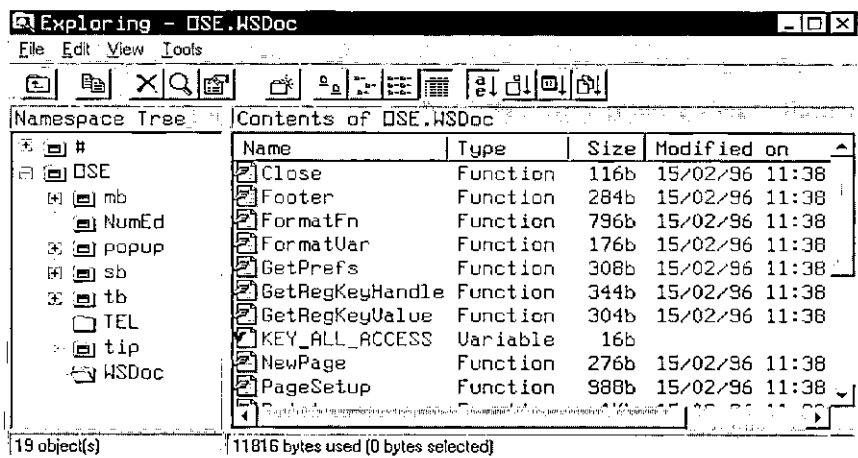
Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

dyalog APL

The Definitive APL for Windows™



Dyalog APL Namespaces let you ...

- **Organise** your workspace into self-contained sub-systems
- **Encapsulate** functions and variables *within* GUI objects
- **Isolate** utilities from your application code

plus The new Version 8 **Explorer** lets you browse namespaces, drag-drop objects from one to another, and a whole lot more.

That's why Dyalog APL/W remains the **professional** choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.
Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



News from Sustaining Members

Compiled by Gill Smith

Dyadic Systems Limited

At APL96, Dyadic announced two major new developments for Dyalog APL. The first of these is a new type of defined function called a *Dynamic Function*. Dynamic functions are fast, lightweight functions with intriguing possibilities for improving performance and reliability.

The second major new feature is the capability to turn a Dyalog APL namespace into an *OLE Server* object that can be called from Visual Basic, C++, Excel and so forth. This feature will allow you to package and distribute your APL code in a form that can readily be incorporated into non-APL applications in a totally standard manner.

Articles describing these exciting new features may be found in this issue of Vector.

In addition to the capability to act as an OLE Server, Dyalog APL/W Version 8.1 will also include the ability to act as an *OLE Automation* client. This facility is also founded upon the direct correspondence between Dyalog APL namespaces and OLE objects and will allow you to drive numerous Windows applications from APL.

Finally, Dyadic is announcing *TCP/IP socket support* for Version 8.1. Not only will this allow you to access the Internet directly from APL, but it will provide the basis for future client/server operations between Dyalog APL for Windows and Dyalog APL for UNIX platforms.

Causeway Graphical Systems Ltd

The major news for this quarter is the long-awaited release of NewLeaf and RainPro for both the Dyalog APL/W and APL+Win platforms. Adrian and Gill have just been demonstrating the APL+Win versions at the APL2000 user show in Orlando.

RainPro is an extension to the Rain package distributed as shareware with Dyalog APL/W. It includes 3D tower charts, scatter plots and surfaces and has the facility to copy graphs to the clipboard as metafiles. It comes with both the standard help file and a 48-page user guide which has many worked examples.

NewLeaf is a completely new product which breaks fresh ground in providing the capabilities of a desk-top publishing package to the APL programmer. It is a "frame-paragraph-style" based product which takes the difficulty out of producing well laid out reports in proportional fonts. It can print complex tables with ease and supports tree-structured table headings in the same way as the Dyalog APL/W grid control. Indeed it would be the ideal tool to provide the print method for that control.

Each of RainPro and NewLeaf is a set of APL functions that have simple syntax, take array arguments and that you can integrate into your code easily. The Dyalog APL/W and APL+Win versions of NewLeaf and RainPro are functionally identical although the Dyalog versions use namespaces to hide the functions. They are written entirely in APL and so are fully modifiable by the user, something not true of any OCX or VBX solution you might consider.

The second piece of news is that at the end of October we will be joined by Jon Sandles. Many will be familiar with Jon from his role as recruitment and meetings organiser for the BAA and from his work on the Vector Working Group.

Jon has been involved in the development of Causeway from the start and has a deep understanding of its internal structure as well as having written perhaps more Causeway applications than anyone else we know. He also has experience of maintaining a site of Causeway developers and developing his own Causeway classes, that having been part of his job. His experience in this area will be incorporated into the forthcoming CausewayPro for which we have received many requests and on which we are currently working. He will be a major asset to the company and will allow us some much-needed flexibility in scheduling our new developments.

Insight Systems / Adaytum Software

Converting applications to Windows is an unpredictable business. At Insight, we have just completed the process, including rollout of the apparently inevitable bugfix release, for Adaytum Planning, our multi-dimensional business planning product — with only slightly more than a 100% overrun of the project plan. Fortunately, despite the delay, sales of Adaytum Planning have continued to grow rapidly; we now have several customers in Denmark and opened an office in the USA in September. The success allows us to grow our development team to cope with the workload; since August we have recruited an APL Developer, a Quality Assurance Manager, and an Internal Support Manager. We expect to recruit additional APL and C developers during the next six months; if you are

an experienced APL or C developer (especially if you are both), and interested in a job in Denmark, drop us a line.

As a result of the delays in our Windows conversion project, even only slightly attentive readers will have noticed that our sustaining news items have remained pretty static; almost all our announced product releases have gradually slipped by between 6 and 12 months. The ideas for and the driving force behind our Client/Server products originate in our own requirements for tools which make it possible to position our APL applications as first class citizens of the emerging world of distributed applications. It is not easy to get ideas while fighting bugs and getting an application to behave itself under Windows.

The good news is that we have now actually started work on new releases of our APL Client/Server products, for internal use in the next release of Adaytum Planning and for general release to APL users. Following discussions with APL vendors and users at APL96, we have decided to start with a revision of the SQAPL Server, with the primary goal of improving performance and making the server available on low-end platforms such as Windows NT, OS/2 and even "stand alone" under Windows 3.x or '95. A revised SQAPL Client, with functionality as described in the April 1996 Vector, will follow later.

So, if any of you intend to downsize to windows and are planning to base connectivity on our products, you may proceed with confidence. By the time you are done with the conversion and have the bugs out, the new SQAPL generation will be waiting for you.

HMW Trading Systems Ltd.

HMW have completed the first two phases of porting their APL*Plus based Trading Room product 4XTRA to Dyalog for Windows Version 7.4. The third and final phase includes converting the remaining code which requires a GUI interface, and then switching to Version 8.

The functionality of the new system is in fact a superset of the existing 4XTRA versions, adding Money Market products such as FRAs and Financial Futures. To reflect the increased capability of the product, the Windows version is now known as *iDeal*.

The GUI interface uses both Causeway and specialised routines making direct Dyalog calls (dealing systems have some uncommon requirements). The system also has interfaces to databases (using the ODBC links provided by Insight systems) and other generally used Dealing tools, such as Excel.

We have also decided to make several of our own internal products available generally. All are available for APL*Plus and Dyalog, but some features may vary between the versions. They include:

Object Manager — This enables the programmer to build a series of system files, comprising APL functions, global variables, screen designs, etc. and group them into modules (which can be grouped into larger modules). A complete range of search, edit, copy, delete functions are provided, including point and edit, version comparison, etc. Interfaces to Word provide a standardised documentation method.

Configuration Management Suite — based around the system file concept described above, this gives version control of each object (functions, variables and groups) enabling them to be managed through a development, test, release, live cycle. Promotion and rejection mechanisms co-ordinate movement between these levels (which can be redefined) and full client configurations. It is also capable of having multiple development input sources, preventing clashes and documenting the dependency of changes. Interfaces to Word for release documentation in a standard format. Designed to be used with a compression tool, such as PKZIP to give a series of documented modules for upgrades or patches which can be placed on a bulletin board or despatched to the client via modem or diskette.

Multi-Processor Sequence Scheduler. This enables a systems designed to define a series of tasks to be run on any number of PCs and have the activity interlocked and automated as much as possible. Special hardware requirements can be associated with a PC and with a task, ensuring the task gets run with, say, the correct peripherals (e.g. some backup device). The process fully allows for restarts and checkpoints, reprinting of report images, etc. and is very fault tolerant. The task activity is co-ordinated ultimately by file requirements and this is used to drive a task dependency list — a task cannot start until the files it requires have been produced and are available, even though they have been created on a completely different PC. Individual tasks may be assigned different priorities, including "optional", i.e. the failure of the task is not critical and processing may continue. Once started a sequence can cycle indefinitely, initiating or "sleeping" according to parameters, reducing operator intervention to practically nil — one client described it as "the nearest thing to perpetual motion" he had ever seen.

Process Monitor — a central process, running on a single PC capable of monitoring the activity across a network. It can check the status of devices such as printers, monitor errors on user workstations (with the suitable error trapping

routines running), broadcast messages (or even code to execute on a user's PC) and provides a graphical monitor of tasks being run via the Scheduler. If any of these tasks fail the APL*Plus version can "take control" of the failed APL session, enabling remote editing of functions and reassignment of variables — in fact about anything you can do directly at the keyboard.

Since we are just in the process of launching these tools as separate products, we haven't decided on a pricing structure yet, but expect the price to be low as development costs have been met largely by sales of the 4XTRA system, which makes extensive use of them.

We are also expanding the general consultancy side of the business, which has languished a bit of late due the development work undertaken on the major products. It is after all where we started 15 years ago! this will be not only APL (and other languages) design and programming, but the project management skills and methodologies that we have developed over the years to implement and maintain our projects.

THE EDUCATION VECTOR

October 1996

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.13 No.2. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Sylvia Camacho, 11 Auburn Road, Redland, BRISTOL, BS6 6LS. Tel: 0117-973 0036.

Contents

Editorial	Ian Clark	26
The Hidden Variable Approach?	Walter Spunde	27
Functional Programming in J (2a)	Howard Peelle	29
J-ottings 11	Norman Thomson	37

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 01388-527190

Email: 100021.3073@compuserve.com

Editorial

Many times over the past few years the *New Scientist* magazine has deplored the quality of science knowledge in Parliament and Central Government. Perhaps the fashionable 'good schools' beloved of the establishment are mostly incapable of teaching numerate disciplines to any pupils who aren't well-grounded in them already? This tempts one to suggest that any school identified as 'elite' should reserve a sizeable proportion of its places for pupils with existing numerate skills, otherwise the only qualification for rubbing shoulders with the high and mighty of tomorrow will continue to be parental ability to pay.

In Education VECTOR, we have constantly agitated for the provision of good materials in all schools to teach Science and Mathematics. Sufficient computers are simply a question of money, training to use the computers is largely a question of buying the right hardware and software in the first place, and we believe we have the software answer in APL and J. But how does one encourage those magic individuals who can actually teach Mathematics to non-mathematicians? The best we have been able to do is agitate against the massive discouragement that both good and bad teachers have had to endure in recent years.

But there's another factor, equally vital. I remember reading about an American study of the effects of giving high quality education in Mathematics (among other things) to children from deprived backgrounds. It reinforced my suspicions that the intellectual apparatus of a rich child is not necessarily any better than that of a poor child, provided the latter hasn't actually been brain-damaged by deprivation. But it laid down certain essential conditions for a child to benefit. Chief of which was "freedom from inappropriate disapproval". This of course can come from peers, or family, or even the authorities ("This isn't for the likes of you").

That's perhaps the heart of the matter in Great Britain today. Maybe it always has been. We have a culture which is hostile to any sort of 'cultured' ability, Music, Mathematics, Pure Science, whether on the playing fields of Eton or the street corners of Brixton. Whilst this culture pervades, the able pupil learns to hide his or her light under a bushel and the able teacher learns not to cast pearls before swine. Even with a deliberate programme to suppress it, it may take a generation before the practice of Thuggee, both institutionalised and informal, passes away. Only then will 'good' schools flourish, 'good' in our special sense of teaching good Science and good Mathematics to all who want it.

The Hidden Variable Approach?

From: Walter Spunde (spunde@usq.edu.au)

March 1996

In *J-ottings 8* Norman Thomson provided a different view of the uniform partition function. The approach scales up a division of (0,1) to the width of the given interval and moves it to the correct starting point. Writing Norman's function a little differently, but assuming the same syntax "*n* sample *a,b*" for a partition of the interval (*a,b*) into *n* sub-divisions:

<code>start_point_a =. {.@]</code>	NB. "a" in n sample a,b
<code>indices =. i.@ >:</code>	NB. 0 1 2 3 4 5 ... n
<code>sample_01 =. %~ indices</code>	NB. 0, 1/n, 2/n, 3/n, ..., 1
<code>n_width =. -/</code>	NB. a - b
<code>scaled_sample =. (* sample_01)~ n_width</code>	NB. (a-b)* (0, 1/n, 2/n, ..., 1)
<code>sample =. start_point_a - scaled_sample</code>	NB. a + (b-a)*(0, 1/n, 2/n, ..., 1)

The fixed form of sample is `{.@] - (( * (%~ i.@>:))~ -/)` and unfortunately has two more characters than Norman's function, but it has only one reference to `] .` On my machine it ran marginally faster for small samples, and marginally slower for larger samples.

It is a novel and an exhilarating experience to write a function like this, but note what has to be remembered:

1. functions `start_point_a` and `scaled_sample` must have the same dyadic syntax as `sample` to fit the final fork
2. `indices` is monadic, BECAUSE it will be used as second verb in the monadic hook `sample_01`
3. `sample_01` is monadic because it is used in the hook `scaled_sample`
4. `scaled_sample` is a dyadic hook and `n_width` will be applied monadically.

I think the point I was trying to make was that any pupil in junior high school will be able to see how `a + ((b-a)%n ) * indices` or `a + (b-a) * indices%n` works. They might have more trouble with the hidden variable approach or they may enjoy it, but their teachers will probably have a pink fit just thinking about it.

The "go-get-the-variable" approach is bound to have repetitions, because with tacit definition there is nowhere to store data. Incidentally, if one notes that the sample points are given by `((1 - sample_01) * a) + sample_01 * b`,

then we have a couple more symbols in (+/ . * (-. ,: +)@(%~ i.@>:))~ and 30% slower execution time, but no square brackets at all. The original {. @] + ({). @] - {. @]) % []) */ (i.@>: @ [] with the most symbols and [and] references is slowest of all for small samples, but faster than all of them on my machine when n is large.

Execution times:

n	T	R	S	M
113	0.0297	0.0307	0.0374	0.0346
913	0.0758	0.0692	0.0697	0.0984
3917	0.256	0.2224	0.2126	0.3405
9919	0.6124	0.5223	0.4455	0.8327

where:

$T = \{. @] - ((* (%~ i.@>:))~ - /)$

$R = \{. @] + i.@>: @ [\% (% - / @ -)$

$S = \{. @] + ({). @] - {. @]) \% []) */ (i.@>: @ []$

$M = (+/ . * (-. ,: +)@(%~ i.@>:))~$

Introduction to Functional Programming in J (Part II)

by Howard A. Peelle (hapeelle@educ.umass.edu)

This is Part II of a four-part article. Part I (in Vector 13.1) presented fundamentals of J in a framework of functional programming strategies, with examples expressly aimed toward developing an illustrative program to generate prime numbers. Part II (here) demonstrates how to develop the program directly, while characterizing a J programmer's thinking. Part III (next issue) will continue development of the same program with higher-level strategies. Part IV will consider alternative algorithms for iterative, recursive, and array-processing programs and will end with meta-strategies for packaging and documentation.

Program Development Review

As a review of Part I, here we will develop a program for generating prime numbers, from the beginning, in bottom-up style. We will also characterize a J programmer's thinking along the way, and the reader may notice the fundamental strategies involved (from Part I) which are noted in the text like this [0]:

- [0] Think Arrays
- [1] Select Primitive Functions
- [2] Select Primitive Operators
- [3] Use Parallel Processing
- [4] Name Utility Functions and Variables
- [5] Compose Functions (Define Programs).

The programming task is to generate a list of primes up to (and including) a given integer. To begin with, the programmer might well think: I want to use arrays — especially since the result should be a list of integers. [0] And: what primitive functions (verbs) does J offer which might be immediately useful? [1]

The verb `Integers` (`i.`) generates integers from 0 to $n-1$ for an input n . This is a reasonable first step in computing prime numbers; that is, I know that primes are somewhere in the list of positive integers. But I want positive integers up to and including n . It is easy enough to use the primitive verb `Increment` (`>:`) to add 1 to each number, as in `>: 1. 12` which produces integers from 1 to 12. [3]

Or, I can use the utility verb: [4]

```
Pos =. >: 0 i.
```

NB. Pos is Increment Atop Integers

For example:

```
Pos 12
```

NB. Positive integers 1 to 12

```
1 2 3 4 5 6 7 8 9 10 11 12
```

This list will be useful as a test case. So I'll name it: [4]

```
p =: Pos 12
```

NB. p is 1 2 3 4 5 6 7 8 9 10 11 12

Now the question is how to detect which of these integers are prime. (A prime number is defined mathematically as a positive integer greater than 1 which is divisible only by itself and 1). A useful primitive verb — Remainder (!) — comes to mind because 'divisible' means a remainder of 0. [1] And, looking ahead, I also expect to use the primitive verbs Equal (=) and Zero (0:). [1]

Right away, I can compute a list of remainders all at once using J's built-in parallel processing: [3]

```
p | 12
```

NB. (1 2 3 ... 12) Remainder 12

```
0 0 0 0 2 0 5 4 3 2 1 0
```

NB. 0s indicate which divide evenly

And identify the zero remainders: [3]

```
(p | 12) = 0
```

NB. Or 0 = p | 12

```
1 1 1 1 0 1 0 0 0 0 0 1
```

NB. 1 means true (divisible)

NB. 0 means false (not divisible)

Now I'll define a program for this by composing functions in a "fork" (where three verbs in sequence mean that the middle verb uses results of the left verb and the right verb, as introduced in Part I): [5]

```
Divides =. | = 0:
```

NB. Divides is Remainder Equal Zero

NB. where 0: returns 0

So, from now on I can use its name: [4]

```
p Divides 12
```

```
1 1 1 1 0 1 0 0 0 0 0 1
```

NB. 1s mark divisible items

Next, I want to get the actual integers — the "divisors" — which are marked with 1s. Recall the utility verb If (from Part I): [4]

If =. #~

NB. If is Copy-Commute

Use it to select the divisors from p:

p	If p Divides 12	NB. 1 2 3 4 5 6 7 8 9 10 11 12 If
1 2 3 4 6 12		NB. 1 1 1 1 0 1 0 0 0 0 0 1
		NB. Divisors

Compose the verbs involved (including Pos which produced p) and give the program a name: [5]

Divisors =. Pos If Pos Divides]	NB. Divisors is Pos If
	NB. (Pos Divides Input)
	NB. where] is the input

This program computes the divisors of any given input integer. Examples:

Divisors 12	NB. Divisors of 12
1 2 3 4 6 12	
Divisors 13	NB. Divisors of 13
1 13	

Now I can use this to detect a prime number. Recognizing that a prime has exactly two divisors, I only need to count the divisors! [0] This suggests the primitive verb Tally (# used with one input) which counts the number of items. [1] For example:

#Divisors 12	NB. Tally Divisors of 12
6	
#Divisors 13	NB. Tally Divisors of 13
2	

Next, ask if these results are equal to two: [1]

(#Divisors 12) = 2	NB. (Tally Divisors 12) Equal 2?
0	NB. No
(#Divisors 13) = 2	NB. (Tally Divisors 13) Equal 2?
1	NB. Yes

Compose verbs and name a program for this: [5]

Prime =. #0Divisors = 2:	NB. Prime is
	NB. (Tally Atop Divisors)
	NB. Equal Two
	NB. where 2: returns 2

Examples of its use to detect primes:

Prime 12	NB. Is 12 prime?
0	NB. No
Prime 13	NB. Is 13 prime?
1	NB. Yes

Finally, I want to apply this to a list of positive integers, as in Prime p — expecting a list of 0s and 1s which can be used to select primes, as in p If Prime p. So I attempt the final program:

Primes =. Pos If Prime@Pos	NB. Primes is
	NB. Pos If (Prime Atop Pos)

While this initial program development seems correct, there is a subtle bug in the program which will be rectified in the next section (numbered 6 to continue from the strategies presented in Part I).

6. Test and Debug Programs

After the initial development of a program, the critical next strategy is to test it and debug it if it doesn't work properly. This includes trying not only typical inputs but also special cases. (Other debugging strategies are not discussed in this article.)

First, test the Primes program for a typical input:

Primes 12	NB. Primes to 12
out of memory	NB. Error message

J reports an error message here indicating that the computation requested cannot be done. Actually, Primes 12 doesn't work because subprogram Prime was designed for a single integer input but, instead, received a list (Pos 12). That caused subprogram Divisors, its subprogram Pos and Integers (1.) to fail unintentionally because the input list produced a large 12-dimensional array (see end of section 8) beyond the computer's capacity.

In order to correct this bug, use the Rank operator (") to force Prime to apply to each item in the input separately. [2] For example:

Prime"0 p	NB. (Prime Rank 0) 1 2 3 ... 12
0 1 1 0 1 0 1 0 0 0 1 0	NB. Results for each item

Rank (") is a "conjunction" operator which requires a verb (on the left) and an integer (on the right) which indicates the rank of the items to which the verb will apply. Rank 0 refers to single values; rank 1 refers to lists; rank 2 refers to tables; etc. Here Prime"0 applies the verb Prime to each rank-0 integer in the input list and produces a 0 or 1 for each. This resulting list of 0s and 1s can be used with utility verb If to select the primes from a list of positive integers. Correct the definition of Primes accordingly:

```
Primes =. Pos If Prime"0@Pos NB. Primes is Pos If
NB. ((Prime Rank 0) Atop Pos)
```

Now test it:

```
Primes 12 NB. Primes to 12
2 3 5 7 11

Primes 13 NB. Primes to 13
2 3 5 7 11 13

Primes 100 NB. Primes to 100
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89
97
```

While this programming task is essentially completed, it is wise to test the program further for special cases. For instance, check if an input of 1 works:

```
Primes 1 NB. Primes to 1
NB. None
```

According to the definition of a prime, 1 itself is not prime. Since there are no primes up to (and including) 1, an empty list is an appropriate result here. In J, an empty list can be entered as follows:

```
i.0 NB. Empty numeric list
'' NB. Empty character list
```

Both of the above entries print a blank line, but the results may be used in other expressions. For instance, try program Primes with an empty input:

```
Primes i.0 NB. Primes in empty list
NB. None
```

Testing a program for empty input is a strategy often practised by J (and APL) programmers as a precaution. Even though the program itself may not be normally used with such special cases, it is more robust if it returns a (correct)

result anyway — instead of an error message — because it may be used in other contexts, perhaps within other programs which don't constrain its use.

Other special cases here include zero (`Primes 0` is empty); a small integer input (`Primes 2` is a list of one item); very large integers (results depend on computer limits); negative integers (`Primes -13` returns primes in reverse order); decimals (domain error); etc.

7. Modularize Programs

A good strategy is to develop programs modularly, that is, create and test each subprogram separately and assemble them into larger programs when needed. An important advantage of this is that any subprogram may be used (as is) at other times in other contexts.

Since the programs developed above are already modularized, here we will just show their structure. First, define a utility verb to display boxes around the parts of a program:

```
Boxed =. 5I:2 @ <
```

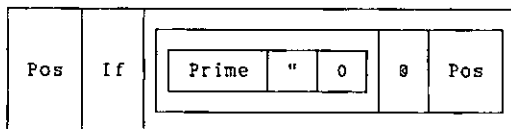
NB. Boxed is 5 Foreign 2 Atop Box

This uses the Foreign conjunction (`!:`) which controls various aspects of the J system, along with the conjunction `Atop` (`@`) and the verb `Box` (`<` with one input). See [Ref 1] for details.

Now the structure of each defined program may be displayed:

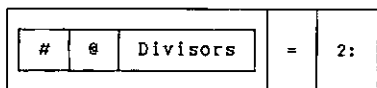
```
Boxed 'Primes'
```

NB. Boxed display of
NB. Primes program

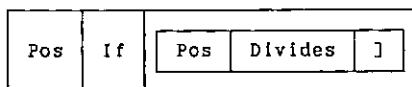


```
Boxed 'Prime'
```

NB. Subprogram Prime

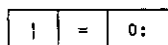


Boxed 'Divisors'



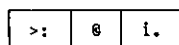
NB. Subprogram Divisors

Boxed 'Divides'



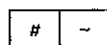
NB. Subprogram Divides

Boxed 'Pos'



NB. Subprogram Pos

Boxed 'If'



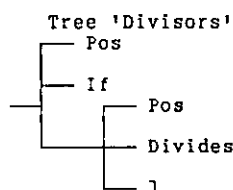
NB. Subprogram If

Other representations are also available in J. For instance:

Tree =. 5! : 4 @ <

NB. Tree is 5 Foreign 4 Atop Box

This displays a tree structure. For example:



NB. Tree display of subprograms

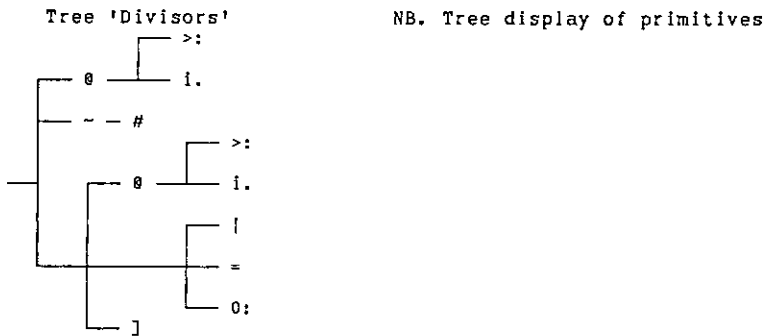
NB. Fork within a fork

J also provides an adverb called Fix (f.) which replaces all subprogram names with their primitive definitions. This compilation is particularly advantageous for efficient execution of a program. (See next section.) For instance:

Divisors =. Divisors f.

NB. Divisors is Divisors Fix

Now its tree structure looks like this:



And its definition is in terms of primitives:

Divisors NB. Definition of Divisors
 @1. #->:@1. (!=0:)] NB. Pos If Pos Divides Input

This linear representation is the default, obtained simply by entering the program name by itself.

Note that you may switch between these representations conveniently via the View menu on a J system.

Howard Peelle
University of Massachusetts
Furcolo #10
Amherst, MA 01003
Tel: (413) 545-1114

Reference

- [1] Iverson, K. E. *J Introduction and Dictionary*, Iverson Software Inc., Toronto 1995

J-ottings 11

by Norman Thomson

In [1], Chris Burke outlines the concepts of frame and cell which underlie J data construction in general and the J rank conjunction in particular. One possible sub-structuring of $b = .1.2\ 3\ 4$ is that of six 1-cells laid out in a 2 by 3 frame thus:

0 1 2 3	4 5 6 7	8 9 10 11
12 13 14 15	16 17 18 19	20 21 22 23 24

The rank conjunction joins a verb and an integer k . Applying ravel joined to various rank conjuncts reveals much about the nature of rank in J, as the following table shows:

frame-size	cell-size	b contains	n= frame-rank	\$. "n b
1.0	2 3 4	1 3-cell	0	2 3 4 1
2	3 4	2 2-cells	1	2 3 4
2 3	4	6 1-cells	2	2 12
2 3 4	1.0	24 0-cells	3	24

Chris did not mention that the integer conjunct of rank can be negative, which allows the application of its verb to be thought of in terms of frames. For example, $._2\ b$ means "apply ravel to the rank-2 frames", which in the case of a rank-3 object is the same as applying it to the 1-cells. For rank-3 objects such as b , a table of equivalences for $._m\ b$ is as follows:

m in $._m\ b$	Shape	Notes
0 $._n$	2 3 4 1	where $n > 2$
1 $._2$	2 3 4	$b = .1.2\ b$ is true for all arrays
2 $._1$	2 12	first row is 1.12
3 0	24	same as APL ravel

Other monadic verbs behave in ways which now become readily predictable. If $y = .1.2\ 2\ 2\ 2$, $+/_2\ y$ is equivalent to $+/_1\ 2\ y$, and means sum down the columns of the 2-cells, of which the first is the matrix

0 1 In full, +/"2 y is 2 4
2 3 10 12

18 20
26 28

+/"_1 y means take the 1-frames, or equivalently the two 3-cells, each of which is of rank 3, and the first of which is

0 1
2 3

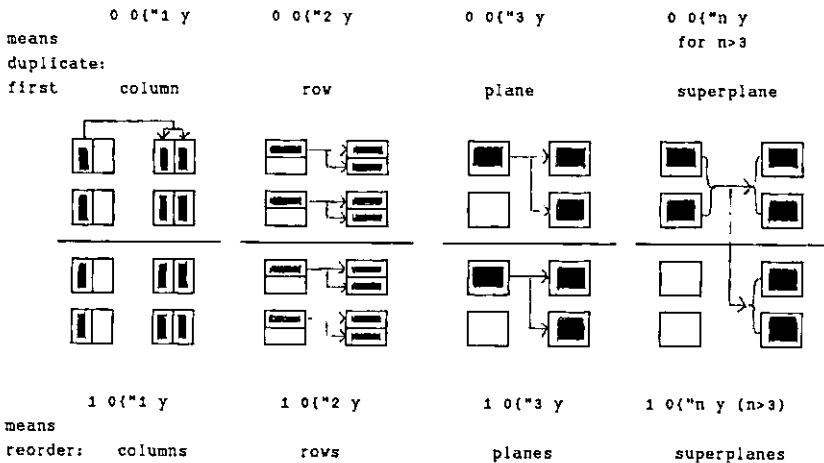
4 5
6 7

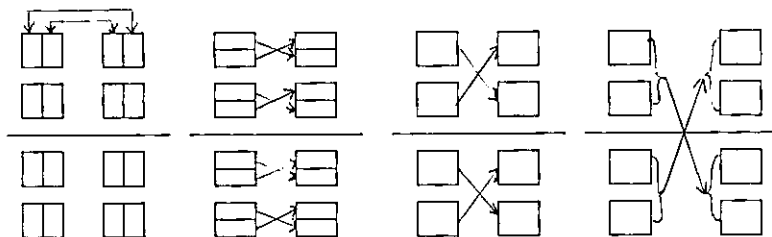
and add along the first axis. The result of this addition for the first 3-cell is

4 6 and the full result of +/"_1 is 4 6
8 10 8 10

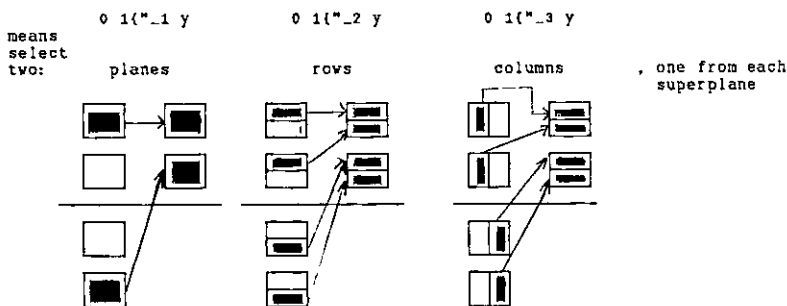
20 22
24 26

Next, consider the rank conjunction applied to from (t). Ringing changes on the indices can be used as a means of replicating or re-ordering the various dimensions. This is described more easily in diagrams than in words. In each sub-diagram the original matrix is shown on the left.





Frame-based indexing is a little different, since it has the effect of rank reduction, and also, unlike the cell-based index vectors which may be of any length, the length of the indices for negative rank conjuncts must now match the length of the first axis.



There are interesting relations with "squad" indexing in zero origin in APL2. These are given in the following table of equivalences:

duplicates first:		
$(\langle 0\ 0\rangle)\ \overline{[3]}\ y$	$0\ 0\{'_0\ y$	columns rows planes superplane
$(\langle 0\ 0\rangle)\ \overline{[2]}\ y$	$0\ 0\{'_1\ y$	
$(\langle 0\ 0\rangle)\ \overline{[1]}\ y$	$0\ 0\{'_2\ y$	
$(\langle 0\ 0\rangle)\ \overline{[0]}\ y$	$0\ 0\{'_3\ y$	
	$0\ 0\{'_n\ y$	

where $n > 3$ including $_$ (infinity)

Note: For the case $\{'_0\ a, 0$ and $(\#a)\$0$ are the only valid indices.

APL2 axis qualification is frame- rather than cell-oriented in the sense that axes are counted from major to minor, whereas cells are reckoned in a bottom-up fashion.

For negative rank conjuncts, the situation is a little different, and is best illustrated by changing the data to an array in which all the dimensions are of different magnitude. Setting $z = .1\ 2\ 3\ 4\ 5$, we have

with the same selection in
each superplane, select two:

$(\>(\ c4\ 5)\rho^{\prime\prime}c2\ 1)\ _0^{\prime\prime}c[1]z$	$2\ 1\{^{\prime\prime}_1\ z$	planes
$(\>(\ c3\ 5)\rho^{\prime\prime}c3\ 3)\ _0^{\prime\prime}c[2]z$	$3\ 3\{^{\prime\prime}_2\ z$	rows
$(\>(\ c3\ 4)\rho^{\prime\prime}c4\ 3)\ _0^{\prime\prime}c[3]z$	$4\ 3\{^{\prime\prime}_3\ z$	columns
z	$0\ 0\{^{\prime\prime}_n\ z$	

where $n < 3$ including $_0$ is equivalent to
 $0\ 0\{^{\prime\prime}_0$ - see note below previous table.

The indices 2 1, 3 3 and 4 3 above are arbitrary, but they must be valid so as not to create index errors. It looks as though the above series might be extended upwards so that for example $(\>(\ c4\ 5)\rho^{\prime\prime}c1\ 0\ 1)\ _0^{\prime\prime}c[0]z$, which constructs a single super-plane with a mixture of planes (1 0 1) from each of the superplanes of z , might have an equivalent using $\{^{\prime\prime}_0$, but this is not the case. To achieve this in J, it is necessary to index superplanes explicitly using a more general scattering indexing technique:

$(1\ 0\ 1, \&. > 0\ 1\ 2)\{^{\prime\prime}_4\ z$

Reference

- [1] *APL and J(1) - Function Rank*, Chris Burke, Vector Vol.13, No. 1 pp.96-101

Product Guide — Updates

compiled by Gill Smith

VECTOR's exclusive *Product Guide* aims to provide readers with useful information about sources of APL hardware, software and services. We also welcome information on APL clubs and groups throughout the world.

Getting Ready for the Web

The Vector Web site already contains a full upload of the existing product guide, from Vector 13.1, with the updates shown in the following few pages. We want to get the information as current and complete as we can, and it would be good to maintain the on-line copy as soon as new information becomes available. To prepare for this, we shall shortly be mailing everyone on the address list asking for updates and confirmation of the current entry. If you don't reply, we will assume you no longer exist, and your entry will be removed.

Updating your Entry

You will soon be able to mail in your updates using the form on the web site, but you can still send material by traditional email or by post to the address below.

Gill Smith
Brook House
Gilling East
York YO6 4JJ

Tel: +44 (0) 1439-788385
Email: 100331.644@compuserve.com

Product Guide Updates to 29th Sept 1996

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
Beautiful Systems	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KSS) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. I have been working in APL for 27 years.
AUSCAN Software	Consultancy	poa	APL software development, training
Entropy Software Ltd	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.

CHANGED & NEW ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
Adaytum Software	Douglas Rowley	13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
AUSCAN Software Ltd	Richard Procter	8 Springmount Ave, Toronto, Ontario M6H 2Y4 Canada Tel: +1-416-651-4037 Email: rjp@interlog.com Web: http://www.interlog.com/~rjp/auscan/

Entropy Software Ltd	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY UK Tel: 01442-878065 Email: 100412.1305@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	5 Thorsden Court, Guildford Road, Woking, Surrey, GU22 7QS Tel: 01483 771028 Email: bob@khamsln.demon.co.uk
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE UK. Tel: 0181-763 2490 Fax: 0181-763 2491 Email: 100551.1401@compuserve.com
Toronto S/G	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: rjp@interlog.com www.slgapl.mtnlake.com/slgapl/welcome.html

COMPLETE ADDRESS LIST

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel:+1 (212) 626-0606 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX, UK. Tel: 01245-495647; Fax: 01245-495646.
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@lbm.net
Adaytum Software	Douglas Rowley	13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel +31-3474-2337 Fax: +31-3474-2342
Andrews	Dr Anne D Wilson	23, The Green, Acorn, YORK YO2 5LL, UK Tel: 01904-792670
APL-385	Adrian Smith	Brook House, Gilling East, York YO6 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: +1 (415) 928-2058 Email: frgp21a@prodigy.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnengeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Club Germany	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332, 1461
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108
APL Interest Group, South Africa	Mike Montgomery	Private Bag X11, Rivonia 2128, South Africa Tel: +27 (11) 803-7200 Fax: +27 (11) 803-9134 Email: mikemont@spl.co.za
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS UK. Tel: 01225-462602
APL Solutions Inc	Eric Landau	1107 Dale Drive Silver Spring MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: +33 1 69-08-79-63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
AUSCAN Software Ltd	Richard Procter	8 Springmount Ave. Toronto Ontario M6H 2Y4 Canada Tel: +1-416-651-4037 Email: rjp@interlog.com Web: http://www.interlog.com/~rjp/auscan/
BACUS	Joseph de Kerf	Rooienberg 72, B-2570 Outfel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 889-2838; Fax: +1 (215) 886-4888
The Bloomsbury Software Co Ltd	Peter Day	3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481; Fax: 0171-436 0524; CompuServe: 100010,1467
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730036, email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM

Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874897 Email: 100430.740@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK Tel: 0171-404 5401. Compuserve: 100343,3210
Causeway Graphical Systems Ltd	Adrian Smith, Duncan Pearson	5 The Maltings, Castlegate, MALTON, North Yorks YO17 0DP UK Tel: 01653-696760 Fax: 01653-697719 Compuserve: 100265,1564
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 USA C'serve:73040,3032
Cinerea AB	Rolf Kornemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652
ComLog Software	Jeff Pedneau	PO Box 5570, Derwood, MD 20855 USA Tel: +1 (301) 990-7063 Email: Jeff@softmed.com
CPCUG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
David Crossley		187 Le Tour du Pont, Quartier Le Moure, 84210 ST DIDIER, France Tel: +33 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: +46 31-45 97 40. Fax: +46 31-45 24 23.
Peter Cylrax Systems	Peter Cylrax	22 Hereford Road, London W2 4AA UK. Tel: 0171-229 5344
Danish User Group	Per Gjerlov	Email: gjerlov@ibm.net
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX UK. Tel: 01604-760241
Dinosoft OY	Pertti Kallojärvi	Lönnroinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 0 70026825 Fax: +358 0 70028824
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP UK Tel: 0181-520 6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 3474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS UK. Tel: 01689-824741
Entropy Software Ltd	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY UK Tel: 01442-878065 Email: 100412,1305@compuserve.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS UK. Tel: 0181-964 9537
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-5290 Fax: +1 (519) 439-5381 Email: info@godin.on.ca
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Mallard F465, Springfield, MA 01111 USA Tel: +1 (413) 788-8411x2838
H.M.W.Trading Systems Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	5 Thorsden Court, Guildford Road, Woking, Surrey, GU22 7QS Tel: 01483 771028 Email: bob@khamsin.demon.co.uk
HRH Systems	Dick Holt	3902 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Email: dick.holt@aol.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-527190. Compuserve: 100021,3073
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reutermail (Sharp): ACAM
	J C Business Services (for pre-paid orders only)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-825181
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (=2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2

Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD UK Tel: 01732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulenlin Lane, St. Petersburg 191186 Russia. Tel:+7 812 312-2673 Fax:+7 812 311-2184 Email: a/m@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel:+45 42 10 70 22 Fax: +45 42 10 75 74 Email: insight@inet.unl-c.dk
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 0171-265 1120
Interprocess Systems Inc.	Stella Chamberlain	11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: +1 (404) 410-1700. Fax: +1 (404) 410-1773 Cserve: 70373,2676
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559
J Austria	Joachim Hoffmann	Münzgrabenstr. 68, A-8010 Graz, Austria. Tel: +43 (0)316 814529 Fax: +43 (0)316 816683 Email: JoHo@magnet.at
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1995 Fax: +1 (905) 831-5172
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessemer Drive, Stevenage, Herts SG1 2DX UK. Tel: 01438-310155 Fax: 01438-310131
Lingo Allegro USA Inc.	Victoria H. Fil	113 McHenry Road, Suite 161, Buffalo Grove, IL 60089 USA Tel:+1 (708) 459-7529 Fax:+1 (708) 459-8501 Email: info@llingo.com
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voice: 01426 98 3585 Compuserve: 100010,33
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel and Fax: +64 (4) 232-4440 Email: 100242.2535@compuserve.com
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia Tel: +61 3 586 7574 Fax: +61 3 586 7600 Email: hld@dar.csiro.au
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8866 Fax: 0171-928 1006 Email: MicroAPL@microapi.demon.co.uk
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants UK. Tel: 01730-283843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Oasis Systems	Theo Zwart	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: 100447.431@compuserve.com
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE UK. Tel: 0181-763 2490 Fax: 0181-763 2491 Email: 100551.1401@compuserve.com
Potomac APL SIG	John Martin	51 Monroe Street, Plaza East Two, Rockville MA 02850-2421 USA Tel: +1 (301) 762-9372 Fax: +1 (301) 762-9375 Email: jam@acm.org
QB On-Line Systems	Phillip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB UK. Tel: 01276-855890 Fax: 01276-855301
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (212) 864-3078
RE Time Tracker Oy	Richard Eller	PO Box 363, FIN-00101 Helsinki, Finland. Tel: +358-0-400 2777
Rochester APL	Gary Dennis	Soliton Associates, 1100 University Avenue, Rochester, NY 14607 USA Email: gsd@ipselab.tor.soliton.com
The Rochester Group Inc.	Robert Pullman	50 S. Union St., Rochester NY 14607-1828, USA. Tel: +1 (716) 454-4360. Fax: +1 (716) 454-5430
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30244, USA Tel: +1 (770) 972-3755 Email: seapdoc@aol.com
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks HP8 4QH UK.
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eecg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250

Solliton Associates	Laurie Howard	Solliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email:sales@solliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Email:askom@apl2.obninsk.su
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (812) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rswain@acm.org
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: +1 (817) 577-0165 Compuserve: 73700,2545 Email:staryarus@uniconp.net
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServe: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sydney APLUG	Rob Hodgkinson	PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel:+61 2 257 5313
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: rjp@interlog.com www.sigapl.mtnlake.com/sigapl/welcome.html
Unlware	Eric Lescasse	Tour Neptune, Cedex 20, 92086 Paris la Defense 1, France. Tel: +33 (1) 47-78-78-00. Fax: +33 (1) 40-80-04-11
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd, Whitehaven, Cumbria, CA28 6JD UK. Tel: 01946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL UK Tel: 01203-523359
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (203) 872-7808

WORLD WIDE WEB SITES

ORGANISATION	URL
APL2000	members.aol.com/APL2000
APL-385	www.demon.co.uk/apl385
AUSCAN	www.interlog.com/~rjp/auscan
Causeway	www.causeway.co.uk
COSY (Bob Armstrong)	www.cosy.com
Dyadic Systems Ltd	www.dyadic.com
IBM APL2	www.toro.ab.ibm.com/ap/apl2.html
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch
MicroAPL Ltd	www.microapl.co.uk
SigAPL	www.acm.org/sigapl
Rex Swain	www.pcnet.com/~rthswain
Toronto SIG	www.sigapl.mtnlake.com/sigapl/welcome.html
Unlware	www.unlware.fr
Jim Weigang	www.chilton.com/~jimw

FTP SITES

ORGANISATION	DOMAIN NAME
IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/tparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

RECENT MEETINGS

J User Conference APL96 at Lancaster

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

The APL96 pictures are by Ray Cannon and Dieter Lattermann. Also thanks to everyone who helped to document the sessions and workshops.

There are a lot more pictures (including several of the night out at Thurnham Mill) on the COSY web site at www.cosy.com. Bob Armstrong did excellent work with his digital camera, and this collection is well worth a visit.

The First International J User Conference Toronto, June 24th - 25th 1996

notes by Duncan Pearson

Background

The first J User Conference was held in Toronto on the 24th and 25th June. It was very well attended, with 123 people filling the Pharmacy Management Building of the University of Toronto, and also very well organised by Strand Software, the sole distributor of J. Everyone that I met at the conference was friendly and interested to know what I was doing, as well as willing to discuss his own work.

The barbecue on the Monday evening on Algonquin Island was particularly enjoyable in this respect, as with the food, wine and beer everyone relaxed and mingled freely. The university accommodation where we stayed was comfortable and modern, and only a short walk from the conference building; the whole university being very central in Toronto itself. The area round the university is a marvellous mix of coffee, book, food and computer shops with one thing I hadn't seen before — something rather like a laundromat that instead of having washing machines had computers, I presume for the students' theses. All in all it was a most enjoyable two days and I will certainly plan to return for the second conference at the same time next year.

The conference itself consisted of two plenary talks followed by parallel talks in two streams. The first plenary, which also served as an introduction to the conference, was by Eric Iverson and during it I was unfortunately busy compiling the final versions of the code for our demonstration in the second plenary, during which I was obviously again too busy to take notes. Of the remainder of the conference I attended mostly the talks about the use of J in conjunction with other software rather than those on the more esoteric aspects of the language, so this account of the conference may be a little one-sided. I have picked three talks that particularly interested me.

Data Rummaging in J — Joey Tuttle

Joey's business is something that I didn't know existed till the conference. He is a "junk fax" distributor. For clients that have a fax to send identically to a large mailing list his company will handle the distribution. This includes the resending

of failures, the logging of unobtainable numbers, the removal of duplicate addresses and, very importantly to them, the correct billing of the client.

The general data analysis is done in J acting on flat ascii files produced by the various automated stages of the process. The disregard of J for the size of the data, its consistent handling of the empty case and its ease of development make it the ideal tool for the job. One thing that came across in the talk was the variety of different formats in which the data arrived and the way in which those formats changed, often with little notice, necessitating a change to the analysis software. The other was the sheer size of the data. Part of the analysis was of an itemised 'phone bill the paper version of which ran to 20,000 pages. However the most instructive thing was the choice of simple format ascii files for the data communication between processes as the most flexible and reliable method. This was reflected later on in Bjorn Helgason's talk.

Use of J to Manage Large Computer Networks — Bjorn Helgason

This talk was interesting to me for two reasons. Firstly it reinforced Joey Tuttle's message about the use of simple ascii files for communication between asynchronous processes and secondly it gave advice about the development of a complex system using J. Interestingly Bjorn's presentation package was not Powerpoint, but a J program written with the Window driver, reminiscent of David Piper's presentation at the BAA GUI development day.

The essence of his approach to network management was that everything that happens on the network should be logged in an ascii file in a specific place on the network — the design of the directory structure is obviously important here — and that J programs should analyse these files. The logs contain meaningful keywords, another thing central to Joey Tuttle's approach, and the J programs' analysis of the files is based on these keywords. The J programs are MAPI enabled and if necessary can send mail to alert administrators to non-standard situations. Again following the principle of simplicity in communication, the only protocols used are TCP/IP and FTP, the FTP scripts being generated and run automatically by J programs.

The J of the system is, in contrast to most published J, very simple. It needs to be maintained not only by Bjorn but also by others with only a rudimentary knowledge. Control structures and a consistent naming scheme are used for clarity with all global variables used in a script defined first in the same way as defined constants in a C source file. In general Windows NT is used, in preference to Windows 3.1 or Windows 95, for its greater reliability. A dedicated machine is used for each discrete task, in Bjorn's words "cheap, stand-alone and

left alone". Synchronisation of processes on different machines is done by access to a central control file (as with Clifford Reiter's fractal and Roger Hui's prime calculation).

In addition to the subject matter of the talk Bjorn also gave us the benefit of his practical experience in developing J, tips on how best to use the development environment and on the structure of large quantities of code in scripts. To me, a relative tyro, these were useful sidelights.

Using J with ODBC and Delphi – John Baker

The title of this talk was slightly misleading as it was nearly all about the OLE interface to J run from Delphi. The simplicity, high capacity and reliability of this was undoubtedly something with which John was clearly very pleased. As he said "What a concept — software that works". He demonstrated a simple use of it in the prime factorisation of the first 1000 integers (in J the verb `q:`) and the display of the results in a Delphi table control. In the resulting timings only 5% of the overall time was spent in communication, while 85% was spent by Delphi in populating the table.

The code showed the simplicity of the interface. In C the programmer is exposed to the complexity of the variant data structure that is used to communicate in OLE. In Delphi the interface to OLE hides the complexity to a greater extent. However John noted that in the process it loses any J error information. The process of starting a J server and of executing expressions in it was very simple and involved creating an OLE object in the Delphi module and executing its "Do" method with the expression to be executed as argument. The result was an array the dimensions of which could be queried and which could be indexed in the normal way. This is much easier than in C.

John then showed an application using J to calculate sunrise and sunset times. It was very polished and showed something of Delphi's capabilities. Clearly for the J programmer who wishes to ship easily installed applications with fast user interfaces, and who has access to Delphi, this is a combination that deserves serious consideration.

APL96 AT LANCASTER

A Conference Diary

by Ray Cannon

Sunday 28 July

I arrived at Lancaster University at 2:30pm after a long but uneventful drive up from Hampshire. On entering the University Grounds I drove past a red warning triangle with a duck on it. Was this the first Dyalog Demo?

I booked into Lonsdale College and was shown to my room, which was a trifle Spartan but very functional. It was very clean and the decor was in pristine condition, quite different from the rooms in County College. (They looked like they had been lived in by students. The walls bore all the signs of generations of posters held up by BlueTack, and the communal kitchen having suffered several chip-pan fires despite the 6 warning notices above the cooker hob.)



On leaving my room, I met Peter from Germany (all surnames will be withheld to protect the innocent) who was also attending the conference and staying in the room next to mine. We walked down to the George Fox Conference Centre, which took us about 10 minutes. The way was not exactly direct, but was well sign posted. En route we passed through the main square of the university with its bookshops, banks, travel offices and newsagents. We also encountered a

peacock strutting around the open walkways. Knowing how students are, I was surprised to see it had what appeared to be a full set of tail feathers.

When we got within sight of the George Fox building, we met some other APLers and was told that the centre was closed until registration which was not until 5pm. Peter and I decided to drive into Lancaster to see the sights.

Once in Lancaster, we walked round the semi-deserted streets and found ourselves at the tourist information office, where we discovered that we were just in time for the last guided tour of the castle.



**The Front Desk at APL96 manned by Angie, Gary
and John De Lorenzo of ACM**

Peter and I then drove back to the University and registered for APL96. Even though I was late registering (and paid on site) this was a painless and very quick procedure, and I was soon wandering round with my IBM APL-2 conference bag. The next act was to sign up for the workshops and tutorials. The conference had two computer labs (each containing about 20 PCs) where the workshops took place. With about 160 attendees, several of the workshops were over-subscribed 2 or 3 times.

Since I had not booked up for the evening meal, I drove into town to a local Kebab House and had an excellent (if a little greasy) supper.

At 9pm I went (in the pouring rain) to the bar of the Lancaster House Hotel where the Vector Working Group had arranged to meet for a pre-conference briefing and try to see how we could cover as much of the conference as possible.

Monday

Up at 6:30 for an early (luke-warm) shower and down to a full English breakfast of cereal, eggs, bacon, sausage, tomatoes and hash browns. Plus toast with marmalade and tea.

8:45: Initial announcements followed by a very interesting talk by Jack Rudd on *Real-time APL Prototype of a Wide Area Differential GPS System*. The now familiar Global Positioning System can (with the aid of a \$200 civilian hand-held unit) give your position to an order of accuracy of 50-100 metres. The military units give an accuracy of about 16 metres. The talk described how (with the aid of extra ground-based receivers) superb navigational accuracy can be achieved using civilian receivers.

After the mid-morning coffee break, I attended a tutorial on *Advanced Windows Programming with APL and Delphi* given by Eric Lescasse. This covered much of the same ground as his recent article in Vector (Vol.13 no.1, July '96). The idea of mixing APL and Delphi (a Borland product that could be described as "Visual Pascal") has some merits, (speed, good GUI designer, low Delphi programming content), but also several disadvantages (including the difficulty of debugging any "mixed mode" system, having to learn to write Pascal, "pointers", difficulty in putting up multiple forms, hard to set up callback functions in APL triggered from events in Delphi).



Eric Lescasse giving the Delphi Tutorial

Lunch was provided in the main part of the George Fox building, where the vendors had set up their stalls and all the conference notices were displayed. This area provided a very pleasant focal point to the conference.

After lunch was the IBM Vendor Forum. They announced several new features and products including the beta version of APL2 for Windows (NT and Windows 95 only), a full "APL out of a box". Also a new session manager and editor. Enhancements to their TCP/IP AP to include encryption for cross platform shared variables.

The IBM forum was followed by the conference General Welcome. Strange not having the welcome at the start of the conference, but practical, allowing late arrivals not to miss it.

I then attended Robert Bernecky's tutorial on *APEX: The Peak of APL Parallel Performance*. This was well up to Bob's usual very high standard, and included many amusing one liners such as "the only thing more frustrating than writing a thesis, is debugging Windows code". (Bob is currently doing his Master's degree, and writing a compiler for APL: APEX the APL Parallel EXecutor.) Topics covered included an overview of APEX (currently a "program" not a "product"), the performance of compiled APL (including a suggested target of producing code that will run at over 80% of the speed of hand-coded Fortran 77), the rationale of compiling APL, the future of APEX, Implementation of APEX, and APL design issues. I look forward testing out the final product.

Tuesday



Heinz Roggenkemper of SAP

Up at 7am, hot shower, breakfast as per Monday.

8:45: Announcements followed by a talk by Heinz Roggenkemper of SAP (the world's 6th largest software company with annual sales of £1.2 billion) whose paper was entitled *The "Organisational Architect" Project*. It was all about how an outside company had completed a major project from very loose specifications, with a very tight schedule, on time and on budget using APL. "This would not have been possible with any other development environment." The project was started in Nov 1994 and shipped in July 1995 to 2000 customers.

The outside company was Causeway Graphical Systems. Heinz went on to say that about 30 problems were reported, of which 15 were installation problems (see later) and 6 were requests for Windows NT versions. Fewer than 10 errors were reported back to Causeway (snapshots were sent via ISDN from SAP in Germany to Causeway in Yorkshire) with responses returned in 20 minutes on average.

Heinz then went on to say that in the next phase of the project, the APL was being replaced by C++ and J (again written by Causeway), and this is to be shipped to 10,000 customers. Two main reasons were given for the APL (Dyalog) being replaced by C++ and J. The first was to overcome the installation difficulties with Dyalog APL/W. The complex command line needed, along with long path names, can require more than the 128 character limit imposed by Windows 3.1. The second was the lack (at the time) of support for double-byte characters, required by SAP's Japanese customers. The speed improvements were not an issue. Causeway performed the conversion (including a learning curve for both J and C++) in 60 man-days.

After Coffee, I attended a tutorial by Ed Shaw on *EDI: Messaging Standards on a PC*. This tutorial gave a very good introduction into what EDI (Electronic Data Interchange) is and how important it is becoming in the USA (e.g. in 1993 a target was set that by Jan 1 1997, all US government agencies must use EDI for all purchases under \$100,000). Ed described how APL is being used to produce programs that translate data to and from EDI transaction streams. Unfortunately, I have lost the excellent handout that accompanied the tutorial so cannot go into more details here.

After lunch was the Dyadic Vendor Forum. This had two show stoppers. Firstly John Scholes demonstrated an experimental version of their interpreter which supports "Dynamic Functions" (Dyadic's name for their extended Direct Definition.) Not only was the resulting APL "Beautiful" (as one member of the audience stated) but it also ran faster. (After the forum John showed me some benchmarks where for some problems it ran 7 times faster than standard Dyalog APL.)

Peter Donnelly then demonstrated full OLE support with Dyalog APL, via 1 short line of APL. (In the Soliton vendor forum it was described thus "OLE server from Dyadic is a 1st class piece of work".) Peter demonstrated the OLE technology by "mapping" an instance of an OLE object into an instance of a Dyalog Namespace, the objects methods into the function in the namespace, and the Object properties into the data in the namespace. The demonstration ended with some "Duck à l' OLE".

Following the SIG/APL meeting, I attended the workshop *New Life for Legacy Systems* (having been banned from the Designing with Causeway workshop by Adrian). This workshop was hosted by Lynne Shaw and Chris Lincoln. (Its findings were presented in the final session on Thursday afternoon.) The workshop defined what legacy systems are and what to do with them. As

someone who has potentially about 400 legacy systems written in APL on the mainframe, I found the workshop useful.

At the end of the workshop, I met up with Ellis, and with three others (Bob from New York, Bob from Woking and Cape Town, and John from Houston and Yorkshire). Ellis had been doing his homework with the Ordnance Survey map, and found a "PH" (public house) west of the university on the coast by a small harbour. We all piled into Ellis' car and off we went. Bob from NY found the concept of single track roads interesting, especially when you could touch the roadside hedge, from the car on either side.

When we finally got to the harbour, we had a choice of 3 pubs, and selected the nicest looking (i.e., it had the best selection of real ales). After the first round we asked for the menu, and were very disappointed to find out that it was the chef's day off, but the landlord very kindly pointed us to the nearby Thurnham Mill, an hotel and restaurant.

We found the converted mill down a farm track located between the river's tidal estuary and the canal (which in the past provided the water to the mill race). On entering, we knew we were at the right place, by the contents of the top shelf running the full length of the bar: Pure Malt Whisky bottles (I think the landlord said he had £3000 of pure malts in stock).

At about midnight, we were suddenly told that we had better leave quickly, or we would be cut off by the tide! However, by this stage the last thing we could do was anything quickly. So after about 15 minutes, when we were all in Ellis' car (who had refrained from drinking all evening, and was quite enjoying the show), we set off. Only to get 100 yards up the track to a hump in the road, with the sea on the other side.

We were cut off, so back we went to the bar and waited for the tide to drop. We finally got back to Lancaster University at about 2:30 all ready for a good night's sleep.

Wednesday

Up at 7:00 am, two paracetamol, hot shower and a full English breakfast. (I never knew I had it in me, and almost could not keep it there!) This morning's talk was given by Per Gjerløv and was on *The Song of the Whale*. Per showed how students used APL to analyse whale song recordings and used them to recognise individual whales.

After (a very welcome) coffee and 3 orange juices I attended John Scholes' Hands On Workshop on Dyalog Namespaces. This was fine, except, the keyboard of the PC I was using was set up as an Adrian Smith special, which meant that some (or many) of the system function keys had their meaning changed. Worst of all to me was that the ESC key when pressed in a function did a quit, rather than (the default Dyalog action) save and exit. Keyboard aside, this was an excellent introduction to Namespaces.

Over lunch I complained to Adrian about his keyboard, and he went over to the workshop and set all the machines to use the standard Dyalog keyboards.

(Question for Dyadic here. When has the keyboard translation input field in the configure dialogue box been changed from a "drop down list" to a simple input field, between versions 7.1 and 7.2?)

The Soliton Vendor forum followed, but I was still suffering a little from the previous night. I can't remember much of it other than many diagrams full of mnemonics (such as TCP/IP, OCDE MVS, UNIX, SAM, SAX and UNAMIT, I think).

I then attended Timo Laurmaa's hands-on workshop on TCP/IP which used Dyalog APL/W and the AP119/W from Lingo Allegro. (See Vector Vol.12 No.4, April 1996.) With the course notes, we all succeeded in getting TCP/IP connections going and sending messages to one another (including getting APL commands entered on one machine executed on another). An excellent demonstration. Because of over-subscription, this workshop was divided into two shorter sessions which was a pity since we missed trying out some more complex stuff.

After Afternoon Tea, I attended the meeting of APL User Groups, where we all agreed to try to communicate with each other more, particularly via E-Mail and web pages. I announced the arrival of www.vector.org.uk.

Thursday

Up and breakfast as Wednesday. I was very happy to be waylaid en route to the conference centre by a Russian with a bottle of Vodka for me. "The Spirit of International APL Co-operation".

I arrived late for Bo Grave's talk on *Modelling Single Interlocking Systems using Array-Based Logic and APL*. Remembering that APL2000 and Dyalog have just both (fairly recently) introduced C type control structures, I found it quite amusing that one of the reasons that APL was used was because the C

programmers got lost in the complication of the IF-THEN-ELSE nesting needed to solve the more complex problems!



Attendees at Morten Kromberg's ODBC Workshop

After coffee I attended Morten Kromberg's excellent hands-on workshop on ODBC. It was worth coming to the conference for this workshop on its own. I learnt a lot, and felt that the "mysteries" of "Open Data Base Connectivity" had been removed. This was a full day's session, broken for lunch and the APL2000 Vendor Forum.

It was good to see new life being put into APL2000, and the latest version of the product is starting to look very attractive. I can't remember any new announcements beyond those stated in the BAA Vendor Forum in London earlier this year.

After afternoon tea, in the closing session, it was announced that APL98 is likely to be in Rome. This was followed by Chris Lincoln reporting on results of the earlier workshop *New Life For Legacy Systems*. (Overheard: "They are trying to get rid of APL at our site, but we can write new systems faster than they can replace the old ones!")

Then back to my room to change for the APL96 Banquet. Then on to the banquet. We were a bit disappointed by the cheese board, a choice of three cheeses, a poor showing for our international visitors. There are so many good British cheeses to choose from.

After the wine ran out, it was down to the "Winning Post" (the Lonsdale college bar) for a bottle of Newcastle Brown Ale, and at closing time a group of us retired to drink my bottle of Vodka. (I have vague recollections of drinking another bottle of Vodka at Lancaster university some 25 years earlier, but that's another story.)

Friday morning

A slight lie-in before a (late) breakfast. Pack my bag, and round to pick up Walter and his daughter Anna, to give them a lift south to their friends' house in Cheshire.

En route we stopped off in Blackpool, where we indulged in a ride on "The Big One", the world's tallest and fastest roller-coaster. This was followed by a virtual reality roller-coaster simulator, a two-mile walk down the Golden Mile to the Blackpool Tower, and a ride back up the Golden Mile in a horse-drawn carriage. This was followed by a final ride on "The Big One" and back to the car.

After dropping off my passengers, I had an uneventful drive back down to Hampshire. I can't wait for the next conference (New Jersey?). Maybe I can sneak over to the APL2000 conference in Orlando.

Thanks to all the people who worked so hard to make APL96 the best APL conference that I can remember.

TCP/IP – Timo Laurmaa

recorded by Jon Sandles

Items in bold in this workshop were typed by Timo at his screen – non-bold by the participants.

Introduction to Basic TCP/IP Programming Concepts

Timo explained that he was going to be showing us how to apply the TCP/IP protocol to our own environment – he was NOT going to be showing us how to implement a low level TCP/IP protocol – just how to use the existing application level interfaces available. The majority of the workshop concentrated on using API19 from Lingo Allegro under Dyalog APL.

Timo supplied a large number of workspaces which looked at different ways of using API19. We began with the following ...

```
)load start
```

```
You are called Jon
```

```
Your IP address is 148.88.152.202
```

The IP address is the method that TCP/IP uses of identifying my machine. We noted this information down and also noted down Timo's IP address as we were going to be communicating with him soon ...

Timo then set himself up as a server. *TCP* is a cover function that passes calls to the API19 auxiliary processor.

```
SID←TCP 'SOCKET' 'STREAM'
SID
```

```
0
```

This creates a socket – in this case referred to as 0. Sockets are a bit like shared variables. *STREAM* is a type of socket – this particular type is what Timo recommends as it seems to ensure no loss of data.

```
TCP 'BIND' SID 2741 '0.0.0.0'
```

```
0
```

This creates an association with the socket (0) and a port (2741). The port number is any number. But to avoid clashes with other well known applications (ftp, http, telnet) you need to use a number greater than 1024. The final parameter '0.0.0.0' indicates that Timo wanted to be a server.

```
TCP 'LISTEN' SID 5
0
```

This effectively sets up 5 buffers on socket 0 for incoming requests from clients. We can process these when we want. Once the 5 buffers are full new clients will get error messages until we clear some of the buffers.

```
TCP 'ACCEPT' 0
```

Which causes the system to hang and wait for incoming requests from clients ...

```
1 1035 148.88.152.189
```

This was the first person making a connection...how to connect to Timo was documented in the handout - but for the record this is what I did ...

```
SID+TCP 'SOCKET' 'STREAM'
0
TCP 'CONNECT' SID 2000 '148.88.152.100'
TCP 'SEND' SID 0 'A' 'Hello Server'
13
```

After a while some people in the room started to get errors - this was because Timo's 5 buffers were full because he was not reading in data as fast as we were sending it. He showed us how to test if a client was sending requests ...

```
TCP 'SELECT' 0 0 0 1
1 0 0
```

The 1 in the return indicates that there is something pending. Thus we should go back up and issue an ACCEPT as above. This code needs to effectively execute in a loop until you don't want to be a server any more! Timo repeated this loop until he had ACCEPTed all the requests from the workshop (17 PCs). These were effectively associated with sockets 1...17 on the server. The server can then query what state these sockets are in...

```
TCP 'SELECT' (17) 0 0 1
0 0 0 0 1 1 0 0 0 1 0 0 0 0 0 1 0
```

This vector shows the state of all 17 sockets. The 1s indicate that those ports have sent data that can be read. The server can thus read this data

```
TCP 'RECV' 4 0 'A'
Hi, Timo
TCP 'RECV' 5 0 'A'
Hello, Server
```

Once the server has tired of reading from a certain socket it can close it...

```
TCP 'CLOSE' 4
```

The client can check to see if the server has closed its socket with

```
TCP 'RECV' 0 0 'A'
```

which will return an empty vector. The client will then clean up the socket at its end as well...

```
TCP 'CLOSE' SID
```

This ended the introduction to the talk. We had learnt both how to set up a TCP/IP server task and how to communicate with it as a client. We then learnt how the server can read its client requests and how both ends can be disconnected when the communication is complete. Next up was how to apply it ...

Four Real Applications of TCP/IP Communication in APL

1. DyalogAPL to DyalogAPL communication

```
)load tcpserver
```

This workspace uses DyalogAPL's event-driven facilities to set up an APL server available for clients to run lines of APL code. This code is very similar to the shared variable method of communicating with TCP/IP in APL2 (Jim Brown, APL95).

Timo initialised the server

```
2 SERVER2INIT 2742
```

We connected to it as clients ...

```
2742 TALKTO '148.88.152.100'
```

and then requested information about Timo's DyalogAPL workspace ...

```
REQUEST 'APLXQT' 'WA'
```

Indeed quite quickly a form appeared on Timo's screen - someone had realised that you could run any valid APL expression - including those that manipulate the Gui on the server! For example ...

It did not take too long for someone to run

```
REQUEST 'APLXQT' 'EX nl 3'
```

which meant that the server stopped working and this particular part of the workshop came to an abrupt end!

2. A dating server

```
)load tcpbase
```

The basis of the event-driven programming technique that Timo uses is to make a hidden form which handles all the TCP/IP events and calls appropriate functions within APL. This workspace contains all the basic low level functions that you need to implement any TCP/IP client server application. This workspace was documented in Timo's handout. If you are a server you would run ...

```
1 TCPSOCKET 0 2468 'APL'
```

and the client runs

```
0 TCPSOCKET '148.88.152.100' 2468 'HTTP'
```

The application specific code can now be placed into the generic framework created by this workspace. The dating program (and the following 2 programs) are thus copied into this workspace. This showed us that the code is very generic and just needs to be tailored for each application.

```
)copy dating
```

Timo used this workspace to set up a dating service. When the clients were connected they were paired off with another connected client. Our new-found friends would be our opponents in the tic-tac-toe game that followed.

3. Chat

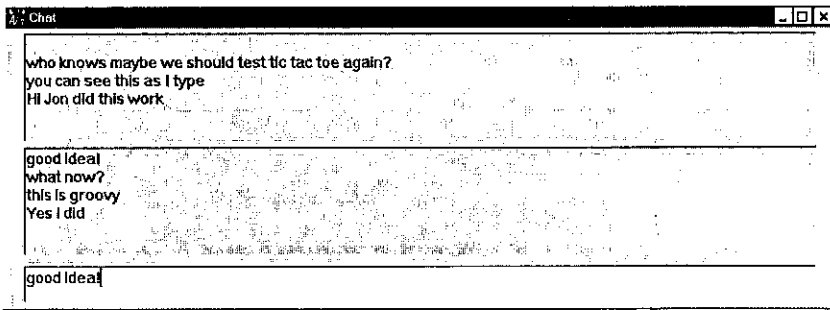
```
)copy chat
```

Timo then set himself up as a chat server

```
Chat '0.0'
```

We could then all send messages via the server

```
Chat '148.88.152.100'
```



4. 2 player 3D Tic-tac-toe

```
)copy tdtts
```

With this we were able to play 3D tic-tac-toe with our new-found friends. You either create yourself as a server or as a client. It depends who wants to start the game. If you weren't sure who you were playing with then

```
Tell 'Who are you?'
```

helped a bit.

Using TCP/IP in Applications on the WWW

As a client - i.e. retrieving info from the WWW ...

```
)load tcpbase
TCP SHARE 0
TCP 'GETHOSTID' 'www.atlanta.olympic.org'
```

returns the ip address

```
)copy web brows  
AltaVista 'APL96'
```

As a server - i.e. creating info on the WWW ...

```
)load tcpbase  
)copy web serv  
  
1TCPSOCKET '' 80 'HTTP'
```

Then in the client's browser

```
http://148.88.152.100/APL 10/3 4 56  
http://148.88.152.100/FORM
```

etc.

ODBC Client Server - Morten Kromberg

recorded by Bronwen Tuffen

Morten Kromberg from Insight Systems ran two workshops on using ODBC with APL. These are notes on the first session which was using APL as the client getting ODBC data from external data sources.

There is a workspace supplied with Dyalog held in the `dyalog/ws` directory called `sqapl.dws` which contains all the functions needed to try these examples. They are a basic set of functions which allow you to connect to, and interact with an external ODBC data source.

SQAINit - This initialises the environment for using ODBC and should always be run before starting a session.

SQACONNECT 'X' 'ODBCSource' - This connects you to the ODBC data source 'ODBCSource' and gives the connection a name of X. You can specify any name for this connection and it will be used for all subsequent communication with the data source.

Once you have connected to your data source there is a variety of things you can do. Firstly you can query it to see what information is available there. There are 2 useful query functions.

SQATables 'X' - This returns all the tables available on the data source, and some information about them. The result is 3 elements, of which 2 seem useful. The first is a return code, the second the actual table information.

0	TABLE_QUALIFIER	TABLE_OWNER	TABLE_NAME	TABLE_TYPE	REMARKS
	c:\Dyalog95\ws\TestODBC		lakes\$	SYSTEM TABLE	
	c:\Dyalog95\ws\TestODBC		People\$	SYSTEM TABLE	
	c:\Dyalog95\ws\TestODBC		SQATST\$	SYSTEM TABLE	
	c:\Dyalog95\ws\TestODBC		lakes	TABLE	
	c:\Dyalog95\ws\TestODBC		People	TABLE	
	c:\Dyalog95\ws\TestODBC		SQATST	TABLE	

SQAColumns 'X' 'People' - This returns all the columns available within a table and their data types, again the result is 3 elements, the first being the return code, the second the data and the third appearing unused.

0	TABLE_QUALIFIER	TABLE_OWNER	TABLE_NAME	COLUMN_NAME	DATA_TYPE
	c:\Dyalog95\ws\TestODBC		lakes	lake	12
	c:\Dyalog95\ws\TestODBC		lakes	distance	8
	c:\Dyalog95\ws\TestODBC		lakes	length	8

continued....

TYPE_NAME	PRECISION	LENGTH	SCALE	RADIX	NULLABLE	REMARKS	ORDINAL
TEXT	255	255	0	0	1		0
NUMBER	15	8	0	10	1		1
NUMBER	15	8	0	10	1		2

Once you know what is available on the data source there are several ways to extract and manipulate this. The simplest way is *SQADo*.

SQADo 'X' 'select * from People' - *SQADo* passes through a SQL statement to the data source. This can be a select statement, or a create/update/insert statement. Used without a left argument it creates a cursor, prepares the statement, executes it and returns the results. It returns a 4 element result: the first is the return code, the second the cursor name used, the third the data retrieved, and the final one is empty.

If the amount of data you are requesting is large you may not want to retrieve it all in one large lump. In that case you can pass a left argument to *SQADo* of 0. This sets up the cursor and the statement as before but only returns the first block of data, you retrieve the subsequent blocks of data with *SQADo* 'Cursorname'.

If you want to do slightly more complicated things or have more control over the process then you can use the lower level SQA functions. These allow you to prepare SQL statements (optionally with parameters in them) and then execute the statement a successive number of times in a more transaction-based way. To prepare a statement, execute it and then close the transaction you would need the following functions:

SQAPrepare 'X.il' 'insert into people values(:x,:y<1:)' - this sets up the statement with a cursor named X.il.

SQAExec 'X.il' 'Joe' 43 - This runs the prepared statement with 'Joe' as parameter x and 43 as parameter y. You would run this for as many lines as you wanted to insert data.

SQATransact 'X' 0 - This is a commit on the transaction. If the ODBC driver you are using supports rollbacks passing in a 1 instead of a 0 rolls back the transaction.

This method can also be used for updating data within a table (optionally the where clause can also be parameterised, e.g.where name=:x:)

The constraining factor on using this code seems mainly to be with what the ODBC driver being used, does or does not support. The Excel one I tested it with did not seem to support parameters and also seemed to return some numbers in an odd format. Also if running in Dyalog 8 it accesses the 32-bit ODBC so you can only use those data sources for which you have a 32-bit ODBC driver defined.

Phil Benkard's Nested Arrays Workshop

recorded by Jon Sandles

This session was a three-hour workshop split into two halves by a coffee break. The first half was well attended and I suspect that those who made it all the way through felt that although they probably didn't get what they came for they got a fascinating insight into what drives any language writer. Phil has obviously been involved in the introduction of nested arrays into APL for many years. His insights in the whys and wherefores leave you with only one question - if they fit in so well then why weren't they in the language in the first place?!



Jon Sandles with
Phil Benkard

This talk should never have been a workshop. Although a few people tinkered at the keyboards, the level of the talk was mainly theoretical and the exploratory workspaces that Phil supplied were too difficult to get to grips with in the time available - more something to play with back home. Many people had come to find out what nested arrays could do for them back in the office - certainly a nested array workshop would be suited for this - but Phil's theoretical direction prevented this being the case.

Phil introduced what the elements of the APL language - **Tokens**, **Syntax**, **Structure** and **Semantics**. Of course APL is famous for its rich quota of single character tokens. Syntax is 'what goes with what'. Semantics are 'where the action is'.

The crux of the discussion was that when APL was extended to encompass nested arrays it was done in such a way that the existing structure of APL was unchanged - also no new functions would need to be defined. (Unlike the concept of BOXED arrays which needed new, arguably more arbitrary, functionality and a new data type.)

Scalar conformance was introduced as a critical concept.

Enclose was introduced. Although a new function it was just a shorthand for existing functionality

`' 'pX X <-->1[]X X<-->(X X)[1]<-->cX`

in APL2`2 2[1]<-->c2`

and in APL\360`2 2[1]<-->2`

and hence`2 <--> c2`

and of course`c2<-->c2. etc etc`

This is the key to scalar conformance. *"Think of a scalar as infinitely enclosed – it's so deep you have lost sight of it – that's what makes scalar conformance work"*

Later on we were introduced to a similar set of identities for each (') and scalar functions (functions that return scalars). This led to Phil introducing a simple set of functions which allow outer products to be expressed in nested notation. The same was also done for complex numbers.

The talk taken as a whole was too unstructured to provide a good introduction to nested arrays. Perhaps some theoretical exercises to be done on the PCs were what it needed to give it a bit more direction.

But every now and again Phil has a way of describing what you thought was complex and making it sound very simple – and this made it very worthwhile and enjoyable.

GUI Programming and Multi-Dimensional Databases at APL'96

notes by Marc Griffiths



Gill Smith, the "Local Arrangements Chair"

Before I describe specific sessions, I want to thank everyone who helped to make APL'96 possible: the organising committee and the conference staff at Lancaster University; the programme committee and the plenary speakers, authors and reviewers; the many people who prepared and led the tutorials and workshops; the sponsoring organisations and vendors who continue to develop the software products; and, finally, to everyone who thought it important enough to attend. Special thanks to Richard Smith and to the university's systems support staff who were instrumental in setting up the software and preparing the personal computing labs for the workshops.

As an application developer, I came away from the conference with a relatively warm feeling. However, I could not help but reflect upon what was not there. First, a significant number of younger delegates. If the age of APL conference attendees continues to reflect the age profile of baby boomers, the big question for vendors will be whether to use an 18-point or a 24-point font size as the default for the installed interpreter. Second, I saw little evidence that APL has been touting its successes in other forums, although I admit that I could easily have missed something here. (I hope that the report about APL and legacy systems succeeds in dispelling the air of a rearguard action that this topic has about it.) Third, and most noticeable for me, was the emphasis on the hardware and software to the detriment of system development approaches and the user as a fundamental part of these systems. Ironically, Lancaster University is where Peter Checkland devised his Soft Systems Methodology (SSM) over 15 years ago. In this, he argues that information systems are fundamentally social systems, and that no amount of clever hardware or software will enable any system to succeed

that has not taken into account the often disparate views of its stakeholders. SSM has subsequently been employed as a framework for participatory design, user-centred design, rapid prototyping and so on, all of which have found proponents in a number of areas of system development, and all of which lend themselves readily to APL and J. Handled correctly, here is an opportunity to (re-)establish the technical and commercial viability of APL and J. *Carpe diem*.

Finally, a few general comments about the workshops. Those I attended focused on the use of features built into APL, either directly or through third-party software. While the 'how' is undoubtedly useful, I would like to improve my understanding and so would welcome more background information ('what' and 'why') about the topic, even if only in the form of some references or a summary of performance characteristics within a variety of environments. This is perhaps more true of ODBC and TCP/IP than of the GUI design workshops, but this is indicative of my greater familiarity with the latter.



One of the Two Workshop Labs used at APL96

From the presenters' point of view, it would help to distribute feedback forms to the attendees and ask that the forms be filled in and returned at the conclusion of the workshop or conference. As many of the workshops deal with one or more interpreters and third-party products, the presenters are likely to receive more pertinent comments than they would in almost any other situation. And a note for future organisers: the sad truth of many computing labs is that they are not designed for presenting material to a large group.

The arrangement of furniture, the acoustics, the lighting and so on, do not support the effective presentation of material to a group, especially when there are two people sharing most machines. It would benefit everyone concerned to ensure that, in addition to adequate hardware and software, the facilities also support overhead or LCD projectors. If not, then the presentations may be delivered more effectively in a room with these facilities, even if it is not adjacent to the lab.

An Introduction to GUI Programming in Dyalog

presented by Peter Donnelly

Peter presented this Monday afternoon session to a group of over 40 people, most of whom sat two to a PC. He apologised that the machines did not have Windows 95, especially as he had prepared the workshop on the basis that Microsoft's newer offering would be available. Nonplussed, he proceeded to introduce the features in Dyalog APL for Windows that support the design of graphical user interfaces (GUIs). As soon as individuals figured out the keyboard mappings, and one or two problematic machines were identified, we were off to the races.

No one appeared to have any difficulty following Peter's clear explanations and illustrations of the main GUI system functions. We created objects and then proceeded to get and set various properties. While we were thus engaged, Peter managed to explain the 'dot' notation and the different coordinate systems without apparently losing anyone. After the first half hour or so, the course of the presentation changed a little according to the questions that people asked. After a brief review, we spent some time experimenting with event handling, callbacks and the event queue. Finally, Peter used the WDESIGN workspace to introduce Dyadic's implementation of namespaces, and to demonstrate a more structured environment in which to develop GUIs. We were then left to construct the application on the back page of the latest Dyalog APL for Windows brochure.

Advanced Features in Dyalog APL for Windows

presented by John Daintree

This late Monday afternoon workshop became a presentation owing to the absence of Windows 95 on the PCs in the lab. And, despite unfavourable lighting for the LCD overhead display, John was able to provide us with a good introduction to several features available in versions 7.2 and 8.0 of Dyalog APL for Windows.

John began by demonstrating the new 'GetEnvironment' method and the monadic use of `⌈WC`, both of which are documented in the online help. He then proceeded

to demonstrate how to use the `greet_bitmap` command-line parameter. For example, under Windows 3.1, if the command line:

```
c:\dyalog72\dyalog72.exe greet_bitmap=c:\dyalog72\myhello.bmp
```

is the command line associated with the Dyalog APL version 7.2 executable, the 'myhello' bitmap is displayed on the screen when the interpreter starts to load and is removed when the session appears. Alternatively, if a workspace is loaded together with the interpreter, the bitmap can be removed under program control through `ⓘNQ`:

```
2 ⓘNQ '.' 'GreetBitmap' 0
```

Peter Donnelly then asked whether anyone had specific requests for advice on how to do something or for new features. The Grid object attracted a lot of attention, especially in terms of having a 'print' property. John pointed out that the Grid had many attachments and parameters, all of which added up to a very complicated general case. More likely to be looked into was a request for improved Grid paging. The final question was for the list of files and the directory structure required to create a run-time Dyalog APL application. Unfortunately, I was able to note down only a partial list, so we must wait until someone provides a complete answer in Vector for each operating system.

An Introduction to Designing with Causeway

presented by Duncan Pearson

The first session on Tuesday afternoon saw about 45 people arrange themselves two to a machine in the lab. There was a slight delay while Adrian Smith fashioned curtains from some bedsheets to reduce the amount of light coming in the windows; for some reason, the overhead projections were even fainter than those during the Dyalog workshops on the previous day. In any event, the improvised solution worked well enough for the session to get underway.

Duncan introduced Causeway by having us construct the 'flipper' application that has almost become the company's unofficial trademark. A few of those who were less familiar with object-oriented nomenclature found the move from operators, functions and variables to methods, classes and properties a bit problematic, but with Duncan taking questions at the front and Adrian supporting people at the back of the room, there was never any danger of leaving anyone behind. At the conclusion of the flipper exercise, Duncan demonstrated how to add a variety of objects — a multi-line edit field, a group of radio buttons, a drop-down listbox — to a form through the Causeway development

environment. As the emphasis in this workshop was on rolling up our sleeves and getting our hands dirty, Duncan distributed a page containing a series of exercises that would eventually yield a simple personnel record management system. Everyone stayed hard at it until tea-time.

Creating and Modifying Classes in Causeway

presented by Duncan Pearson

After tea on Tuesday, about 25 of us re-convened in the PC lab for the second Causeway workshop. This session aimed to explain how individual classes are defined, and to demonstrate how we could customise the default classes that ship with Causeway.



Duncan Pearson & Adrian Smith taking a well-earned rest

Duncan began by explaining that each row of the `Δclasstab` variable contains the methods and properties that the Causeway development environment uses to create a new instance of a specific object. Each class has up to five generic methods (create, refresh, read, resize and design) as well as specific methods for any event-handling. All of the methods are functions stored in `FOR` form. The first step in creating a new class is to find one within `Δclasstab` that contains a class with similar properties, then to concatenate that row to the bottom of matrix. Second, change the first item in this new row to be a unique uppercase text identifier. Third, change the second item in this same row to be a brief,

appropriate text description. The remaining changes involve the use of utility functions in the *maint* namespace to edit the method functions.

The example Duncan used to demonstrate all of this was the creation of a text edit field for password input and that returned an encrypted text string. Starting from the standard text entry class, he took us through the changes to the create method. We were then encouraged to make changes to the refresh method (adding decryption), the read method (adding encryption) and the keystroke event handler, with support from Duncan or Adrian only if necessary.

Multi-dimensional Databases

presented by Phil Chastney

I am continually amazed by the resourcefulness of speakers when things go wrong. Phil had planned to illustrate his tutorial on multi-dimensional databases by demonstrating the system he has been working on at Eurostat. Unfortunately, his departure from Luxembourg had been delayed a few days, and he arrived in Lancaster only to find that he was unable to decompress some of the data files. The solution? Bring some fine art posters, of course. Having been numbed by two-and-a-half days of high technology, the 25 or so of us who filed into the tutorial room on Wednesday afternoon saw an interesting array of low technology equipment: a free-standing notice board with drawing pins, a standard overhead projector with acetates, and a whiteboard. Clearly it was time to put on our lateral thinking caps. Phil's use of the posters to illustrate or emphasise various points in his talk was both entertaining and a refreshing change.

The main objective of the tutorial was to introduce multi-dimensional databases (MDD), and to use the work at Eurostat to illustrate various concepts and problems. Eurostat is responsible for the collection and analysis of various data from countries in the European Union as well as several other non-EU countries such as Iceland. Data collection takes place in Luxembourg, where Phil has been working, and data analysis in Brussels. Essentially, the Eurostat database is an n-cube that must lend itself to two-dimensional slicing along any axis. A large part of the data collection process involves quality assurance. This is a non-trivial matter, given the diverse types of data gathered (industrial production, public transportation facilities, tourism, etc.), and the different ways they are collected, consolidated and reported by each country. Even indicating a missing value is complicated. Phil's experience at Eurostat supports the notion of "sixteen types of nothingness" which is clearly exemplified by the Icelandic railways problem: there are none. So, is this missing data the same as all others? Another challenge is how to deal with data which include classes that are not mutually exclusive.

Phil gave us a potted history of databases before pointing out that, although MDD is a hot topic, it lacks a clear definition. However, almost all articles describing multi-dimensional databases refer to the star model. The main data table lies in the midst of a number of service tables. In practice, this has evolved into a snowflake schema in which the primary service tables need service tables to accommodate all of the ancillary information associated with the data in each dimension. Much of the development work at Eurostat is an attempt to refine the dimensions of the data model, for example, to standardise on a basic reporting period (one month), or to remove from the dimensions various units of measurement (length of period, currency, valuation, etc.) to allow for a change of basis in reporting.

All of this has been implemented in APL*PLUS/PC on networked PCs. The APL constructs sets of SQL commands that are passed to an Oracle database. The performance of the network was highly problematic at the outset, but this has been corrected. The performance of the APL and Oracle DBMS have been good, although Phil recommended doing joins 1k at a time in APL rather than in SQL. The development is being pushed by the IT group in the direction of using ODBC under Windows NT 4.0 and of ISO 9000 compliance.

It was interesting to note that the users, when asked to rank order a list of possible future developments, chose functional improvements over migration to Microsoft Windows. On a more cautionary note, we were advised to select analogies carefully whenever we need to explain some concept. Some of the users did not understand the notion of a z-dimension in the context of viewing 2D slices through a particular dimension. Phil tried to use some file dividers with coloured tabs to explain it, but only succeeded in having the users associate specific colours (from the tabs) with different planes. This resulted in additional development work, as the colours had to be incorporated into the user interface of the next release.

Finally, there were two quotations that I found particularly relevant. On the amount of effort that is going into the overall design of the Eurostat system, and particularly the MDD model, Phil cites the contention of Frederick Brooks, Jr. that "... conceptual integrity is the most important consideration in system design." And, with regard to the major challenge facing anyone designing a multi-dimensional database, Lao Tse offers this: "The wise man ... knows there is no limit to dimensions."

APL96 — A Consultant's View

by Richard Procter (rjp@interlog.com)

APL96 was a great conference for the Array Processing Languages consultant. The variety and depth of practical, hands-on training and information sessions was well beyond the norm, and greatly appreciated by attendees. New developments in APL/J products mean unprecedented opportunities for consultants to apply their skills in many business areas not previously open to array-oriented solutions. Connectivity and compatibility with other software was a big theme at the conference. Fitting APL/J into the rest of the business software world is facilitated by a number of new features and products now or soon to be available. Consultants should be looking to incorporate these emerging technologies into customer solutions in many application areas.

The conference facilitators arranged for two hands-on labs equipped with PCs running Windows (sadly, not Windows 95). Even though only about 150 attended the conference in total, several of the tutorial sessions were so popular, they had to be split into two sessions to accommodate the demand. The hottest sessions involved connectivity between APL and the rest of the world: e.g. TCP/IP and ODBC; and GUI programming techniques: Dyalog, Causeway. In addition to these topics, there was also plenty of traditional, and practical tutorial information for both beginners and experts: sessions on utility functions, enclosed arrays, operators, J programming techniques. Several papers and tutorials also addressed what I call the "academic side" of the APL/J community: neural networks, functional J, genetic algorithms, etc.

Presentations of submitted papers, it was mentioned at conference wrap-up, were generally poorly attended at the conference, due to the fact that these were competing for attention with the highly popular workshop/ tutorials. Future conference organizers will have to be mindful of this dichotomy if the emphasis on workshops continues, which hopefully it will.

I find that the best part of these conferences are the subtle surprises. They sometimes happen, for example, when you attend a session thinking that it has little relevance to your work, and will probably be dull, but there's not much else happening so you go. In fact, that session then turns out to be one of the most fascinating of the conference, and furnishes you with great ideas for future work and study that you never even thought of before. The other great thing of course about conferences is the chance to meet colleagues from around the world and share ideas at the leading edge of APL development and application.

Another theme at the conference was the word "legacy". So-called legacy APL systems are apparently still alive but some are not so well, and many need our help. The suggestion is that new products and features: Windows GUIs, database connectivity, OLE mechanisms, client-server techniques, etc. can and should all be used to spruce up these systems, bring them up to speed with the modern world, and blow any would-be competitive products out of the water, just like APL has been doing all along, some would say. Trouble is, as usual, APL suffers from a historical image problem (slow, weird, unmanageable, whatever...) that is often insurmountable. Eric Baelen of APL2000 made a passionate plea at the conference wrap-up to get out there and "blow the APL horn", trumpet APL success stories from the highest hills, and build on these triumphs. Not a bad idea.

Here are my conference highlights, with details:

Techniques/Workshops

ODBC — stands for Open Database Connectivity, a Microsoft standard for I/O with SQL-type relational databases (Oracle, Access, Dbase, etc.). APL systems have been talking to these centralized databases as a client for some time now, but what's more exciting, APL can now act as an ODBC server, allowing other non-APL systems to talk to *its* database. This important feature will help existing APL systems keep doing what they do best, handle information in array-oriented fashion, yet still talk the same language as the rest of the world. Especially helpful for situations, as workshop presenter Morten Kromberg pointed out: "...when managers ask, 'what's the database written in?' You reply: 'why, it's ODBC of course', ...and they shut up", he says.

TCP/IP — again, a big buzz-acronym in the client/server world, this technology, as session leader Timo Laurmaa puts it, is "... *the* communications protocol to connect computers together", possibly running totally different systems on different platforms, allowing these to communicate across the office (Intranet) or across the world (Internet). Using an auxiliary processor developed by Lingo Allegro, APL can talk to other sessions, send and receive instructions and data, or act as host to other APL or non-APL client inquiries. The tools that were presented in this workshop are the building blocks for all sorts of interesting networking possibilities, locally and world-wide.

GUI Programming — several excellent sessions were devoted to this topic:

First was Eric Lescasse's presentation using Oracle's Delphi, a sort of enhanced Visual Basic-like programming and GUI development environment that has

many powerful features for creating Windows forms and DLLs written in Delphi/Pascal. Linking these features to Dyalog APL afterwards seemed remarkably trivial, meaning that sophisticated and efficient applications can be created by using these tools to their best advantage along with APL left to do what it does best, handling data.

Next, Causeway and Dyalog, a couple of workshops led us step-by-step through the pitfalls and triumphs of putting together a GUI interface and its underlying code. Causeway especially simplifies the task of linking together objects on the GUI form with corresponding data, and makes the whole process of event-driven programming somewhat automatic. Causeway featured highly in one of the plenary session presentations, discussing an application where it was used to rapidly develop a highly sophisticated company business organizational model for the sixth largest software company in the world!



Ed Shaw (APL Group Inc) leading the EDI Tutorial

EDI — this was one of my surprises. I went not knowing a thing about Electronic Data Interchange, and came out of the session thoroughly convinced that here was an application that APL, with its rapid development time and array-handling properties, could excel at.

Ed Shaw of The APL Group, Inc., led the session, pointing out how important EDI is to the US government and business community (use it or you'll soon be out of business!), and did a good job of showing how APL lends itself to what at first seemed to me to be a non-APL application. Definitely something to watch for in future.

J Functional Programming — Toronto's Chris Burke led this fascinating session, explaining in his usual precise and excellently clear manner how the programmer who is still stuck back in APL-think can adopt and embrace functional J techniques by some simple extensions to that mindset. I scribbled four pages of notes, and left thoroughly convinced that J has a great future with its superior approach to array handling. ...now if only somebody would pay me some money to program in it...

Vendor Forums

These sessions were popular and as usual, important for the consultant, bringing us all up to date with recent product releases and new features. Several very important announcements and features were unveiled at APL96:

IBM: announced to an ovation from the audience that finally, it would soon be releasing a version of APL2 for Windows ('95/NT), instead of "that other operating system" OS/2. This is welcome news, as I don't know many people who were willing to put in the time, effort and hardware required to master OS/2, just to do APL. Apparently IBM will be shipping Beta versions of their new product soon, so get yourself on their mailing list if you want to be one of the Beta testers.

Dyalog: wild excitement swept the room when Dyalog demonstrated two stunning new features for its Version 8 product (Windows 95 & NT, and soon to be made backward compatible for Windows 3.1).

The first was "dynamic functions", a concept which extends the notion of assigning snippets of function code to a name, so that small functions can now be defined using something similar to Iverson's original "direct definition" methodology. Using this powerful feature means that writing small, readable functions and applying them, say to the cells of a nested array by using the each operator, now becomes trivial. Dynamic functions bring back the concept of specifying function definitions with implicit arguments (α , ω) and an implicit result, with all variables automatically localized.

Dyalog's second big crowd-pleaser was the announcement of full OLE support, so that programming done in APL can now be packaged up using Namespace methodology, into an "instance" and called from any non-APL task that talks OLE, in a background manner, without having explicitly to start up an APL interpreter. J has had this "background server" or "calculator engine" feature for some time, and now we will soon have it in APL as well. Again, welcome news, with lots of exciting implications for the APL implementor.

APL2000: Eric Baelen presented its Windows product, covering much of the same territory he demonstrated to a Toronto audience in June. Once again, good to see that development and enhancements to this excellent APL environment are ongoing. APL2000 also announced the return of its own annual User Conferences, with the first coming up in Florida this year in October.

APEX: not quite a product so not quite a vendor forum, but Bob Bernecky put on a brave performance to a sceptical and highly critical audience, outlining the latest progress and developments in his APL compiler. Seems that Bob has some distance to go before the naysayers of the APL community are convinced it will fly, but watch out if it ever does, as this product could go far in terms of dealing with those perennial problems that legacy APL system sceptics like to harp about.

Other conference info

J vs APL? — this topic seemed to be a non-issue at APL96. Most people attended J and APL sessions both with enthusiasm. J is being embraced by APL people as a potentially useful array processing language, and seen by some as better than APL, although the majority still argue it requires a lot more careful study to comprehend and use.

APL employment? — again, another non-issue at this conference. Only a couple of jobs were posted on the bulletin board, and I heard almost nobody discussing the issue. I don't believe that means that the issue has gone away, maybe it just took a back seat to other more important matters?

APL SIGs worldwide — a meeting among SIG executives from around the world was convened to discuss a few issues. We agreed to cooperate more fully to share information, be that electronically by posting announcements and newsletters, or in the traditional manner by sending each other newsletters. Some sort of centralization of APL SIG info worldwide was proposed, especially when it comes to speakers and meetings/event schedules. Look for a global SIG calendar online soon. We also discussed the lamentable situation



Professor Annibali announcing the choice of Rome for APL98

that future APL conferences are in trouble due basically to lack of availability of people willing to volunteer their time and skills to help put one on. Apathy is reaching crisis proportions: APL96 was a miracle, considering how much work had to be done by so few people at the last minute. APL97 will be held somewhere on the East coast of United States, and APL98 is set to be hosted by an apparently enthusiastic local group in Rome, Italy. Conference costs are also a big discussion factor: how can we reduce them, boost attendance, and still deliver the goods?

Finally, Lancaster itself proved to be a hit with attendees. The historical centre, with its great selection of friendly pubs and restaurants made a welcome detour from the main conference venue, which was at the university, a short bus or cab ride out in the countryside. Getting past that classically unappetizing English cafeteria breakfast at the uni took some doing, and it was reassuring to see no radical changes to the traditional British "pub grub" on most menus in town.



**Sasha Skomorokhov and Chris Lincoln talking to Dieter Lattermann
(Conference Chairman) and Mike Kent (Treasurer - back view)**

I would like to thank personally the organizers of APL96 for putting on what I think was the best conference ever in terms of excellent presentations and informative workshop and tutorial content that consultants like myself can put to immediate use in day-to-day customer support and future exciting APL/J projects. Let's hope this trend continues.

APL96 Roundup

by Adrian Smith

This felt like a very international gathering, certainly the University Conference Office had never seen anything like it before; the Lancaster House Hotel also coped remarkably well once it had recovered from the initial shock.

Analysis of Delegates by Country

Here are the approximate numbers, taken from delegate list available at end of Conference — see note below. Cast in order of appearance!

Finland	15
USA	36
Spain	1
Italy	3
Germany	16
Canada	10
Netherlands	9
UK	22
Denmark	13
Sweden	7
Belgium	1
France	6
Switzerland	4
New Zealand	1
Japan	4
Russia	4
Ireland	1
Australia	2
Austria	1
Israel	1
Portugal	2



Part of the Finnish Delegation

Note this does not include any "extras" as in accompanying persons. It also has inaccuracies, for example it includes at least one Frenchman who wasn't there. Note too that "nationality" refers to current place of work/accommodation, such that for instance Mironov is recorded under France not Russia.

From my point of view, the thing I was most grateful for was everyone's tolerance over the first day and a half, after which things started to move reasonably smoothly. Much of the impetus for this conference came very late in the day, and there were a good many pieces to pick up on the Saturday and

Sunday morning, particularly when it came to setting up the labs and making sure the right kit was in the right rooms in the George Fox centre. Looking back over a hectic week, I can see a few names missing from the list of people who really ought to be thanked:

- Hilary Barraclough, who runs the University Conference office. She was remarkably forgiving when we decided at the last minute to change the layout of the tutorial rooms, which meant shifting out around 20 tables and moving in the seats to make space for an extra 30 or so delegates. She also managed the campus room-bookings with very little fuss and considerable efficiency.
- John Stewart, the audio-visual technician for the centre. At times we had three rooms on the go, all with high-quality computer projection and microphone sound. He was faced with a wide variety of hardware, and everything from Per Gjerløv's Whale Songs (played directly from his PC) to Willi Hahn's animations (from a portable RS/6000) worked first time. The last time we came close to this quality of AV support was at Canterbury in 1988, when the requirements were a lot simpler.
- Ian and Mark, the lab technicians from the engineering department, who worked late on Sunday to get us off to a solid start on Monday morning, and then came in every evening after the last workshop had finally ended (usually after 6.00) to set things up for the following day. We deliberately left the 'hairy' sessions (Timo and Morten) until the end of the week, by which time the setting-up was almost completely automatic. It was marvellous to set up a single 'master' machine and have it cloned around the room in less than a minute.
- Gary and Angie, who worked the front desk, got things copied, put up notices and managed not to lose their rag even when the lack of planning finally caught up with us and they were faced with a string of bemused delegates and a marked lack of helpful signposts on the Monday morning.
- John and Mrs De Lorenzo, who kept the bookings under control, knew who was entitled to what, and made sure everyone got the right bits and pieces when they arrived.
- Mike Kent, who had had it up to *here* with APL95, but was still willing to pick up the form-filling, look after the budget, make sure bills got paid and Vendors got invoiced, and was probably the person most responsible for the small profit that SigAPL made from the conference.
- The staff at the Lancaster House Hotel, who seem to have had remarkably little problem with the 'evening seminars' which typically disgorged at around 3.00am. I spoke to a good many delegates who were staying at the hotel, and none of them had anything but praise for the welcoming and relaxed atmosphere.

- Dieter Lattermann, who accepted the chairman's role when we lost the original co-chairs back in September, and kept the conference quietly moving forward when it really looked as if it might stall completely. I think it is very likely that without his steadfastness, and the solid backing of the APL Club of Germany, APL96 would not have happened.

Finally, the family Smith, who came along for a pleasant week's holiday and ended up spending most of the time fetching and carrying. Richard did a splendid job on Sunday installing Dyalog APL on around 20 PCs — doing it the hard way from floppy disk! At one point he had a perfect pipeline running — all seven disks installing simultaneously in seven machines. Then we hit one with a slow A: drive Gill managed to talk French to the French and German to the Germans (alas she has yet to learn to converse in Finnish) and generally act as a human signpost and point of reference for a large number of delegates. Tim helped with the signposting, then just kept out of the way, playing Pool with the Baarghorn boys.



The Main Lecture Hall

Conference Highlights

I thought we had a really solid set of plenary sessions, and that the vendor forums were well-attended and very well presented. The stuff on dynamic functions (Dyadic) came as a complete surprise, and looks to be a real step forward in bringing some of the J ideas back into the APL mainstream. For the first time since I-APL, we now have the option of developing our applications

entirely from script files and using the workspace simply as a superb memory manager. I can't wait to try it — my only regret is that I was not wearing my APL88 tee-shirt which John Scholes could have executed $\{\omega \omega \circ . * 0\}$ to obtain an average.

The most memorable moment came in Chris Burke's tutorial on J operators. He worked mostly in Dyalog APL, which has many of the advanced features such as function assignment and a commute operator, and only after demonstrating the APL approach did he reproduce the effect in J. Phil Benkard came in a few minutes late, and sat down looking extremely sceptical — then he began to get seriously interested as he realised that J has been using its weird ascii spelling to conceal a good many ideas that Phil has been trying to get into APL2 for years!

Summarised Results from Dick Holt's Feedback Survey

The table shows the average of non-blank scores on a scale of 1 (most satisfied) to 5 (least satisfied) for various Conference features. Scores from the more general survey in 1994 are shown to the right for comparison.

APL96		1994 Survey (for comparison)	
-----		-----	
<i>Tutorials</i>	1.95	<i>Proceedings</i>	1.73*
<i>Workshops</i>	1.95	<i>Location</i>	1.76*
<i>Location</i>	2.11*	<i>Vendor information</i>	1.80
<i>Vendor Information</i>	2.20	<i>Program</i>	2.00
<i>Housing</i>	2.26	<i>Social/Vacation</i>	2.08
<i>Program</i>	2.54	<i>Workshops</i>	2.20
<i>Software Exchange</i>	2.60	<i>Banquet</i>	2.23
<i>Cost</i>	2.74	<i>Birds of a Feather</i>	2.52*
<i>Business Opportunities</i>	2.83*	<i>Tutorials</i>	2.58
<i>Birds of a Feather</i>	2.83*	<i>Housing</i>	2.59
<i>Social/Vacation</i>	2.83	<i>Poster Sessions</i>	2.67*
<i>Banquet</i>	3.15	<i>Software Exchange</i>	2.69
<i>Job Opportunities</i>	3.50*	<i>Cost</i>	3.01
<i>Poster Sessions</i>	3.70*	<i>Business/job oppty</i>	3.38*

* not comparable; Birds of a Feather, Poster Sessions, and Proceedings were missing at APL96. Business opportunities and job opportunities were combined in the 1994 survey, but split in the APL96 survey. The fact that data aren't comparable is a healthy measure of the ability of APL Conferences to evolve and adapt to changing conditions.

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.

Dynamic Functions in Dyalog APL

by John Scholes, Dyadic Systems (john@dyadic.com)

A Dynamic Function (D-Fn) is an experimental function definition style which bridges the gap between named function expressions such as *rank+pop* and APL's conventional 'header' style definition. Dynamic Functions are so called because they can be defined 'on the fly' at run time.

Simple Expressions

The simplest form of a D-Fn is: `[expr]` where `expr` is an APL expression containing '`α`'s and '`ω`'s representing the left and right argument of the function respectively. For example:

```
{(+/ω)÷ρω} 1 2 3 4      ⍝ Arithmetic mean
2.5
2 {(α-⊖io)+ιω-α-1} 5     ⍝ Sequence α ... ω
2 3 4 5
{{ω*0.5}+/ω*2} 3 4      ⍝ Pythagoras (Note inner D-Fn).
5
```

A D-Fn can be used in any function context ...

```
{α+ιω}/25ρ1              ⍝ Golden ratio repeated fraction.
1.618033989
```

... and of course, assigned a name:

```
root←{ω÷α}               ⍝ α'th root.
```

D-Fns are ambivalent. Their right (and if present, left) arguments are evaluated irrespective of whether these are subsequently referenced within the function body.

```
2 {ω} 3
3
4 {α} 5
4
```

Guards

A guard is a boolean-single valued expression followed by ':'. A simple expression can be preceded by a guard, and any number of guarded expressions can occur separated by '◊'s.

```

α<0:                                a Left arg negative.
0==w:                               a Right arg simple scalar.

```

Guards are evaluated in turn (left to right) until one of them yields a 1. Its corresponding **expr** is then evaluated as the result of the D-Fn. A guard is equivalent to an If-Then-Else or Switch-Case construct.

```

{w=0:'zero' ◊ w≠0:'non zero'} 3
non zero

```

A final simple **expr** can be thought of as a default case:

```

{2|w:'odd' ◊ 'even'} 3                a Parity.
odd
{w<0:'neg' ◊ w=0:'zero' ◊ 'pos'} 0    a Signum.
zero

```

The '◊'s in the previous examples can be replaced by newlines. During input, automatic indentation reminds us that we are typing into a D-Fn. If the closing '}' is the first character typed in a line, the interpreter's input mechanism 'snaps' it back a tab stop before echoing it. For readability, extra null phrases '◊ ◊ ◊ ...' can be included. The parity example above becomes:

```

{
    2|w: 'odd'                a Parity
    'even'                   a Odd case
                             a Even case
} 4
even

```

D-Fns can be reviewed using the system editor: `)ed` or `⌈FX/⌈CR`, and note that you can comment them in the normal way using '`⌈A`'.

The following example interprets a dice throw:

```

dice+{
  w=6 6: 'Box Cars'
  w=1 1: 'Snake Eyes'
  =/w: 'Pair'
  7=+/w: 'Seven'
      'Unlucky'
}

```

Local Definition

The final piece of D-Fn syntax is the local definition. An expression whose principal function is a simple or vector assignment introduces a name which is local to the D-Fn. The name is shadowed dynamically as the assignment is executed. As we shall see later, this mechanism can be used to define local nested D-Fns.

```

mean+{                                     A Arithmetic mean.
  sum+ +/w
  num+ p w
  sum÷num
}

roots + {                                A Real roots of quadratic.
  a b c+ w                               A Coefficients.
  d+ (b*2)-4*a*c                         A Discriminant.
  d<0: 0                                 A No roots
  d=0: -b÷2*a                            A One root
  d>0: (-b+-1 1*d*0.5)÷2*a               A Two roots
}

```

A nuance: the special syntax α +expr is executed only if the D-Fn is called with no left argument. This is equivalent to, but neater than:

```

1(0=⊖nc'Larg')/'Larg+...'

```

in a conventionally defined function.

```

root+{                                    A  $\alpha$ 'th root
   $\alpha$ +2                                  A Default to sqrt.
  w** $\alpha$ 
}
root 64                                   A Square root.
8
3 root 64                                 A Cube root.
4

```

Recursion

D-Fns use recursion in place of iteration. Reference to the current function can of course use the function's name explicitly. However because we allow unnamed D-Fns to be incorporated in larger APL expressions, we also need a special symbol to represent self-reference. ' ∇ ' does the job.

' ∇ ' has a further small advantage in that a recursive function can be renamed without having to locate and change all the explicit self references within the function body.

```
fact+{                                A Factorial.
    w≤1: 1
        w×fact w-1                      A !
}

{0<w:∇ w-1}1e6                        A Benchmark: iterate a million times.
                                         A *

gcd+{                                  A Euclid's greatest common divisor.
    w=0: α
        w ∇ w|α                          A *
}

fib+{                                  A w'th Fibonacci number.
    1≥w: w
        +/∇ w-1 2                        A !
}

ack+{                                  A Ackermann's function.
    α=0: w+1
    w=0: (α-1)∇ 1                        A *
        (α-1)∇ α ∇ w-1                  A *!
}

osc + {                                A Oscillate - probably returns 1.
    1=w: 1
    2|w: ∇ 1+3×w                          A *
        ∇ w÷2                            A *
}
```

These examples show that D-Fns contain a minimum of syntactic baggage - allowing the underlying algorithms to 'shine through'. For example, compare the *osc* example above with the equivalent coding using control structures:

```

      V Z+OSC n
[1]   :If 1=n
[2]     Z+1
[3]   :ElseIf 2|n
[4]     Z+OSC 1+3×n
[5]   :Else
[6]     Z+OSC n÷2
[7]   :End
      V

```

A happy consequence of reducing syntax is that the interpreter has less work to do. In the current experimental implementation, D-Fns significantly out-perform their conventionally defined counterparts.

Tail Recursion

In the previous examples, the recursive calls marked (A*) are said to be **tail-recursive** whereas those marked (A!) are not. A call is tail-recursive if its result is passed back unchanged to its own calling environment.

Tail recursion can be implemented in the interpreter by branching to, rather than calling, the current function. This delivers a substantial performance benefit, both in terms of execution time and workspace used.

It is often possible to turn a 'stack-recursive' D-Fn into tail-recursive form by accumulating the result on the way into, rather than on the way out of, the recursion. In general, this technique employs an **auxiliary function** which takes an **accumulating parameter**. APL's infix syntax is particularly suited to this technique as we can use the left argument exclusively to accumulate result-so-far, and the right argument to specify work-outstanding. The calculation then consists of moving work from the incoming right argument into the left argument of the next recursive call.

```

fact+({                                     A Tail recursive Factorial.
  1 {
    w≤1: α
    (α×w) V w-1                               A *
  } w
})

```

```

fib+({                                     A Tail recursive Fibonacci.
  1 0 {
    w=0: 1+α
    (1+α, +/α) V w-1                         A *
  } w
})

```

We can simplify the coding by expressing the auxiliary function as a dyadic case of the outer monadic function:

```
fact+{
  α+1
  ωs1: α
  (α×ω)∇ ω-1
}

sieve+{
  α+0
  next+1+ω
  mask+×next|ω
  ^/1+mask:α,ω
  (α,next)∇ mask/ω
}

sieve 1+120
2 3 5 7 11 13 17 19
```

A *

A Sieve of Eratosthenes.
A No primes yet.
A Next prime and ...
A Mask of non-multiples.
A No multiples, finished.
A Sieve out further primes.
A *

A Sieve primes from 2 .. 20

It's easy to check if a call is tail recursive by tracing it. Tail recursion appears to re-use the current trace window by branching back to the first line of the function, whereas a stack-recursive call pops up a new trace window.

D-Ops: Dynamic Operators

The operator equivalent of a D-Fn refers to its left and right operand using 'αα' and 'ωω' respectively. A D-Op is distinguished from a D-Fn by the presence of either of these symbols anywhere in the definition. A D-Op containing only 'αα's is monadic, whereas one which contains at least one 'ωω', is dyadic.

Note: in these and following examples, $\square ml$ migration level is set to 3. In particular, $\triangleright$ means mix and $\dagger$ means first.

```
else+{
  α: αα ω
  ωω ω
}

inner+{
  αα/>(+α)◊.ωω+⊞ω
}
```

A Condition f else g ...
A True: apply left operand.
A False: apply right operand.

A Inner product.

The following example makes use of a coding technique, well established in Dyalog APL, of setting an undefined left argument to be the identity function.

The following line then applies its function operands with the same valency as the derived function itself.

```
with+{
  α+{w}
  (α α α w)(α w w w)
}
      A Both operands applied.
      A Larg defaults to identity fn.
      A Return 2-element vector.
```

For recursive D-Ops, there are two kinds of self reference: 'v' refers to the current derived function, that is, the operator bound with its operand(s). This is by far the most frequently used form of self reference. Occasionally however, we might need a recursive reference to the operator itself and then we would use: 'vv', although a simple example doesn't spring to mind!

```
pow+{
  0=α: w
  (α-1)v α α w
}
      A Explicit function power.
      A Apply function operand α times.
      A α α α α α ... w
      A *

until+{
  w w w: w
  v α α w
}
      A Conditional function power.
      A Until w w,
      A Apply α α α ...
      A *

limit+{
  z+α α w
  z=w: w
  v z
}
      A Function power limit.
      A Apply operand until
      A ... it has no effect.
      A Else, repeat.
      A *

perv+{
  1=α w: α α α w
  α v w
}
      A Scalar pervasion, apply ...
      A ... fn between simple scalars.
      A ... else, traverse each arg.
      A !

saw+{
  2>=w: α α w
  v w
}
      A Simple-array-wise.
      A Simple, apply operand.
      A Nested, visit each subarray
      A !
```

A contrived use of operator self reference 'vv', would be to re-code the last example:

```
saw+{
  2>=w: α α w
  α vv w
}
      A Simple-array-wise.
      A Simple, apply operand.
      A Nested, visit each subarray
      A !
```

Example: Pythagorean numbers

The following sequence shows an example of combining Dynamic Functions and Operators in an attempt to find Pythagorean triangles (3 4 5)(5 12 13) ...

```

      sqrt+{w*0.5}                # Square root.
      sqrt 9 16 25
3 4 5

      hyp+{sqrt+/{w*2}            # Hypotenuse of triangle.
      hyp(3 4)(4 5)(5 12)
5 6.403124237 13

      intg+{w=[w}                 # Whole number?
      intg 2.5 3 4.5
0 1 0

      pyth+{intg hyp w}           # Pythagorean pair?
      pyth(3 4)(4 9)(5 12)
1 0 1

      pairs+{, [w w}              # Pairs of numbers 1..w.
      pairs 3
1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3

      filter+{(α α w)/w}          # Op: w filtered by αα.
      pyth filter pairs 12        # Pythagorean pairs 1..12
3 4 4 3 5 12 6 8 8 6 9 12 12 5 12 9

```

So far, so good, but we seem to have some duplicates: for example, (6 8) is just double (3 4).

```

      rpm+{                        # Relatively prime?
      w=0:α=1                     # C.f. Euclid's gcd.
      w ∇ w|α
      }/"                          # Note the /"

      rpm(2 4)(3 4)(6 8)(16 27)
0 1 0 1

      rpm filter pyth filter pairs 20
3 4 4 3 5 12 8 15 12 5 15 8

```

We can use an operator to combine the tests:

```

      and+{                        # Lazy parallel 'And'.
      mask+α α w                  # Left predicate selects...
      mask\w w mask/w             # args for right predicate.
      }

```

```
pyth and rpm filter pairs 20
3 4 4 3 5 12 8 15 12 5 15 8
```

Better, but we still have some duplicates: (3 4) (4 3).

```
less+{</>w}
less(3 4)(4 3)
1 0
```

```
less and pyth and rpm filter pairs 50
3 4 5 12 7 24 8 15 9 40 12 35 20 21 28 45
```

A larger example

The *display* function uses nested local D-Fns. The details of the code are not important - the example is included to show the appearance of a larger function.

```
display+{[IO [ML+0 3
  0:=w:{
    fmt+,*w
    num+0:=+0pw
    bar+num>' - '
    >' 'fmt bar
  }w

  tax+{
    axes+1+*w
    size+2[pw
    (-size)+axes
  }pw

  box+{
    vrt hrz+'|'- '{
      1+wpw
    }"pw

    top+'-e+'[~1+tax],hrz
    bot+α,hrz
    rgt+'|',vrt, ''''
    lvex+'|φ+'[~1+tax],~cvrt
    lft+~|',(>lvex), ''''
    lft,(top,w,bot),rgt

  }

  xnulls+{
    (1[pw)+w
  }
```

A Boxed display of array.
 A Simple scalar ...
 A ... formatted
 A Numeric?
 A Underscore if char.
 A Formatted scalar.

A Type of axis:
 A 1:null 2:non-null.
 A Size of result.
 A 0:extra padding.

A v Boxed char array.
 A Vert and horiz lines.

A Upper border with shape.
 A Lower border with type.
 A Right border, corners.
 A Left border, shape
 A ... and corners.
 A Fully boxed array.

A v Expose nulls.

```

1=:w:{
    type+{
        ^/w:'~'
        v/w:'+'
        '~- '
    }0=+0p<,w
    type box [FMT xnulls w
}w

subs+[FMT v"xnulls w
mask+^/subs=' '
'e'box(~1 1<mask)/subs
}

```

A Simple array.
A Type:
A ~: all numeric.
A +: mixed.
A -: all char.
A Box simple array.
A Formatted subarrays.
A Extra blank cols
A Nested array

```
display 'a'(1 0 2p<0)(2'b')
```

```

+-----+
| ..+-----+ ..+---+ |
| a +φ .e. .e. | 2 b | | | | | | |
| - || |0| |0| | '+---' |
| || '~- '~- ' | |
| 'e-----' | |
+-----+

```

Summary

The syntax of a D-Fn / D-Op is shown in the railway diagram:

```

+-----+-----+
|---LDEF+EXPR---|
|---GUARD:EXPR---|
--(-+-----EXPR+-)--
|+-----o-----|

```

A Null phrase,
A Local definition,
A Guarded result expression,
A Unguarded result expression,
A ... separated by o-s.

LDEF: Definition local to the D-Fn using simple or vector assignment.
GUARD: Expression which evaluates to a Boolean single.
EXPR: Arbitrary APL expression: the result of the D-Fn.

Following special characters are substituted:

α: Function Left Argument. (α+... if left argument omitted.)
w: Function Right Argument.
v: Function (and Derived Function) Self reference.

αα: Operator Left Operand.
ww: Operator Right Operand.
vv: Operator Self reference.

The phrases are evaluated in turn until an unguarded EXPR, or one whose guard evaluates to 1, is encountered. The corresponding EXPR is then evaluated as the result of the D-Fn.

Conclusion

D-Fns give us the opportunity to combine the power of expression of APL with an alternative, and in some circumstances, more convenient functional framework.

They encourage (but don't compel), the programming of 'pure' functions. There is evidence to suggest that coding in such a style produces more reliable software.

D-Fns provide true local functions. They encourage the use of sub-functions which may otherwise be avoided owing to a (perceived) degradation in performance or code readability. For example, it is often tempting to replace a call on a small function applied under each (") by embedding the code directly in the calling line and peppering it with ' 's or 'o's.

Using D-Fns in this situation improves both speed and readability. For example:

```

t o p p "vec          =>  {t p p w}"vec          A Number of axes.

+ / " ^ " CVEX ε " c t c  =>  {+ / ^ \ w ε t c} " CVEX  A Leading TC chars.

```

Implementation Status

D-Fns are currently implemented in Dyalog on an experimental basis. While some issues are yet to be resolved, the results so far look encouraging. Coding examples such as those included in this article show that they have a reasonably wide application. Measurements of their performance are particularly pleasing.

These ideas were first presented in the Dyadic Vendor Forum at APL96 where they appeared to meet with general approval. Depending on the reaction of the APL community at large, we hope to include many of them in a forthcoming release.

OLE Automation with Dyalog APL/W

by Peter Donnelly (Dyadic Systems Limited)

Introduction

Dyalog APL/W Version 8.0, released in May, already includes support for OLE Controls (OCXs). This capability allows you to incorporate external GUI objects in your Dyalog APL applications just as easily as if they were built-in controls.

At the APL96 Dyadic vendor forum, Peter Donnelly announced support for *OLE Automation*: the next part of the OLE jigsaw.

OLE Automation allows you to drive one application from another and to mix code written in different programming languages. In practical terms, this means that you may write a subroutine to perform calculations in (say) C++ and use the subroutine directly in Visual Basic 4 or Excel 7. Equally, you could write the code in Visual Basic and call it from C++. Dyalog APL/W is now a member of this code-sharing club.

OLE Automation is, however, much more than just a mechanism to facilitate cross-application macros because it deals not just with subroutine calls but with *objects*. An object is a combination of code and data that can be treated as a unit. Without getting too deeply into the terminology, an object defines a *class*; when you work with an object you create one or more *instances* of that class.

Namespaces and Objects

There is a direct correspondence between the object model and Dyalog APL namespace technology, a correspondence that is thoroughly exploited in the implementation of OLE Automation.

An OLE object is simply a collection of methods (code that performs tasks) and properties (data that affects behaviour). An object corresponds directly to a Dyalog APL namespace which contains functions that do things and variables that affect things. Furthermore, OLE objects are hierarchical in nature; objects may contain sub-objects just as namespaces may contain sub-namespaces. To complete the picture, an OLE Server is an application that provides (exposes) one or more OLE objects. Thus an OLE Server corresponds directly to a workspace that contains one or more namespaces.

However, one element of the equation was missing. When you access an OLE object, you do so by creating an *instance* of its class and you may work with several instances at the same time. Furthermore, several applications may access the same OLE object at the same time, each with its own set of instances. Each instance inherits its methods (functions) and the initial values of its properties from the class. However, different property values will soon be established in different instances so they must be maintained separately.

Dyalog APL/W Version 8.1 includes the capability for a namespace to spawn instances of itself. Initially, a new instance is simply a pointer to the original namespace (not a copy), but as soon as anything in it is changed, the new value is recorded separately. Thus instance namespaces will typically share functions but maintain separate sets of data.

Defining a Dyalog APL OLE Server

Peter demonstrated just how easy it is to create your own OLE Server. Just 3 steps are involved.

1. Create a workspace containing at least one namespace which itself contains some functions and/or variables.
2. Tell Dyalog APL that you wish to export the namespace as an OLE object by changing its Type to OLEServer. You do this by executing `⎕WC` inside the namespace.
3.)Save the workspace. At this point, Dyalog APL automatically updates the Windows registry and type libraries with all of the information needed to make your objects accessible by OLE.

Example of a Dyalog APL OLE Server

As an example, Peter loaded a workspace called LOANOA. This contained a single namespace called *Loan* which is used to calculate monthly repayments on a loan. Within *Loan* there was a single function *CalcPayments* and a variable *PeriodType*. The *CalcPayments* function takes a 5-element numeric vector as an argument whose elements specify :

1. loan amount
2. maximum number of periods for repayment
3. minimum number of periods for repayment

4. maximum annual interest rate

5. minimum annual interest rate

CalcPayments also uses the "global" variable *PeriodType* which specifies whether the periods (above) are years or months. This is done solely to illustrate how another application can manipulate an APL object via its variables (properties) as well as by calling its functions (methods).

CalcPayments returns a matrix. The first row contains the period numbers (from min to max). The first column contains the interest rates (from min to max in steps of 0.5%). Other elements contain the monthly repayments for the corresponding number of periods and interest rates. The transcript of the APL session and a listing of *CalcPayments* is given below.

```
Dyalog APL/W Version 8.1
Serial No : 000043 / Pentium
Wed Aug 7 10:58:44 1996
clear ws
)LOAD DEMO\LOANOA
DEMO\LOANOA saved Wed Jul 31 13:25:02 1996
)OBS
Loan
)CS Loan
#.Loan
)FNS
CalcPayments
)VARS
PeriodType
```

```
CalcPayments 10000 5 3 6 3
0      3      4      5
3  290.8120963 221.3432699 179.6869066
3.5 293.0207973 223.5600105 181.9174497
4   295.2398501 225.7905464 184.1652206
4.5 297.4692448 228.0348608 186.4301924
5   299.708971 230.2929357 188.7123364
5.5 301.959018 232.5647523 191.0116217
6   304.2193745 234.8502905 193.3280153
```

```

)CS
#
Loan,[]WC 'OLEServer'
)SAVE
DEMO\LOANOA saved Wed Aug 7 11:00:58 1996

[0] PAYMENTS←CalcPayments X;LoanAmt;LenMin;LenMax;IntrMin;IntrMax;
PERIODS;INTEREST;NI;NM;PER;INT
[1] a Calculates loan repayments
[2] a Argument X specifies:
[3] a LoanAmt Loan amount
[4] a LenMax Maximum loan period
[5] a LenMin Minimum loan period
[6] a IntrMax Maximum interest rate
[7] a IntrMin Minimum interest rate
[8] a Also uses the following global variable (for illustration)
[9] a PeriodType Type of period; 1 = years, 2 = months
[10]
[11] LoanAmt LenMax LenMin IntrMax IntrMin←X
[12]
[13] PER←PERIODS÷1+LenMin+1+LenMax-LenMin
[14] PERIODS←PERIODS×12 1[PeriodType]
[15] INT←INTEREST÷0.5×1+(2×IntrMin)+1+2×IntrMax-IntrMin
[16] INTEREST←INTEREST÷100×12 1[PeriodType]
[17]
[18] NI←ρINTEREST
[19] NM←ρPERIODS
[20]
[21] PAYMENTS←(LoanAmt)×((NI,NM)ρNM/INTEREST)÷1-1÷(1+INTEREST)÷PERIODS
[22] PAYMENTS←PER,[1]PAYMENTS
[23] PAYMENTS←(0,INT),PAYMENTS

```

Using the Server from Visual Basic

Peter next started Visual Basic Version 4.0 and loaded a Visual Basic application he had prepared earlier. This was based upon an example (VB\SAMPLES\GRID\LOAN.VBP) which is distributed with Visual Basic and is described in the *Programmer's Guide* Chapter 13. Peter had modified the code so that instead of performing the calculations in Visual Basic, it employed the Dyalog APL Loan object instead.

LoanSheet

Options Down Payment Loan Length

Purchase Amount: 150000

Percent Down: 20

Years in Loan:

Maximum: 35

Minimum: 20

Interest Rates:

Maximum %: 15.0

Minimum %: 7.0

Show Payments

Show Amortization

%	20 Years	21 Years	22 Years
7.0%	£930.36	£910.17	£892.11
7.5%	£966.71	£947.00	£929.41
8.0%	£1,003.73	£984.51	£967.41
8.5%	£1,041.39	£1,022.69	£1,006.09
9.0%	£1,079.67	£1,061.50	£1,045.41
9.5%	£1,118.56	£1,100.92	£1,085.35
10.0%	£1,158.03	£1,140.94	£1,125.90
10.5%	£1,198.06	£1,181.52	£1,167.01
11.0%	£1,238.63	£1,222.65	£1,208.67
11.5%	£1,279.72	£1,264.29	£1,250.85
12.0%	£1,321.30	£1,306.44	£1,293.53
12.5%	£1,363.37	£1,349.06	£1,336.67
13.0%	£1,405.89	£1,392.14	£1,380.27
13.5%	£1,448.85	£1,435.64	£1,424.29
14.0%	£1,492.22	£1,479.56	£1,468.72

The changes required to the original Visual Basic code were as follows.

Firstly, in the declarations section, Peter had declared a variable called APLLoan to be of type Object :

```
Dim APLLoan As Object
```

Secondly, in the module that gets executed when the application starts (the Load procedure for the Form), he had inserted the statement:

```
Set APLLoan = CreateObject("dyalog.Loan")
```

The effect of these two statements is that when the application is run, VB asks OLE to provide the external object called "dyalog.Loan". This name was recorded by Dyalog APL when the workspace was saved and OLE knows that to satisfy this request it must start Dyalog APL with the corresponding workspace name. When it starts, Dyalog APL recognises that the request has come from OLE and responds in the appropriate manner. In particular, it creates an instance of the Loan namespace which is connected to the Visual Basic process that has requested it. Note that the APL Session is not displayed, although this *can* be specified for debugging purposes.

The final change was to rewrite the VB subroutine that performed the calculations to call instead the Dyalog APL Loan object. This subroutine is considerably shorter than the original and is shown below.

```
Private Sub CalcPmnts()
    Dim PeriodType
    ' Calculate the number of periods and interest rates to display.
    Periods = (LenMax - LenMin) + 1
    Rates = ((IntrMax - IntrMin) * 2) + 1
    If mnuOptLen(0).Checked = True Then
        PeriodType = 1
    Else
        PeriodType = 2
    End If
    ' Set PeriodType property of APLLoan object
    APLLoan.PeriodType = PeriodType
    ' Call CalcPayments method of APLLoan object
    Payments = APLLoan.CalcPayments(LoanAmt, LenMax, LenMin, IntrMax, IntrMin)
End Sub
```

There are really only two statements worth comment. The first is the line:

```
APLLoan.PeriodType = PeriodType
```

In Visual Basic terms, this statement sets the *PeriodType* property of the *APLLoan* object to the value of a local variable, also called *PeriodType*. What actually happens, is that the APL variable *PeriodType* in the corresponding running instance of the *Loan* namespace is set to the requested value.

The second statement of note calls the APL function *CalcPayments* and receives the result.

```
Payments = APLLoan.CalcPayments(LoanAmt, LenMax, LenMin, IntrMax, IntrMin)
```

In Visual Basic terms, this statement invokes the *CalcPayments* method of the *APLLoan* object. In practice, it calls the *CalcPayments* APL function with the specified argument and puts the result in the local variable *Payments*. Note that the conversion between a Dyalog APL floating-point matrix and the corresponding Visual Basic data type is performed automatically for you.

Using the Server from Excel

Peter next started Excel Version 7.0 and loaded a spreadsheet that he had prepared earlier and is shown below. The *Payments* button fires a simple macro (Excel now uses Visual Basic as its macro language) which once again calls our

APL Loan object to perform repayment calculations. In this case, the loan period is always expressed in years.

The screenshot shows a Microsoft Excel window titled "Microsoft Excel - Loan.xls". The menu bar includes File, Edit, View, Insert, Format, Tools, Data, Window, and Help. The toolbar contains various icons for file operations and calculations. The spreadsheet has columns A through H and rows 1 through 10. The data is as follows:

	A	B	C	D	E	F	G	H
1	Loan Amount		10000		0	3	4	5
2	Max Years in Loan		5		3	290.8121	221.3433	179.6869
3	Min Years in Loan		3		3.5	293.0208	223.56	181.9174
4	Max Interest Rate		6		4	295.2399	225.7905	184.1652
5	Min Interest Rate		3		4.5	297.4692	228.0349	186.4302
6					5	299.709	230.2929	188.7123
7			Payments		5.5	301.959	232.5648	191.0116
8					6	304.2194	234.8503	193.328
9								
10								

The status bar at the bottom shows "Ready" and "Sum=10000". The sheet tabs at the bottom are "Module2", "Module1", "Sheet1", "Sheet2", and "Sheet3".

The macro code is (as you would expect) almost identical to the Visual Basic example, except that this time the `PeriodType` is always set to 1.

```
Sub Calc()
    Dim APLLoan As Object
    Dim Payments As Variant
    Set APLLoan = CreateObject("dyalog.Loan")
    LoanAmt = Cells(1, 3).Value
    LenMax = Cells(2, 3).Value
    LenMin = Cells(3, 3).Value
    IntrMax = Cells(4, 3).Value
    IntrMin = Cells(5, 3).Value
    APLLoan.PeriodType = 1
    Payments = APLLoan.CalcPayments(LoanAmt, LenMax, LenMin,
    IntrMax, IntrMin)
    For r = 0 To UBound(Payments, 1)
        For c = 0 To UBound(Payments, 2)
            Cells(r + 1, c + 5).Value = Payments(r, c)
        Next c
    Next r
End Sub
```

So what's going on?

At this stage, we have two applications (Visual Basic and Excel) connected to a single Dyalog APL interpreter and a single workspace LOANOA. Each application is connected to a separate running instance of the *Loan* namespace, although there is only one copy of the *CalcPayments* function in memory. Only

the variable *PeriodType* is duplicated in the two instances of *Loan*. Note that the synchronisation of OLE requests from the different applications is entirely handled for you by Dyalog APL.

Accessing an OLE Server from Dyalog APL

Although Peter ran out of time in his presentation, Dyadic also demonstrated how Dyalog APL can act as an OLE client. This is simply a matter of creating a new *OLEClient* object giving it the name of the server object to which it is to be connected. For example, to use the *Loan* object from Dyalog APL you could do the following:

```
'LN'⎕WC'OLEClient' 'dyalog.Loan'
)OBS
LN
)CS LN
#.LN
)FNS
CalcPayments
)VARS
PeriodType
CalcPayments 10000 5 3 6 3
0      3      4      5
3  290.8120963 221.3432699 179.6869066
3.5 293.0207973 223.5600105 181.9174497
4  295.2398501 225.7905464 184.1652206
4.5 297.4692448 228.0348608 186.4301924
5  299.708971 230.2929357 188.7123364
5.5 301.959018 232.5647523 191.0116217
6  304.2193745 234.8502905 193.3280153
```

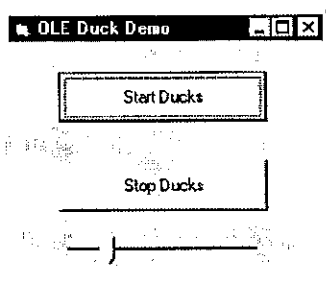
Once again, the mapping between the object and a namespace is direct and natural. If the OLE object is a foreign (non-APL) one, as it would normally be, the functions in the client namespace correspond to the object's methods, and the variables in the namespace represent its properties. In the above example, we have two separate copies of Dyalog APL running; one is the client, the other the server and the mapping between the corresponding functions and variables is direct. Effectively, the client namespace *LN* is an instance of the remote namespace *Loan*. This may seem an odd thing to want to do, but add the fact that network OLE allows you to run client and server processes on different machines and suddenly this capability becomes more exciting.

Conclusion

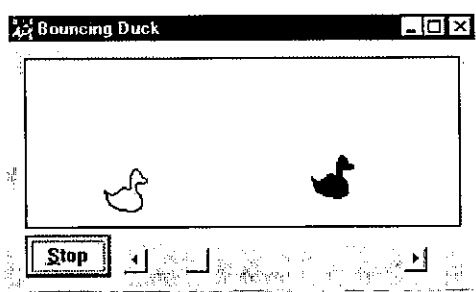
Dyalog APL with OLE technology offers a new range of opportunities for APL developers. Not only can you deliver complete packaged GUI applications in APL, but you can now market stand-alone software components that can be incorporated into other products which are constructed with other languages.

For companies with legacy mainframe APL applications, Dyalog APL offers you the chance to leverage this code in a Windows environment without the need to rewrite the user-interface in APL. You simply discard the existing user-interface, package the application as a set of OLE objects, and give it to your Windows programming staff to call from the programming environment of their choice.

Peter further illustrated this point using his own legacy APL application, the bouncing ducks demo, which has been presented in various guises at previous APL conferences. Only this time, he ran it from Visual Basic.



The Visual Basic Automation Controller



The Dyalog APL OLE Duck object

Perfect Shuffles In J

by Howard A. Peelle (*hapeelle@educ.umass.edu*)

The following problem was presented in [0]:

Magicians often perform amazing card tricks based on skilful execution of a "perfect shuffle". A perfect shuffle divides a deck of cards into two halves and interweaves them by alternating one card at a time from each half deck.

Here we will similarly shuffle alphabetic letters. For example, suppose the letters 'TECHNOLOGY' are written separately on index cards and stacked into a deck. Now divide the deck into halves and shuffle the cards — perfectly. The new deck spells 'TOELCOHGNY'. Further, we ask: if this deck were shuffled perfectly again, repeatedly, what spellings would result? The sequence below may be surprising.

'TECHN' shuffle 'OLOGY' yields 'TOELCOHGNY'
'TOELC' shuffle 'OHGNY' yields 'TOOHEGLNCY'
'TOOHE' shuffle 'GLNCY' yields 'TGOLONHCEY'
'TGOLO' shuffle 'NHCEY' yields 'TNGHOCLEOY'
'TNGHO' shuffle 'CLEOY' yields 'TCNLGEHOOY'
'TCNLG' shuffle 'EHOOY' yields 'TECHNOLOGY'

After six perfect shuffles, the original word is reproduced! This occurs similarly for other words.

The problem for you to program (in a language of your choice) is posed at three levels:

- 1.1 Write a program which inputs a word (string) and outputs a perfect shuffle of that word.
- 1.2 Write a program to compute the number of repeated perfect shuffles required to reproduce the original input word.
- 1.3 Explore perfect shuffles of words which produce anagrams. For example, the second perfect shuffle of 'PLATES' is 'PETALS'.

Solutions to these and related problems are developed here using J.

1.1 Perfect Shuffle

To begin with, a perfect shuffle can be done directly in J using the primitive verb `/:` (called Sort) for the given example:

```
'technology' /: 0 2 4 6 8 1 3 5 7 9
toelcohgny
```

This sorts the list on the left according to the order specified on the right (first is item from the left in position corresponding to 0 on the right, then item corresponding to 1, then 2, etc.).

The indices 0 2 4 6 8 1 3 5 7 9 here represent the order to select items for this particular perfect shuffle.

A J program to do a perfect shuffle for any input list is:

```
psh =: /: shuffle
```

This program is named `psh` (for perfect shuffle). It is defined as a composition of verbs (functions) with implicit inputs — that is, without pronouns (variables) — and will be executed for an input `y` as `y /: (shuffle y)`. See [1] for introduction to this “hook” structure. Here, the Sort verb `/:` uses the input list and the result of subprogram `shuffle` applied to the input list (which produces permutation indices such as shown above). Subprogram `shuffle` is defined as follows:

```
shuffle =: /: @ perfect
```

Here the primitive verb Grade-Up `/:` (using one input) is applied Atop (`@`) the result of subprogram `perfect`, which is:

```
perfect =: # $ 0: , 1:
```

These verbs are composed as a “fork” within a fork (see [1] for introduction), which will be executed for an input `y` as `(#y) $ ((0: y), (1: y))`.

Review of the J primitive verbs:

- 0: (Zero) is a constant verb which returns 0.
- 1: (One) is a constant verb which returns 1.
- . (Append) joins items on left to items on right.
Example: 0, 1 is 0 1.

(Tally) counts the number of items in a list.

Example: # 'technology' is 10.

\$ (Shape) extends a list of items on right to the length on left.

Example: 10 \$ 0 1 is 0 1 0 1 0 1 0 1 0 1.

/: (Grade-Up, with one input, as in subprogram shuffle) returns indices to sort a list in ascending order.

Example: /: 0 1 0 1 0 1 0 1 0 1 is 0 2 4 6 8 1 3 5 7 9.

/: (Sort, with two inputs, as in psh) rearranges a list on the left into order specified on the right.

Example: 'technology' /: 0 2 4 6 8 1 3 5 7 9 is 'toelcohgny'.

To see intermediate results, execute the subprograms separately with the example input list:

```
perfect 'technology'
0 1 0 1 0 1 0 1 0 1
```

```
shuffle 'technology'
0 2 4 6 8 1 3 5 7 9
```

To do a perfect shuffle, execute the main program:

```
psh 'technology'          NB. Perfect shuffle
toelcohgny
```

Note that this program works for a list of any length — even or odd:

```
psh 'technolog'          NB. Odd length list
toelcohgn
```

It works correctly for degenerate cases:

```
psh 'te'                 NB. Two characters
te
```

```
psh 't'                  NB. One character
t
```

```
psh ''                   NB. Empty list input
                          NB. Empty list result
```

It works for numbers as well as characters:

```
psh 0 1 2 3 4 5 6 7 8 9      NB. Numeric list
0 5 1 6 2 7 3 8 4 9
```

It works for an array of items:

```
psh 'A♠','A♥','A♦','A♣','K♠','K♥','K♦','K♣'
A♠
K♠
A♥
K♥
A♦
K♦
A♣
K♣
```

NB. 8 by 2 matrix

And it works for boxed items:

```
psh 'A♠';'A♥';'A♦';'A♣';'K♠';'K♥';'K♦';'K♣'
```

A♠	K♠	A♥	K♥	A♦	K♦	A♣	K♣
----	----	----	----	----	----	----	----

NB. 8 boxed 2-items

See [1] for introduction to arrays such as the matrix and boxed list above, as well as for definitions of primitive verbs , (Append) and ,: (Laminate) and ; (Link).

1.2 Period of Perfect Shuffles

J provides a primitive operator $\wedge$: (called Power) for iterating a verb (or program) on its own result — which is exactly what can be done here with psh to determine its period. Specifically, `psh $\wedge$ :N 'technology'` will iterate psh N times, starting with the input 'technology'. Doing several iterations at once traces the shuffles:

```
psh  $\wedge$ : 1 2 3 4 5 6 'technology'
toelcohgny
tooheglncy
tgonlhcey
tnghocleoy
tcnlgehooy
technology
```

It is easy to repeat program psh enough times and find its period by inspection. It would be preferable, however, to have the computer iterate until the shuffled

list matches the original list and just count the iterations. So, here is an iterative program to do that:

```

period =: 3 : 0
List =. Input =. ,y.
N =. 1
notmatch =. -.@=:
while. Input notmatch List =. psh List
do. N =. >:N
end. N
)

```

Explanation: This program is named `period` and is defined to be entered explicitly line-by-line. The first line assigns local pronouns named `List` and `Input` to be the original input (`y.`) ravelled (`,`) as a list. The next line assigns pronoun `N` to be 1. The next line defines a utility verb called `notmatch` to be `Not (-.) Atop (@) Match (=:)`. The next three lines use a “while-do” control structure: while the original `Input` does not match the current `List` (which is repeatedly re-assigned to be the result of program `psh`), do `Increment (>:)` `N`. After this ends, the final result is `N` — the number of iterations. For example:

```

period 'technology'          NB. Period of perfect shuffles
6

```

Better yet, mathematical analysis ([2],[3]) reveals that the period is the least common multiple of the lengths of the cycles of the permutation indices produced by `/:` applied to a shuffling pattern given as input. Hence, a more efficient definition is:

```

Period =: lcm @ cycles @ /:

cycles =: # @ > @ C.

lcm =: */

```

Explanation: Program `Period` is a composition of subprogram `lcm` `Atop (@)` subprogram `cycles` `Atop (@)` `Grade-Up (/:)`. The input will be a list of numbers indicating the pattern for shuffling. For example, using the perfect pattern: `/: 0 1 0 1 0 1 0 1` is `0 2 4 6 8 1 3 5 7 9`. Then `cycles 0 2 4 6 8 1 3 5 7 9` is `1 2 6 1`. Subprogram `cycles` is a composition of verbs: `C.` (called `Cycle-Direct`) returns boxed lists of permutation cycles. For example, `C. 0 2 4 6 8 1 3 5 7 9` is `0` and `6 3` and `8 7 5 1 2 4` and `9` in four boxes. Then `>` (`Open`) opens the boxes, and `#` (`Tally`) counts the items in each box — resulting in `1 2 6`

1. Finally, subprogram 1cm Inserts (/) the primitive function *. (Least Common Multiple) between each item, as in $1 * 2 * 6 * 1$, producing 6.

Note that an alternative program might well be defined directly on permutations (as pointed out by Roger Hui). Here, only the shuffle pattern is needed for input. For example:

```
Period 0 1 0 1 0 1 0 1 0 1
6
```

Results of using either program for lists of different lengths to determine the period of perfect shuffles:

```
Length   1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 ...
Period   1 1 2 2 4 4 3 3 6 6 10 10 12 12 4 4 8 8 18 18 ...
```

Such results may be produced automatically by the J program `Periods =: Period@perfect\` with an input list of any length. See [1] for explanation of Prefix (\).

The reader may now study these results further.

1.3 Anagram Explorations

Anagrams can be explored using `^:` (as shown earlier in section 1.2) with iterates up to the period. For example:

```
psh ^: 1 2 3 4 'median'
mieadn
maiden
mdaein
median
```

Words with correct spelling, such as 'maiden', can be found in a dictionary (manually or by computer).

Further explorations of related problems follow.

1.4 Inverse Perfect Shuffle

Here is a J program to undo a perfect shuffle:

```
ipsh =: /: perfect
```

This program is similar to psh but uses subprogram perfect instead of shuffle. Here the alternating 0s and 1s (produced by perfect) cause the inverse perfect shuffle directly. For example, 'technology' /: 0 1 0 1 0 1 0 1 0 1 sorts the list according to the positions of all the 0s then all the 1s. Thus:

```
ipsh 'toelcohgny'      NB. Inverse perfect shuffle
technology
```

Note that applying psh and its inverse ipsh (in either order) always reproduces the input:

```
psh ipsh 'technology'  NB. Identity
technology
```

```
ipsh psh 'technology'  NB. Identity
technology
```

1.5 Out-Shuffles and In-Shuffles

Actually, there are two different ways to perform a perfect shuffle. One is to start with the first item in the first half of the list (call this an "out-shuffle"). The other way is to start with the first item in the second half of the list (an "in-shuffle").

The programs developed so far perform an "out-shuffle". To perform an "in-shuffle" subprogram perfect could be modified easily to use 1: , 0: instead of 0: , 1: . Better yet, generalize (and simplify!) the main program to do both:

```
Psh =: /: /:
```

This program uses two inputs. The list is input on the left, and the input on the right specifies the shuffling pattern. Examples:

```
'technology' Psh 0 1 0 1 0 1 0 1 0 1      NB. Out-Shuffle
toelcohgny
```

```
'technology' Psh 1 0 1 0 1 0 1 0 1 0      NB. In-Shuffle
otleocghyn
```

Note the different uses of /: in a "hook" composition in program Psh. Sort (the first /: on the left) uses two inputs; whereas Grade-Up (the second /:) uses one input. Specifically, the in-shuffle example above is interpreted as 'technology' /: (/: 1 0 1 0 1 0 1 0 1 0). See [1] for more explanation.

The period of an in-shuffle can be computed similarly. For example:

Period 1 0 1 0 1 0 1 0 1 0 NB. In-shuffle period
10

Resulting periods for different length in-shuffles are:

Length	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
In-Period	1	2	2	4	4	3	3	6	6	10	10	12	12	4	4	8	8	18	18	6	...

The reader may observe patterns in these periods and compare them with out-periods (from section 1.2):

Length	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	...
Out-Period	1	1	2	2	4	4	3	3	6	6	10	10	12	12	4	4	8	8	18	18	...

For instance, the out-period of length 2^n is n , and the in-period of length 2^n is $2n$ (for $n > 1$). Also note that several in-periods are the same as their length. Questions: Are there any such out-periods (greater than 1)? Do any in-periods or out-periods exceed their length?

1.6 n Items and n Piles

The generalized Psh program can shuffle n items at a time. For example:

'technologies' Psh 0 0 1 1 0 0 1 1 0 0 1 1 NB. 2 at a time
telochginoes

It also allows shuffles of more than two piles. For example:

'technologies' Psh 0 1 2 0 1 2 0 1 2 0 1 2 NB. 3 piles
tngeioiclehos

Resulting periods for out-shuffles of (n) piles are:

Length	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Period (2)	1	2	2	4	4	3	3	6	6	10	10	12	12	4	4	8	8	18	18	
Period (3)		1	3	4	4	6	2	2	6	5	5	11	6	6	15	16	16	52	4	
Period (4)			1	4	6	3	3	7	9	5	5	18	10	2	2	8	8	4	9	
Period (5)				1	5	6	10	6	6	9	12	11	6	6	15	44	42	9	9	

More piles and more items-at-a-time can be handled similarly — as out-shuffles, in-shuffles, and various mid-shuffles. Indeed, program Psh can shuffle in any specified pattern.

Acknowledgements

Thanks to Lee Dickey, Murray Eisenberg, Roger Hui, Alan Lipp, Eugene McDonnell, Keith Smillie, and Norm Thomson for feedback on this article.

Howard A. Peelle
University of Massachusetts
Furcolo Hall #10
Amherst, MA 01003
(413) 545-1114

References

- [0] Lipp, A. and Peelle, H. "*Problem Solving by Programming: Perfect Shuffles*", Journal of Computer Science Education, Vol. 9 No. 1, Fall 1994 p. 10
- [1] Iverson, K. J *Introduction and Dictionary*, Iverson Software Inc., Toronto, 1995
- [2] Thomson, N. "*Roger and the Amazing Technicolor Ballclock*", Vector, Vol. 12, No. 4, April 1996, p. 128

Gregorian Calendar Calculations in J

by Keith Smillie (smillie@cs.ualberta.ca)

Various calculations for the Gregorian calendar are expressed in J, including those for the construction of calendars for a specified month, quarter or year, ordinal and Julian dates, and the number of days between any two dates. Additional date and time verbs may be found in Burke, Hui *et al* (1996) and in the utility file `c:\j3\main\dates.js`.

Introduction

In the Gregorian calendar, which was introduced in 1582, leap years are defined as those years which are divisible by 4 except for centennial years which must be divisible also by 400. For example, 1996 and 2000 are leap years, but 1993 and 1900 are not. Therefore, a simple test for leap year is given by

```
LY=. ~:/ @ (mod&4 100 400 = 0:)
```

where

```
mod=. |~
```

which has the value 1 if the argument is a leap year, i.e., if the number of the divisors 4, 100 and 400 of its argument is odd, and 0 otherwise.

If we define the constant

```
LENGTHS=. 31 28 31 30 31 30 31 31 30 31 30 31
```

then the verb

```
L=. LENGTHS&+0((LENGTHS=28)&*. )@LY
```

gives a twelve-item list of the lengths of the months with the second item equal to 29 if the argument is a leap year and 28 otherwise. Also we shall define the list

```
DAYS=. ' Su Mo Tu We Th Fr Sa'
```

of day names, and the twelve-item list

```
MONTHS=. _3<\ ' JanFebMarAprMayJunJulAugSepOctNovDec'
```

of month names.

We shall also define the verbs

```
div=. <.@%
```

for the integer quotient, and

```
pos=. >: @ i.
```

for non-negative integers, where $18 \text{ div } 7$ is 2, and $\text{pos } 4$ is 1 2 3 4.

Date number

In order to simplify the calculations we shall consider only dates in the Gregorian calendar beginning with the first day of the seventeenth century, i.e., January 1, 1601, and number the days sequentially with January 1, 1601 as 1, January 2, 1601 as 2, ..., January 1, 1602 as 366, and so on. The sequence number for any date, which we shall call the "date number", may be computed as the sum of the date number of December 31 of the previous year and the ordinal date, i.e., the day number, of the day and month in the given year. For example, the date number for August 1, 1996 is 144484 which is the sum of the date number 144270 for the last day of 1995 and 214, the ordinal date of the given date.

The date number for December 31 of any year is the number of days from January 1, 1601 to December 31 of that year, and is equal to 365 times the number of years plus the number of leap years in the interval. This latter number is the integer quotient of the number of years in the interval divided by 4, minus the integer quotient of this same number divided by 100 to adjust for all centennial years, plus the integer quotient of the number when divided by 400 to account for those centennial years which are leap years. Thus the number of leap years is the alternating sum of the appropriate quotients. Therefore, we may define the verb

```
YearEnd=. (365&* + -/ @ (div&4 100 400)) @ (] @ -&1601)
```

which gives the total number of days from January 1, 1601 up to and including December 31 of the year before the given year, and, for example, $\text{YearEnd } 1996$ is 144270. The ordinal date for any day of the year is the sum of the lengths of the preceding months in the year plus the day of the month and is given by

```
OD=. ({. @])++/@(pos@(<:@{1&{@}))) MonthLength"(0 _) {: @])
```

where

```
MonthLength=. (<:0[]) { 10}
```

gives the number of days in any given month and year. For example, 1 MonthLength 1996 is 31 and 2 MonthLength 1996 is 29, and so on, and 0D 1 8 1996 is 214. (The complement of the ordinal date, i.e., the number of days remaining in the year, is given by

```
ODcomp=. LY@({:0]) + 365&-0D
```

and, for example, ODcomp 1 8 1996 is 152.) The date number is then

```
D=. (YearEnd@({:0]) + OD
```

and D 1 8 1996 is 144484.

Month, quarter and year calendars

Let us number the days of the week from 0 to 6, inclusive, starting with Sunday as 0. Since we are numbering the days from January 1, 1601 which was a Monday, the day number for any date is the 7-residue of the date number, and we may define

```
mb=. mod&7 @ (D @ (1:,[,]))
```

to give the day number of the first day of any month, and, for example, 8 mb 1996 is 4 which corresponds to Thursday.

The numerical part of a month calendar consists of the consecutive positive integers to an upper limit determined by the month length arranged in a table of six rows since any month except February may extend over six consecutive weeks, and seven columns since there are seven days in the week, and beginning in the first row in a column determined by the first day of the month. This array may be constructed by first generating the appropriate list of positive integers as determined by the month length, converting to a one-column table, formatting to a three-column character table, extending this table to forty-two rows, rotating the rows by an amount given by the day number of the first day of the month, and finally reshaping into the appropriate table. These operations are performed by the verb

```
mctable=. ,_10(6 7&$)@((-0mb) |. {42&(. )@{3&":})@,.@pos0MonthLength
```

where the left and right arguments give the month and year numbers. To obtain the desired year calendar the twelve month calendars must be constructed each with column headings for the names of the days of the week and headed overall

with the month name, and finally displayed in one of several possible rectangular arrangements. The verb

```
mc=. (_12&{. @>@({&MONTHS)@<: @[]), (DAYS&, @mctable)
```

gives a labelled month calendar. The dyadic verb

```
cal=. <"2 @ mc"(0 _)
```

will give month calendars as determined by the left argument for the year given by the right argument. For example, (pos 3 4) cal 1997 gives a 1997 calendar arranged in three rows and four columns as the 2-cells of a table of shape 3 4 8 21.

The verb

```
Quarter=. ((pos 3)&+@((3: * <: )@[]))cal]
```

will give the month calendars for any quarter of any year, and, for example, 4 Quarter 1996 gives the October, November and December calendars for 1996. The verb

```
calendar=. (pos 4 3)&cal
```

gives a four-by-three calendar for the year specified by the right argument, and has been used to construct the 1997 calendar given in Appendix 1.

Julian Date and its Inverse

The Julian date is a system of reckoning used in astronomical calculations for harmonizing the various chronologies used in astronomy. It was proposed in 1582 by the philologist and historian Joseph Scaliger and is defined as the number of days after January 1, 4713 B.C. As an example, the Julian date for August 1, 1996 is 2450297. We shall give verbs to calculate the Julian date corresponding to any calendar date on or after January 1, 1601 — a very simple calculation — and conversely to calculate the calendar date corresponding to any given Julian date — a calculation which appears to be anything but simple.

Since the Julian date for December 31, 1600 is 2305813, the verb

```
JD=. 2305813&+@D
```

may be used to give the Julian date for a given calendar date, and JD 1 8 1996 is 2450297. The calculation of the inverse Julian date, i.e., the calendar date corresponding to a given Julian date, will be done by finding first the year from

the integral number of years from January 1, 1601 represented by the date number corresponding to the Julian date and then the month and day corresponding to the remaining number of days which represent the ordinal date in that year.

We shall first introduce the verb

$$\text{DIV} = \{ \} . @ [] ' ((\text{div } \{ . \} , (\text{mod } \{ . \} \text{ DIV } \{ . \}) @ \{ . \} * @ \# @])$$

which may be considered a generalization of the verb *div* for integer division and which will be used to factor the integer left argument according to the divisors given by the right argument. For example, 75 DIV 20 8 1 is 3 2 4, and 75 is equal to 3 2 4 + / . * 20 8 1. Now consider the expression

144484 DIV 146097 36524 1461 365 1

which has the value 0 3 23 3 214, where the left argument is D 1 8 1996, the date number for August 1, 1996, and the first four divisors are, respectively, the number of days in four hundred years, the number of days in one hundred years in which the included centennial year is not a leap year, the number of days in four years, and the number of days in one year which is not a leap year. Therefore, the expression

1601 + 400 100 4 1 + / . * 0 3 23 3

will give the year 1996 corresponding to the date number, while the last item 214 of the factorization is the ordinal date in that year.

The day and month corresponding to an ordinal date may be found by the verb

$$\text{ODinv} = \{ . @ > : @ (12 \ 31 \ \# :) @ (, @ (> @ \text{ODcal} @)) 1 . []$$

where

$$\text{ODcal} = \{ . (+ / \ @ (0 : ,) :) ([+ \text{pos} @] " (0 \ 0) \ \& . >)]] @ L$$

gives the ordinal calendar for a given year in the form of a 12- item list whose first item gives the ordinal dates for January, i.e., 1, 2, ..., 31, whose second item gives the ordinal dates for February, i.e., 32, 33, ..., 59 or 60, etc. For example, OD 1 8 1996 is 214, and 214 ODinv 1996 is 1 8.

The calculations given in the last two paragraphs may be used to find the calendar date for any date number except for those date numbers corresponding to December 30 or 31 of a leap year when the date is given as ordinal date 0 of the following year.

For example,

```
(D 30 12 1996) DIV 146097 36524 1461 365 1
```

has the value 0 3 23 4 0, and $1601 + 400 \cdot 100 \cdot 4 \cdot 1 + / \cdot \cdot 0 \cdot 3 \cdot 23 \cdot 4$ is 1997. The distinguishing feature of these exceptional cases is the number of trailing 0s in factorization of the date number, from one trailing zero in the above example to four for December 31 in a centennial leap year as in

```
(D 30 12 1996) DIV 146097 36524 1461 365 1
```

which has the value 1 0 0 0 0. We can resolve this problem using the number of trailing 0s to indicate that an adjustment of the ordinal date and year is necessary. Therefore, we define the verb

```
DayAndYear=. (((0:<|. )i.1:),(:,Year0:))@(DIV&146097 36524 1461 365 1)
```

where

```
Year=. 1601&+@(400 100 4 1&(+ / \cdot ))
```

and, for example,

```
DayAndYear D 1 8 1996
```

is 0 214 1996,

```
DayAndYear D 30 12 1996
```

is 1 0 1997, and

```
DayAndYear D 31 12 2000
```

4 0 2001. We may then define the verb

```
DayAndYearAdj=. ((. , '({366&-@{2&|@{.},<:@{(:)})@. (0:~: (. ))@DayAndYear
```

to adjust the year and ordinal date, and, for example, `DayAndYearAdj 30 12 1996` is 365 1996. Then the calendar date for *any* date number is given by

```
Dinv=. (((.0Dinv(1&{)) , }. )@DayAndYearAdj
```

and, finally, the inverse Julian date by

```
JDinv=. Dinv@(-&2305813)
```

For example, `JDinv 2450297` is 1 8 1996.

Miscellaneous Calculations

In this section we shall give a number of very simple and useful calculations which follow immediately from the calculation of either the date number or the Julian date or the inverse of either of these quantities.

It was noted above that the day number corresponding to any date is simply the 7-residue of the date number. Therefore, the day of the week is given by

`DayOfWeek=. ;@((_3<\DAYS)&({~})@ (mod&7 @ D))`

and, for example, `DayOfWeek 1 8 1996` is Th.

The number of days between two dates is the difference in their date numbers so that we may define the verb

`DaysBetween=. D@[ -- D@]`

and, for example, `1 8 1996 DaysBetween 25 12 1996` is equal to 146. The date a given number of days from any date is given by

`DaysFrom=. Dinv@([+D@])`

and `21 DaysFrom 29 8 1996` is `19 9 1996`.

One application of some of these verbs could be to find the approximate date of birth of a litter of kittens given the date that the mother cat conceived. Since the gestation period is approximately 64 days, a verb that will give an estimate of the date of birth is

`Kittens=. 64&DaysFrom`

and, for example, `Kittens 1 9 1996` is `4 11 1996`.

"A day for any date"

There are methods of finding the day of the week for any date which, unlike the method developed in this paper, do not depend upon the calculation of a sequence number such as the Julian date or the date number. One such method is due to Lewis Carroll who published it in the March 31, 1887 issue of *Nature*. In this section we shall give Carroll's step-by-step procedure supplemented by the corresponding J expressions which will be gathered together in an explicit verb for performing the calculations. In these expressions the J variables `Day`, `Month` and `Year` will give the values of the day, month and year of the given date.

Carroll's algorithm, as described in Fisher (1975), is as follows with additional commentary added in square brackets:

Take the given date in 4 portions, viz., the number of centuries, the number of years over, the month, the day of the month.

Compute the following 4 items, adding each, when found, to the total of the previous items. When an item or total exceeds 7, divide by 7, and keep the remainder only. [In the J calculations the 7-residue will be found only in the last step.]

The Century-Item: For Old Style (which ended September 2, 1752) subtract from 18. For New Style (which began September 14) divide by 4, take overplus from 3, multiply remainder by 2. [When the Gregorian calendar was introduced in England in 1752, the days between September 2 and September 14 were dropped. In the J expressions calculations for the Julian calendar are not considered.]

$c = . + : 3 - \text{mod} \& 4 \text{ div} \& 100 \text{ Year}$

The Year-Item: Add together the number of dozens, the overplus, and the number of 4s in the overplus.

$y = . ((\text{div} \& 4 +]) \& \text{mod} \& 12 + \text{div} \& 12) \text{mod} \& 100 \text{ Year}$

The Month-Item: If it begins or ends with a vowel, subtract the number, denoting its place in the year, from 10. This, plus its number of days, gives the item for the following month. The item for January is '0'; for February or March (the 3rd month), '3'; for December (the 12th month), '12'. [Fisher suggests that "it would be surprising if he [Carroll] did not eventually come round to formulating a relevant chart as follows to be committed to memory through a specific mnemonic": 0 3 3 6 8 4 6 2 5 0 3 5. The items represent the 7-residues of the total number of days from January 1 to the beginning of each month.]

$m = . (< : \text{Month}) \{ 0 \ 3 \ 3 \ 6 \ 8 \ 4 \ 6 \ 2 \ 5 \ 0 \ 3 \ 5$

The twelve-item list in the above expression may be calculated by

$7 | + / \backslash 0, \} : \text{LENGTHS}$

The Day-Item is the day of the month.

The total, thus reached, must be corrected, by deducting '1' (first adding 7, if the total be '0'), if the date be January or February in a Leap Year; remembering that every year, divisible by 4, is a Leap Year, excepting only the century-years, in New Style, when the number of centuries is not so divisible (e.g., 1800).

```
7|c + y + m + Day - (Month <: 2) *. (LY Year)
```

The final result gives the day of the week, '0' meaning Sunday, '1' Monday, and so on.

As an example Carroll uses the date September 18, 1783. Then in the J calculations Day is 18, Month is 9 and Year is 1783 so that c is 2, y is 19, m is 5 and the value of the last expression is 4. Therefore, the given date falls on a Thursday.

We may use the J expressions given above in the definition of the following explicit verb DayNumber:

```
DayNumber=. 3 : 0
('Day'; 'Month'; 'Year')=. y.
c=. +: 3 - mod&4 div&100 Year
y=. ((div&4 + J)@ mod&12 + div&12) mod&100 Year
m=. (<:Month){ 0 3 3 6 8 4 6 2 5 0 3 5
7|c + y + m + Day - LY Year
)
```

For example, DayNumber 18 9 1783 is 4.

Keith Smillie
Department of Computing Science
University of Alberta
Edmonton, Alberta T6G 2H1

smillie@cs.ualberta.ca
<http://www.cs.ualberta.ca/~smillie/>

References

- Burke, C., R. K. W. Hui, *et al*, 1996. *J Phrases*. Iverson Software Inc., Toronto.
- Fisher, J. (ed.), 1975. *The Magic of Lewis Carroll*. Penguin Books Ltd., Harmondsworth, Middlesex.
- McDonnell, E. E., 1991. *ISI & J*. Iverson Software Inc., Toronto.
- Smillie, Keith, 1992. "Making a calendar in J." *Vector*, vol. 9, no. 1, pp. 87 - 90.
- Smillie, Keith, 1994. "A note on the Easter algorithm in J." *Vector*, vol. 10, no. 3, pp. 78 - 79.

Appendix 1. Calendar for 1997

Jan							Feb							Mar							
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	
			1	2	3	4							1							1	
5	6	7	8	9	10	11	2	3	4	5	6	7	8	2	3	4	5	6	7	8	
12	13	14	15	16	17	18	9	10	11	12	13	14	15	9	10	11	12	13	14	15	
19	20	21	22	23	24	25	16	17	18	19	20	21	22	16	17	18	19	20	21	22	
26	27	28	29	30	31		23	24	25	26	27	28		23	24	25	26	27	28	29	
														30	31						
Apr							May							Jun							
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	
			1	2	3	4					1	2	3	1	2	3	4	5	6	7	
6	7	8	9	10	11	12	4	5	6	7	8	9	10	8	9	10	11	12	13	14	
13	14	15	16	17	18	19	11	12	13	14	15	16	17	15	16	17	18	19	20	21	
20	21	22	23	24	25	26	18	19	20	21	22	23	24	22	23	24	25	26	27	28	
27	28	29	30				25	26	27	28	29	30	31	29	30						
Jul							Aug							Sep							
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	
			1	2	3	4						1	2			1	2	3	4	5	6
6	7	8	9	10	11	12	3	4	5	6	7	8	9	7	8	9	10	11	12	13	
13	14	15	16	17	18	19	10	11	12	13	14	15	16	14	15	16	17	18	19	20	
20	21	22	23	24	25	26	17	18	19	20	21	22	23	21	22	23	24	25	26	27	
27	28	29	30	31			24	25	26	27	28	29	30	28	29	30					
							31														
Oct							Nov							Dec							
Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	
			1	2	3	4							1			1	2	3	4	5	6
5	6	7	8	9	10	11	2	3	4	5	6	7	8	7	8	9	10	11	12	13	
12	13	14	15	16	17	18	9	10	11	12	13	14	15	14	15	16	17	18	19	20	
19	20	21	22	23	24	25	16	17	18	19	20	21	22	21	22	23	24	25	26	27	
26	27	28	29	30	31		23	24	25	26	27	28	29	28	29	30	31				
							30														

Appendix 2. Script file

```
NB. Introduction
LY=. ~:/ @ (mod&4 100 400 = 0:)
mod=. |~
LENGTHS=. 31 28 31 30 31 30 31 31 30 31 30 31
L=. LENGTHS&+@((LENGTHS=28)&*. )@LY
DAYS=. ' Su Mo Tu We Th Fr Sa'
MONTHS=. _3<\ 'JanFebMarAprMayJunJulAugSepOctNovDec'
div=.<.@%
pos=. >: @ 1.
```

```

NB. Date number
YearEnd=. (365&* + -/ @ (div&4 100 400)) @ (] @ -&1601)
MonthLength=. (<:@[) { L@]
OD=. ({.@])++/ @ (pos @ (<: @ (1&{.@]) MonthLength"(0 _) {:@])
ODcomp=. LY@(:@]) + 365&-@OD
D=. (YearEnd@(:@]) + OD

```

```

NB. Month, quarter and year calendars
mb=. mod&7 @ (D @ (1:,[,]))
mctable=. ,"-_1@(6 7&$)@({-@mb) [.
(42&{.)@({3&":)@(.@pos@MonthLength)
mc=. (_12&{.@>@({&MONTHS)@<:@[),(DAY$&,@mctable)
cal=. <"2 @ mc"(0 _)
Quarter=. ((pos 3)&+@({3: * <:.)@[)cal]
calendar=. (pos 4 3)&cal

```

```

NB. Julian date and its inverse
JD=. 2305813&+@D
ODcal=. (+/\@{0:,}) (([+pos@])"(0 0) &. >) ]@L
ODinv=. {.@>:@(12 31&#:)@(>@ODcal@]) i. [
DIV=. ({.@[)'`((div {.), (mod {.) DIV ).@])@.( * @#@])
Year=. 1601&+@{400 100 4 1&(+/. *})
DayAndYear=. (({0:<{.)i.1:, {:,Year@:.)@ (DIV&146097 36524 1461
365 1)
DayAndYearAdj=. (({.)'`{(366&-@{2&{.@{.) ,
<:@{:@})@.(0:~: {.)@DayAndYear
Dinv=. (({.)ODinv(1&{.) , ).@DayAndYearAdj
JDinv=. Dinv@(-&2305813)

```

```

NB. Miscellaneous calculations
DayOfWeek=. ;@(_3<\DAY$)&({~)@{mod&7 @D})
DaysBetween=. D@[ -- D@]
DaysFrom=. Dinv@([+D@])
Kittens=. 64&DaysFrom

```

```

NB. "A day for any date"
DayNumber=. 3 : 0
('Day';'Month';'Year')=. y.
c=. +: 3 - mod&4 div&100 Year
y=. ((div&4 + ])@ mod&12 + div&12) mod&100 Year
m=. (<:Month){ 0 3 3 6 8 4 6 2 5 0 3 5
7|c + y + m + Day - LY Year
)

```

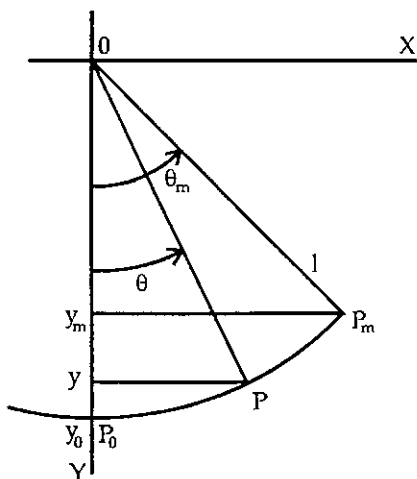
Swinging with APL: The Simple Pendulum

by Joseph De Kerf (Reprinted from APL-CAM Journal, Vol.18 No.1)

Abstract

Calculation of time t as a function of the declination θ of the simple pendulum with length l is reduced to the calculation of an incomplete elliptic integral of the first kind $K(\phi;p)$. For the period T this is reduced to the calculation of a complete elliptic integral of the first kind $K(p)$. Algorithms, based on the common mean concept, are illustrated with APL user-defined functions. Examples are given.

Introduction



In a previous paper [1], we have shown that the rectification of a lot of so-called closed curves is reduced to the calculation of elliptic integrals. The elliptic integrals were evaluated with algorithms based on the common mean concept [2], as such algorithms are much more efficient in cpu time than those based on gaussian quadrature or series expansion. Implementation was illustrated with APL user-defined functions, calling the user-defined functions described in [3] and [4]. In this paper, we follow the same line of thought for the solution of the simple or plane pendulum: the calculation of time t as a function of the declination θ [5].

The Simple Pendulum

The simple or plane pendulum consists of a mass point m (P) at one end of a massless rigid rod with length l , pivoted at the other end (0) and placed in a uniform gravitational field g . Let P (declination θ) be moving from the

equilibrium position P_0 (declination 0) towards its maximum displacement P_m (maximum declination or amplitude θ_m).

Time versus Declination

Kinetic energy of the system is: $\frac{1}{2}mv^2$ with

$$v = \frac{ds}{dt} \quad \text{and} \quad s = l\theta \left(v = l \frac{d\theta}{dt} \right)$$

Potential energy of the system is: $mg l (1 - \cos \theta)$

$$\text{as } y_0 = l \quad \text{and} \quad y = l \cos \theta$$

Conservation of energy gives: $\frac{1}{2}mv^2 + mg l (1 - \cos \theta) = mg l (1 - \cos \theta_m)$

$$\frac{1}{2}mv^2 = mg l (\cos \theta - \cos \theta_m)$$

$$v = \sqrt{2gl (\cos \theta - \cos \theta_m)}$$

Substituting v by $l \frac{d\theta}{dt}$ we get:

$$\frac{d\theta}{dt} = \sqrt{\frac{2g}{l} (\cos \theta - \cos \theta_m)}$$

or:

$$dt = \sqrt{\frac{l}{2g}} \frac{d\theta}{\sqrt{\cos \theta - \cos \theta_m}}$$

Setting the time at the equilibrium point $t_0 = 0$, time t for declination θ becomes:

$$t = \sqrt{\frac{l}{2g}} \int_0^\theta \frac{d\theta}{\sqrt{\cos \theta - \cos \theta_m}}$$

Substituting:

$$\sin \frac{\theta}{2} = p \sin \phi \quad \text{with} \quad p = \sin(\theta_m / 2)$$

$$d\theta = \frac{2p \cos \phi}{\sqrt{1 - p^2 \sin^2 \phi}} d\phi$$

we have:

$$\begin{aligned} \sqrt{\cos \theta - \cos \theta_m} &= \sqrt{2(\sin^2(\theta_m / 2) - \sin^2(\theta / 2))} \\ &= \sqrt{2} p \cos \phi \end{aligned}$$

such that:

$$t = \sqrt{\frac{l}{g}} \int_0^{\phi} \frac{d\phi}{\sqrt{1-p^2 \sin^2 \phi}} = \sqrt{\frac{l}{g}} K(\phi; p)$$

$K(\phi; p)$ being the incomplete elliptic integral of the first kind with

$$\phi = \arcsin\left(\frac{\sin(\theta/2)}{p}\right) \text{ and } p = \sin(\theta_m/2).$$

APL Function

An APL user-defined function *TIME* for calculating time t as a function of the declination θ may be:

```

vTIME[[]]v
vT←LA TIME D;F;P
[1] F←((1+LA)÷980.6294)*0.5
[2] P←100(1+LA)÷360
[3] T←F×(10(100D÷360)÷P) IEI1 P
v

```

with: LA being the catenation of the length l (cm) and the maximum declination or amplitude θ_m (degrees°)

D the declination θ (degrees°)

P the parameter $p = \sin(\theta_m/2)$

F the factor $\sqrt{l/g}$

IEI1 the user-defined function described in [4]

and geodetic constant g being set to $980.6294 \text{ cm/sec}^2$ (Hayford latitude 45°) [6].

Example: $l = 100 \text{ cm}$

$$\theta_m = 60^\circ$$

$$\theta = 10^\circ(10^\circ)60^\circ$$

```

LA←100 60
D←10×16
2 0 12 6 v D, >(60<LA) TIME" D
10 0.056021
20 0.113851
30 0.175831
40 0.245926
50 0.333733
60 0.538320

```

Note:

The time to go from position P_0 to maximum displacement P_m ($\theta = \theta_m$ or $\phi = \pi/2$) is one-fourth of the period T . Therefore, the period of the pendulum is:

$$T = 4 \sqrt{\frac{l}{g}} \int_0^{\pi/2} \frac{d\phi}{\sqrt{1 - p^2 \sin^2 \phi}}$$

$$= 4 \sqrt{\frac{l}{g}} K(p)$$

with $p = \sin(\theta_m / 2)$ and $K(p)$ being the complete elliptic integral of the first kind.

The period T may be calculated with the function *TIME*, setting the right argument to the maximum declination or amplitude θ_m , and multiplying the result by 4. An APL user-defined function *PERIOD* for calculating the period directly from the complete elliptic integral of the first kind, more efficient in cpu time, may be:

```

      ∇PERIOD[ ]∇
      ∇T←L PERIOD A;F;P
      [1] F←(L÷980.6294)*0.5
      [2] P←100A÷360
      [3] T←4×F×CEI1 P
      ∇

```

with: L being the length l (cm)

A the maximum declination of amplitude θ_m (degrees °)

CEI1 the user-defined function described in [3]

and the geodetic constant g being set to 980.6294cm/sec² (Hayford latitude 45°) [6]. The period T is returned in seconds.

Example: $l = 100\text{cm}$
 $\theta_m = 60^\circ$

```

      8 6 * 4×100 60 TIME 60
2.153281
      8 6 * 100 PERIOD 60
2.153281

```

Joseph De Kerf
 Rooienberg 72
 B-2570 Duffel
 Belgium

References

1. J. De Kerf: *Rectification with APL*; APL-CAM Journal, Vol. 17, No. 4, 16th December 1995 pp. 560 - 582.
2. J. De Kerf: *The Common Mean and APL*; APL-CAM Journal, Vol. 16, No. 3, 16th July 1994, pp. 370 - 372.
See also: Vector, Vol. 11, No. 3, January 1995, pp. 21 - 23.
3. J. De Kerf: *The Complete Elliptic Integrals and APL*; APL-CAM Journal, Vol. 16, No. 4, 16th October 1994, pp. 631 - 635.
See also: Vector, Vol. 12, No. 1, July 1995, pp. 102 - 106.
4. J. De Kerf: *The Incomplete Elliptic Integrals and APL - An Extension*; APL-CAM Journal, Vol. 17, No. 3, 16th September 1995, pp. 455 - 460.
See also: Vector, Vol. 12, No. 2, October 1995, pp. 95 - 100.
5. H.C. Corben and P. Stehle: *Classical Mechanics*; John Wiley & Sons, New York, 1950.
6. M. Abramowitz and L.A. Stegun (eds.): *Handbook of Mathematical Functions*; Applied Mathematics Series No. 55; National Bureau of Standards, Washington D.C., 1964 (and subsequent revisions).

Appendix

For convenience of the reader, the APL user-defined functions *CEI1* [3] and *IEI1* [4], referred to in this paper, are reproduced here:

```

      vCEI1[[]]v
      vR←CEI1 P
[1]  R←1,(1-P*2)*0.5
[2]  LAB:←(÷/R+(0.5×+/R),(×/R)*0.5)/LAB
[3]  R←(01)÷+/R
      v

```

```

      vIEI1[[]]v
      vR←F IEI1 P;I;A;B
[1]  +((1÷|P)÷(0÷2)≤|F)/LAB1
[2]  +0,R←300.5×F+0.5
[3]  LAB1:R←(2+I+1),÷÷(1-P*2)*0.5
[4]  LAB2:A←(÷/R)×10F*2*1+I+1
[5]  B←2×+/R×(2-10F*2*I)*2
[6]  F←F-(30A÷B)÷2*I+1
[7]  +((÷/R+(0.5×+/R),(×/R)*0.5)/LAB2
[8]  R←F÷0.5×+/R
      v

```

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner:	Adam Weber	134
A Word (or Two) from our Spenser	Norman Thomson	135
APL2 and J (2) - Indexing	Chris Burke	137
At Play with J: Volutes	Gene McDonnell	144
Extended Integers in J	Roger Hui	154

Hackers' Corner: APL Characters and Windows 95

by Adam Weber

With the arrival of Windows 95, I took a leap of faith and installed it on my machine shortly after its release. Sure enough, it worked first time, and found all my hardware etc. But how would it run my DOS apps?

I still use APL*PLUS PC v.11, and had installed it so that it would run under Windows 3.1. The problem was that after upgrading to Windows 95, I was not able to see APL characters when running in a window. After a little messing around I found the answer in the Properties option in the APL window!

There are several font sizes that DO correctly display the APL character set. These are:

8 x 14

8 x 16

9 x 11

10 x 23

My APL sessions are done in 80 chars, 25 lines, and the above fonts might not work in other configurations. TrueType fonts do not work at all.

In short, if you can't see proper APL characters in a window under Windows 95, simply try the different font sizes in the properties of the APL window and you might just be in luck! If all else fails, then only the full screen will display the proper characters.

As for printing the characters on an HP Deskjet — no chance!!!

A Word (or Two) from our Spenser

by Norman Thomson

My friend Bill McLean from across the western seas (I refer of course to the Clyde, not the Atlantic) recently drew my attention to a script from John Baker in Kingston, Ontario (one small watery step westwards!) which obtains word lists and concordances using J.

I have adapted this in order to demonstrate how simply this can be done using J. Files which scholars might apply these techniques to in earnest would be of considerable size. For illustrative purposes a demonstration file is printed below which contains a text fragment from Spenser:

... to lose good days that might be better spent;
to waste long nights in pensive discontent;
to speed today, to be put back to-morrow;
to feed on hope, to pine with fear and sorrow;
to fret thy soul with crosses and with cares;
to eat thy heart through comfortless despairs ...

Here is the concordancing verb set:

```
pc=: '.,;':!?'()/~#',10 13(a. NB. punctuation characters plus crlf
index=.e.~#i.@#0] NB. returns y.-indices of chars in x.
prepare =: 3 : 0 NB. makes punctuation-free form of text
y.=.' '(pc#index y.))y. NB. replace punctuation with blanks
y.=.(] #~ [: -. ' '&E.)' 'y. NB. remove multiple blanks
y. ).~ - ' ' = {; y. NB. remove trailing blanks
)
wordlist =: prepare@{1!:1)@< NB. reads and prepares text file
words =. /:~ @{];._1) NB. words as matrix in alphabetical order
types =. (~:~#)]@words NB. words with duplicates removed
freqs =. ((#;.1)i.@#)@ -:@words NB. word frequencies in order of occurrence
fwords =. \:@freqs{types NB. words in order of frequency
tab=.. fwords;.,.@ \:~ @freqs NB. word frequency table (monadic)
ktab=. ({.fwords@];){[...@\:~@freqs@] NB. first x. words in tab in
```

Here are some comments on interesting features of J which it highlights:

`words` .. the parentheses contain an example of the conjunction "cut" which joins a verb (in this case, simply `]`), to a code which says where to find the cut delimiter (1=head, 2=tail), and whose sign says whether to include the delimiter if positive, or exclude it if negative. To the left of `@`, `/:` with both(`~`) produces an alphabetical ordering.

types . . uses "nub sieve" (~:) with copy (#) to remove duplicates.

freqs . . "cut" with another verb (#); delimiter is 0, that is the first item in the result of 1., so this cut counts numbers between zeros (including the 0s themselves).

fwords . implied bracketing is (\:@freqs)(types, i.e. grade down of frequencies is applied to types.

tab . . . grade down is applied also to freqs.

ktab . . . selects the x. most frequently occurring words.

The functions of the various verbs are best illustrated by a simple example:

```
x=. 'to be, or .... not to be!'
```

```
]x1=.prepare x
to be or not to be
```

```
words x1
```

```
types x1
```

```
fwords x1
```

```
tab x1
```

```
3 ktab x1
```

```
be
be
not
or
to
to
```

```
be
not
or
to
```

```
freqs x1
2 1 1 2
```

```
be
to
not
or
```

be	2
to	2
not	1
or	1

be	2
to	2
not	1

Now go to the Spenser file, and obtain a count of the most frequently occurring words:

```
4 ktab wordlist 'c:\jfw\spens'
```

to	9
with	3
and	2
be	2

APL and J (2) - Indexing

by Chris Burke

This series of articles entitled *APL and J* will try to explain the J way of thinking, by contrasting APL and J coding examples. The term APL will be used generically to refer to facilities found in most commercial implementations of APL.

Here we look at how J handles indexing and indexed replacement.

Housekeeping

We use the letters *V*, *M*, and *A* for vectors, matrices and arrays of any dimension, optionally followed by numbers to denote their shape. Thus *V* for any vector, *V*5 for a vector of length 5, *M*45 for a 4-by-5 matrix, *A*245 for a 2-by-4-by-5 array. Where actual values are required, *V*5 will be defined as 1 5, *M*45 as 4 5 p 120, for example:

```

      M45
0  1  2  3  4
5  6  7  8  9
10 11 12 13 14
15 16 17 18 19

```

APL terminology will be used, except where J terminology is required for clarity. Since J uses origin 0 only, here $\square i o \leftarrow 0$ is assumed throughout.

Indexing

In APL, you index variables using an indexing expression in brackets. Such expressions are not supported in J; instead, J uses a function *{ from*. Compare the following expressions to select row 2 of a matrix:

```

M[2;]      APL
2{M        J

```

To understand this change, note that bracket indexing in APL does not conform to the rest of the APL language. Indexing is essentially a dyadic function taking arguments of the data and indices, yet the APL expression does not have the form of other APL dyadic functions (i.e. a function with arguments to the left and right).

Moreover you cannot use the APL expression as you would a normal function. For example if *N* is an enclosed array, each item of which is a matrix, you could not use bracket indexing to select row 2 of each item:

```
N '' [2;]
SYNTAX ERROR
```

If you wanted to do this, you could define a cover function to index a matrix:

```
v r+ndx rowndx mat
[1] r+mat[ndx;]
v
```

You could then select row 2 of *M* as:

```
2 rowndx M
```

and row 2 of each element of *N* as:

```
2 rowndx'' N
```

The corresponding J expressions are:

```
2 { M
2 {each N
```

It may be seen that *{* is simply a functional form of bracket indexing. Because it is an ordinary function, you can use it in functional expressions, as in the example above. The idea of *{* is similar, but not identical, to the index function *[]* in APL2.

Note that in the above example, should you wish to index row 2 of a 3D array, you would need to modify *rowndx* accordingly, but the J expression would remain unchanged.

Here are a couple more examples:

```
item2=. 2&{                                function to return item 2
```

```
item2 M45
10 11 12 13 14
```

```
item2 A324
16 17 18 19
20 21 22 23
```

```

mrow=. {&M45           function to return rows of M45

mrow 3 2
15 16 17 18 19
10 11 12 13 14

```

From Left Argument

The left argument can take various forms:

1. It can be one or more integers, which select corresponding items from the right argument. Negative integers select from the end, for example:

```

      2 _1 { M45           row 2 and last row of M45
10 11 12 13 14
15 16 17 18 19

```

It is often convenient to apply rank with this form, for example:

```

      2 _1 {~1 M45         column 2 and last column of M45
      2 4
      7 9
12 14
17 19

```

2. The left argument can be one or more boxed lists, each element of which is a list of integers that select from corresponding axes:

```

      M45[1;3]             APL
8
      (<1 3){M45          J
8

```

This form allows scattered indexing:

```

      M45[(1 3)(2 4)]     APL2 and Dyalog APL
8 14
      (1 3;2 4){M45       J
8 14

```

3. The boxed lists in the left argument may themselves contain boxed lists, each element again selecting from corresponding axes:

```

      M45[1;3 4]           APL
8 9
      (<1;3 4){M45        J
8 9

```

Note that in J, this last form permits scattered indexing where each index may have a different number of elements. The following example has two indices: the first picks row 1, columns 3 and 4; the second picks row 2, column 3. The two results are assembled into a matrix, padding the second row with a fill of 0:

```
((1;3 4);2 3){M45
 8 9
13 0
```

Selecting Entire Axis

In APL, when an axis is elided, the entire axis is selected. This is not the same as using an empty index:

```
M45[ ;2]                pick all rows, column 2
2 7 12 17
```

```
M45[ '' ;2]             pick no rows, column 2
```

In J, the elided axis is given by the boxed empty vector. Thus:

```
(<(<' ');2){M45         pick all rows, column 2
2 7 12 17
```

```
(<' ' ;2){M45           pick no rows, column 2
```

Here, it is convenient to use `a:` *ace*, which is the boxed empty vector:

```
(<a: ;2){M45            pick all rows, column 2
2 7 12 17
```

In general, a boxed argument used as an index selects all but the indices given. So, the boxed empty means select all but the empty, i.e. select all. The following picks all but the first and last row from columns 2 and 3:

```
(<(<0 _1);2 3){M45
 7 8
12 13
```

Indexed Replacement

As with indexing, indexed replacement in APL does not conform to the rest of the language. It requires three arguments – a set of indices, and two sets of data, at least one of which must be named. The named data item and the indices are given to the left of assignment, and the other data item to the right:

$M[2;] \leftarrow V$ replace row 2 of M with V

Indexed replacement is not a functional expression. The “result” of indexing is the values assigned (V in the above example), so that M is updated as a side effect.

In contrast, indexed replacement in J is an ordinary part of the language. It has two steps:

- the indices to be replaced are given as the argument to the adverb *} amend*, returning a dyadic function that merges its two arguments according to the indices.
- the function is then applied to the two data arguments. The elements of the left argument replace the elements of the right argument according to the indices.

For example, to replace row 2 of M with V:

$V\ 2\} M$

The order of execution is shown by:

$V\ (2)\} M$ $2\}$ is a function that merges item 2

To replace row 2 of each item of a boxed list of matrices:

$(\leftarrow V)\ 2\} \text{ each } N$

Note that indexed replacement in J does not require assignment and does not update the arguments, so there is no need for a named argument. The following updates M:

$M =. V\ 2\} M$

Arguments To Amend

The index arguments to } are symmetrical with index arguments to {. For example:

```
      2{M45                                select row 2
10 11 12 13 14
```

```
      (100+1.5) 2} M45                      amend row 2
0   1   2   3   4
5   6   7   8   9
0 100 200 300 400
15 16 17 18 19
```

```
      (<1;1 2 3){M45                        select row 1 columns 1 2 3
6 7 8
```

```
      100 200 300 (<1;1 2 3)} M45           amend row 1 columns 1 2 3
0   1   2   3   4
5 100 200 300 9
10 11 12 13 14
15 16 17 18 19
```

Selective Assignment

Modern APLs can replace a portion of an array selected by an expression given to the left of assignment, this is called selective assignment. For example, the following replaces all occurrences of 2 in V by 20:

```
V←1 2 2 3 1 2
((V=2)/V) ← 20
V
1 20 20 3 1 20
```

Similar facilities in J are provided by using a function argument to amend, which function is used to create the required indices. Thus:

```
      bx 1 0 1 1 0 1 1                      the utility bx returns indices of a boolean
0 2 3 5 6

      f=. bx @ (2:=])                       f returns indices where right argument is 2

      20 f} 1 2 2 3 1 2                     use f to replace 2 by 20
1 20 20 3 1 20
```

Fetch

A new function in J3.02 is `{: : fetch` which when used dyadically as in `x fetch y` returns the subarray of `y` according to the path `x`; the selection at each level is based on `{`, for example:

[A=. 1 2 3;4 5;i.4 5

1	2	3	4	5	0	1	2	3	4
					5	6	7	8	9
					10	11	12	13	14
					15	16	17	18	19

`(2;_1 _1) fetch A` fetch from item 2, the element in the last row
19 and column

`(2;<2;2 3 4) fetch A` fetch from item 2, the elements indexed by 2;2 3 4
12 13 14

This is similar to the *reach* facility of Dyalog APL, except that in J the final selection need not be a scalar:

`A[(2(2 2))(2(2 3))(2(2 4))]`
12 13 14

Reference

Iverson, K.E. *J Introduction and Dictionary*, Iverson Software, 1996

At Play with J — Volutes

by Eugene McDonnell (*eemcd@aol.com*)

This article describes an amazing algorithm that I learned from Joey Tuttle. It produces an integer volute. I call it amazing on good evidence, because I was amazed when he first showed it to me. I had written a function to produce such volutes many years ago [McDonnell, E. E., 'Spirals & Time', *APL QuoteQuad* 7 4, Winter 1977], and thought I had done a fairly efficient job, but Tuttle's analysis was far superior.

An integer volute can be drawn in a variety of ways. All of the cases we'll consider place the integers in the cells of a rectangular, and usually square, table. Two kinds of volutes can be drawn in this manner: an involute and an evolute. In an involute the smallest integer appears in a corner of the table, and the integers increase as they get closer to the centre. In an evolute the largest number appears in a corner of the table, and the integers decrease as they get closer to the centre of the table.

0	1	2	3	4	24	23	22	21	20
15	16	17	18	5	9	8	7	6	19
14	23	24	19	6	10	1	0	5	18
13	22	21	20	7	11	2	3	4	17
12	11	10	9	8	12	13	14	15	16
involute					evolute				

In the involute above the numbers increase in a clockwise rotation. They could just as easily have increased in a counter-clockwise rotation. The number 0 appears in the top left corner of the table, but it could just as well have appeared in any of the other three corners. There are then eight possible ways of drawing the involute: with 0 appearing in any of four corners, and the numbers increasing in a clockwise or counter-clockwise rotation. Similar remarks apply to the evolute, *mutatis mutandis*. Furthermore, either form can be derived from the other by applying the verb `24⊖` to it. The least number is 0 in these volutes. A volute having 1 as the least number can be obtained from one of these by adding 1 to it.

Our verbs will take as argument the length of the side of the square table. For example, if `e` is our evolute verb,

```
e 3
8 7 6
1 0 5
2 3 4
```

I'll give six solutions to the problem, each one increasing in speed. The best is a variation of the marvellous technique shown me by Tuttle.

In the book *Concrete Mathematics* by Graham, Knuth, and Patashnik, Exercise 3.40 takes a scalar approach to the problem of constructing an evolute.

It assumes an x,y coordinate system, with 0 at coordinates (0,0). It gives two ways of arriving at a solution, problems a and b. Both solutions produce a bottom right clockwise volute.

In problem a, it squares (*) its argument, then produces the list of that many consecutive integers, beginning with 0. It finds the x and y coordinates of each number (GKPax, .GKPay), upgrades the resulting two-column table (/:), then reshapes (\$) this upgrade list into a square (,~) table. For example, for a square of side 3, problem a takes $n=.1.*:3$ and yields x and y , upgrades this, and reshapes the upgrade:

```
In=.1.*:3
0 1 2 3 4 5 6 7 8
]xy=. (GKPax,.GKPay) n
0 0
0 1
_1 1
_1 0
_1 _1
0 _1
1 _1
1 0
1 1
]w=./:xy
4 3 2 5 0 1 6 7 8
(,~3)$w
4 3 2
5 0 1
6 7 8
```

In problem a the placement of each integer requires the evaluation of two functions, GKPax to give the x -coordinate, and GKPay to give the y -coordinate. A function evGKPa to give a square evolute of a given order is:

```
GKPae =. 0:=2&|      NB. GKPae y=1 if y is even
GKPao =. 1:=2&|      NB. GKPao y=1 if y is odd
GKPaq =. <.@+:@%:    NB. floor double sqrt
GKPam =. <.@%:       NB. m is floor sqrt
GKPal =. >.@-:@GKPam  NB. ceiling half m
GKPar =. _1:^GKPam    NB. parity (1 or _1)
GKPat =. {>:@GKPam    NB. m*(1+m)
GKPax =. GKPar*((]-GKPat)*GKPae@GKPaq)+GKPal
GKPay =. GKPar*((]-GKPat)*GKPao@GKPaq)-GKPal
evGKPa =. ,~ $ /:@(GKPax ,. GKPay)@i.@*:
```

In problem b the table of x-y indices is used to produce the integers, one at a time.

```
GKPB0 =. +:@(>./"1)@!
GKPB1 =. >@, @({@(;~)@(>.@-:@- + i.)
GKPB2 =. _1: ^ </"1
GKPB3 =. *:@[ + GKPB2@] * [ + +/"1@]
GKPB4 =. (GKPB0 GKPB3 ])@GKPB1
evGKPB =. ,~GKPB4
```

GKPB1 produces the 2-column table of x, y coordinates. GKPB0 gives a list of the doubles (+) of the maximum over (>./) the rows ("1) of the magnitudes (!) of the items in the table. GKPB2 produces a list where the items are _1 where in the corresponding row x is less than y , and 1 otherwise. GKPB3 squares its left argument, and adds this to the product of the _1 1 list with the sum of the left argument and the sum of the rows of the right argument. GKPB4 supplies the appropriate left and right arguments to GKPB3. The verb evGKPB reshapes this list, giving the same result as evGKPa.

For example:

```
]xy=.GKPB1 3
 _1 _1
 _1 0
 _1 1
 0 _1
 0 0
 0 1
 1 _1
 1 0
 1 1
]ns=(GKPB0 GKPB3 ]) xy
4 3 2 5 0 1 6 7 8
(,~3) $ ns
4 3 2
5 0 1
6 7 8
```

In *Vector* 11 4, Keith Smillie gives a suite of functions to produce a square evolute (I've replaced his 'rows' function by #, made it 0-origin, and abbreviated his names QtrTurn and Wind and Spiral). It produces its result by successive windings of new layers onto the beginning empty table.

The function Wd is, you will note, recursive:

```
QT=. [ , "2 |.@|:@]
Wd=. [^(((#@[{.]) QT [) Wd #@[}.])@.(0:<#@[)
Sp=. (i. @ ((-/ @ ]) , 0:)) W i.@(*/)@]
evKS=. (i.@(0:,0:)) Wd i.@*:@]
```

Smillie's volute is top right clockwise and has the advantage of not being limited to square results. You can find the details of the algorithm in his *Vector* article.

In my 1977 article I give a function like Smillie's in achieving its result by winding, but iterative (^:) rather than recursive. The function `evEEM0` below reverses and transposes (|: @|.) its argument (thus giving it a clockwise quarter turn), then appends as a new bottom row a vector of integers (i.) as long as the number of rows in the argument (#), with its first item the number of items in the argument (*/@\$). This is initiated with an empty table (i. 0 0), and is repeated double (+:) the argument times. It is bottom right counter-clockwise.

```
evEEM0=.:@|. , */@$ + i.@#
evEEM1=.:i.0 0"_[] NB. constant empty table
evEEM=.evEEM0^:(+:`evEEM1)
```

I sent an early draft of this paper to Roger Hui for comments, and he replied:

"Motivated by your comment on +/\^: _1, I arrived at the following solution after studying the results of

```
f=: +/\^: _1@evJKT
```

It is more direct but less concise than your solution; it takes less space, and is faster for n greater than about 200."

Here is the kind of thing Roger saw:

```
f 5
24 23 22 21 20
_15 _15 _15 _15 _1
1 _7 _7 _1 _1
1 1 3 _1 _1
1 11 11 11 _1
```

```
f 7
48 47 46 45 44 43 42
_23 _23 _23 _23 _23 _23 _1
1 _15 _15 _15 _15 _1 _1
1 1 _7 _7 _1 _1 _1
1 1 1 3 _1 _1 _1
1 1 11 11 11 _1 _1
1 19 19 19 19 19 _1
```


primitive and derived verbs, using the power conjunction to the minus-1 power ($\wedge: -1$):

```
evJKT 5
24 23 22 21 20
 9  8  7  6 19
10  1  0  5 18
11  2  3  4 17
12 13 14 15 16
```

It seems to me that Joey must have had a series of insights. I assume that his first insight was to ravel this:

```
Jq=.,evJKT 5
24 23 22 21 20 9 8 7 6 19 10 1 0 5 18 11 2 3 4 17 12 13 14 15 16
```

I think he then had the tremendous insight that this list came about as the result of an upgrade. We'll get p , the permutation inverse to q , easily:

```
Jp=.:^:_1 q
12 11 16 17 18 13 8 7 6 5 10 15 20 21 22 23 24 19 14 9 4 3 2 1 0
```

(Since upgrade is self inverse, we could have got p from q even more easily by writing $p=.:q$ — but we're not assuming that all readers will know this fact about upgrade.)

Now the final stupendous insight was to assume that p was produced by a sum scan of some list d , that is

```
p=.:+/\ d
```

We can determine what the d was that gave us p by applying the inverse of sum scan:

```
Jd=.:+/\^:_1 p
12 _1 5 1 1 _5 _5 _1 _1 _1 5 5 5 1 1 1 1 _5 _5 _5 _5 _1 _1 _1 _1
```

This is beginning to look promising. At this point we can write the overall function `evJKT`:

```
evJKT =. ,~ $ /: @ (+/\) @ evJKT2
```

This `sumscans` `(+/\)` the result of `evJKT2`, upgrades `(/:`) it, and reshapes `($)` the upgrade into a square `(.~)` table.

All this is great art — the rest is carpentry. Forget the leading 12 for the moment. If we do, we see that the other items all have the magnitude 1 or 5, and alternate

in sign: first `_1` and `5`, then `1` and `_5`, and so on in alternation. There are nine groups of `_1 5 1 _5`, used cyclically. We note also that the groups increase in count: one each of `_1` and `5`, two each of `1 _5`, three each of `_1` and `5`, then four each of `1 _5`, and lastly an anomalous four of `_1`. We can generate the list of one each of the nine values easily:

```
evJKT1=, <: @+: $ _1: , ] , 1: , -
```

The phrase `_1: , ] , 1: , -` when applied to its argument gives us the right argument to the reshape verb:

```
(_1: , ] , 1: , -) 5
_1 5 1 _5
```

and the phrase `<: @+:` gives us the left argument:

```
(<: @+: ) 5
9
```

so that the whole phrase gives us the list of values to be replicated:

```
(<: @+: $ _1: , ] , 1: , -) 5
_1 5 1 _5 _1 5 1 _5 _1
```

Now we need to specify how many times each of these is to be replicated:

```
evJKT0=, }: @ (2: # >: @i.)
```

This verb begins by giving us a list of integers:

```
i. 5
0 1 2 3 4
```

adds one to this:

```
>: i. 5
1 2 3 4 5
```

gives us two of each:

```
2 # 1 2 3 4 5
1 1 2 2 3 3 4 4 5 5
```

and curtails `():` this list:

```
): 2 # 1 2 3 4 5
1 1 2 2 3 3 4 4 5
```

The last item should be a 4, not a 5, but as you shall see, we'll find it useful to produce an extra item.

The verb `evJKT0` encapsulates the whole:

```
evJKT0 5
1 1 2 2 3 3 4 4 5
```

Now we can write `evJKT2`:

```
evJKT2=. _1&|. @ (evJKT0 # evJKT1)
```

The expression in parenthesis does the main job:

```
(evJKT0 # evJKT1) 5
_1 5 1 1 _5 _5 _1 _1 _1 5 5 1 1 1 1 _5 _5 _5 _5 _1 _1 _1 _1 _1
```

There is an extra `_1` at the end, but the next expression rotates this list one to the right, moving the extra `_1` to the beginning of the list:

```
_1&|. (evJKT0 # evJKT1) 5
_1 _1 5 1 1 _5 _5 _1 _1 _1 5 5 1 1 1 1 _5 _5 _5 _5 _1 _1 _1 _1
```

Why is the last `_1` rotated to become the leading item? A little thought will convince you that the value of the first item doesn't have to be 12; in fact its value is irrelevant — it could be any number at all! It serves only to act as a base for the ensuing `sumscan`, which, in turn, serves as the argument to `upgrade`. The `upgrade` will give the same result no matter what the value of the first item is; only the relative values of the items are important. And, since we are going to discard the last item of the list and then prefix the list with an arbitrary value, it suffices to combine these operations by performing the `_1` (right) rotate of the list.

The verb `evJKT` provides the finishing touches to the result of `evJKT2`:

```
evJKT=. (,~ $ /: @ (+/\) @ evJKT2)
```

It uses `evJKT2` to produce the list which it then `sumscans`:

```
+/\ evJKT2 5
_1 _2 3 4 5 0 _5 _6 _7 _8 _3 2 7 8 9 10 11 6 1 _4 _9 _10 _11 _12 _13
```

Notice that the items of this list are distinct and include all 25 integers from `_13` through `11`.

The upgrade of this list is obtained:

```
(/:@(+/\)@evJKT2) 5
24 23 22 21 20 9 8 7 6 19 10 1 0 5 18 11 2 3 4 17 12 13 14 15 16
```

and, at last, this list is reshaped (\$) into the desired square integer evolute:

```
(,~ $ /: @ (+/\) @ evJKT2) 5
24 23 22 21 20
 9 8 7 6 19
10 1 0 5 18
11 2 3 4 17
12 13 14 15 16

evJKT 5
24 23 22 21 20
 9 8 7 6 19
10 1 0 5 18
11 2 3 4 17
12 13 14 15 16
```

The relative timings of the GKPa, GK Pb, KS, EEM, HUI, and JKT verbs are of interest. The column headings give the size of table generated and the row stubs indicate the verb used. The timings are relative to those of JKT set to 1.

verb\size	5	8	13	21	34	55	89
GKPa	12	23	85	110	200	224	244
GK Pb	5	10	37	47	83	94	106
KS	6	8	20	23	33	167	-
EEM	2	4	10	9	12	13	16
HUI	3	3	5	3	3	2	1
JKT	1	1	1	1	1	1	1

The infinite (-) entry for row KS in the column headed 89 indicates that there wasn't enough memory to complete execution. This was probably due to the many levels of recursion required.

The timings show the superiority of array strategies (KS, EEM, HUI, and JKT) over scalar strategies (GKPa and GK Pb), and the superiority of iteration (EEM) over recursion (KS) and the superiority of a strategy minimizing data movement (HUI and JKT) over strategies involving a great deal of data movement (KS and EEM).

I haven't discussed how the Tuttle algorithm can be modified to yield the other volute types. If you look back at the verb evJKT1, you'll see that the expression to the right of the reshape sign (\$) is

```
_1: , ] , 1: , -
```

so that, given the argument 5, it yields

```
_1 5 1 _5
```

The key is that this consists of `_1 5` followed by its negative, `1 _5`. A little experimentation will convince you that the eight possible changes of sign and order of the list `_1 5` will give you all eight types of evolve:

```
1 5    top right clockwise
1 _5   bottom right counter-clockwise
_1 5   top left counter-clockwise
_1 _5  bottom left clockwise
5 1    bottom left counter-clockwise
5 _1   bottom right clockwise
_5 1   top left clockwise
_5 _1  top right counter-clockwise
```

So that to obtain a top left clockwise evolve, one would use:

```
_5 1 5 _1
```

instead of `_1 5 1 _5`.

A verb to yield involutes is somewhat simpler than the verb yielding evolutes. The greater simplicity arises because no attention has to be paid to the first and last items: as they are generated so they are usable:

```
ivJKT0 5                      NB. just the reverse of evJKT0 5
5 4 4 3 3 2 2 1 1
ivJKT1 5                      NB. essentially unchanged
1 5 _1 _5 1 5 _1 _5 1
ivJKT2 5                      NB. different
1 1 1 1 1 5 5 5 5 _1 _1 _1 _1 _5 _5 1 1 1 5 5 _1 _1 _5 1
+/\ivJKT2 5                  NB. same sumscan
1 2 3 4 5 10 15 20 25 24 23 22 21 16 11 6 7 8 9 14 19 18 17 12 13
/:+/\ivJKT2 5                NB. same upgrade
0 1 2 3 4 15 16 17 18 5 14 23 24 19 6 13 22 21 20 7 12 11 10 9 8
5 5$/:+/\ivJKT2 5           NB. same reshape
0 1 2 3 4
15 16 17 18 5
14 23 24 19 6
13 22 21 20 7
12 11 10 9 8
```


Verbs which have not been extended but will be extended, signal nonce error on extended arguments (for example, the monad `<. @o.`, the floor of π times); verbs which have not been extended and will not be extended (for example `%.`, matrix inverse and matrix divide), signal domain error on extended arguments.

Comparisons involving extended integers are exact.

Various non-integer values can be computed to a large number of decimal places using extended integers. For example, the golden ratio *phi* can be computed to 40 decimal places using several different methods:

Method 1. *Phi* is $-\frac{1}{2} + \frac{1}{2}\sqrt{5}$, half of one plus the square root of 5.

```
-: 1 + %: 5
1.61803
```

```
<. @-: s + <. @%: 5 * *: s=. 10^40x
16180339887498948482045868343656381177203x
```

Method 2. *Phi* is the limit of the continued fraction $(+%) / n\$1$.

```
(+%) / 20$1
1.61803
```

Pairs of integers are used to represent fractions. The number of terms required can be estimated using ordinary (non-extended) operations. Thus:

```
reduce=: % +./
plus   =: [: reduce +/@(*|.) , *&{:
recip  =: |.
```

```
reduce 10 25x
2x 5x
```

Reduce to lowest terms

```
3 4x plus 5 6x
19x 12x
```

Sum in lowest terms

```
recip 3 4x
4x 3x
```

Reciprocal

```
(plus recip) / 20$1x
6765x 4181x
```

20 terms of continued fraction

```
x=. (+%) / \ 20$1
```

Now estimate # of terms req'd

```
(-:1+%;5) - x                               Approximation errors
0.618034 _0.381966 0.118034 _0.04863268 0.01803399 ...
```

```
2 %/\ (-:1+%;5) - x                           Ratios in approximation errors
-1.61803 _3.23607 _2.42705 _2.69672 _2.58885 _2.62931 ...
```

```
_5[\ 2 %/\ (-:1+%;5) - x                       Present in 5 columns
-1.61803 _3.23607 _2.42705 _2.69672 _2.58885
-2.62931 _2.61375 _2.61967 _2.61741 _2.61827
-2.61794 _2.61807 _2.61802 _2.61804 _2.61803
-2.61803 _2.61803 _2.61803 _2.61803 0
```

```
2.6 ^ 10^40                                     About 96 terms are required
96.3917
```

```
y=. (plus recip)/\ 100 2$1x
$y
100 2
```

```
.. <.0%/"1 (95)}. (10^40x 0) *"1 y
16180339887498948482045868343656381177204x
16180339887498948482045868343656381177202x
16180339887498948482045868343656381177203x
16180339887498948482045868343656381177202x
16180339887498948482045868343656381177203x
```

Method 3. Using the verbs previously defined to work on integer pairs representing fractions, we have:

```
(plus recip)/\ 10 2$1x
1x 1x
2x 1x
3x 2x
5x 3x
8x 5x
13x 8x
21x 13x
34x 21x
55x 34x
89x 55x
```

It is seen (and easily proved) that partial sums of the continued fraction are ratios of consecutive Fibonacci numbers. Fibonacci numbers are rows of the powers of the matrix $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$, computed efficiently by repeated squaring. (See Hui [1996], "Linear Recurrences and Matrix Powers".) Thus:

```
x =: +/ .*                                     Matrix product
pow=: 4 : 'x/ x~^: (b#i.-#b=, #: y.) x.'
```

```

      J M=: x: 0 1,:1 1
0x 1x
1x 1x

      M pow 4
2x 3x
3x 5x

      M x M x M x M
2x 3x
3x 5x

      M pow 100
218922995834555169026x 354224848179261915075x
354224848179261915075x 573147844013817084101x

      <.@% / (10^40x 0) * !. {: M pow 100
16180339887498948482045868343656381177203x

```

Finally, the phrase `m@|@^` is supported by special code, on extended integer arguments and ordinary integer arguments alike, making it possible to compute the final result even when the intermediate result is enormous. For example:

```

      1e6 | 2 ^ 24
777216

      2 (1e6@|@^) 24
777216

      2 (1e6@|@^) 10^100x
109376x

```

References

Hui, Roger K.W., *Linear Recurrences and Matrix Powers*, Vector, Volume 12, Number 4, 1996 4.

Sullivan, John, *Multi-Precision Arithmetic: Parts I-IV*, Vector, Volume 12, 1-4, 1995 7 and 10, 1996 1 and 4.

The Last Great Empty-Array Joke

by Adrian Smith (Production Manager)

In some ways, Vector assembly just got a great deal easier. With the new release of Dyalog APL come a set of 'standard' font layouts and mappings which should result in some consistency across submissions from Dyalog users in different countries. The good news is that virtually all the APL and national language symbols have the same mappings as are used in the APLPLUS font from APL2000; the bad news is that the zilde and squad characters are different! The odd thing is that Manugistics chose to upset the Icelanders — by overwriting the thorn and eth symbols — while Dyalog chose to upset the French by using the left and right guillemot. Given the relative market for their products in these two countries, I find this a little strange!

The obvious solution, adopted by Vector from this edition forward, is to upset both the Icelanders and the French by duplicating zilde and squad in my APL2741 layout, so that I can accept material in either font and the APL characters will print correctly. What is more, when pasted from our Web site into the *correct interpreter* they will execute correctly. If you paste anything with a zilde into the wrong interpreter, you will get some rather puzzling syntax errors. Squad is not yet a problem.

The moral of the story is: submit APL material to Vector in either APLPLUS or Dyalog International format and I can cope. I know IBM have high hopes for Unicode for their Windows product, but in the short term it would be wonderful if they too adopted one or the other of the accepted mappings for their Windows 95 release. The font (and Duncan's excellent sans serif screen font to match) are free to anyone who can make use of them.

Vectors with Missing Pages

We have received (so far) 5 reports of copies of Vector 13.1 with pages 32-64 missing. Obviously we will replace these free of charge, so if you have this problem (and it bothers you) please let Vector Production know so that we can mail you a replacement.

Index to Advertisers

Causeway Graphical Systems Ltd	2
Dyadic Systems Ltd	19
Nestlé UK Ltd	6
Strand Software	4
Vector Back Numbers	18

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Compuserve: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Duncan Pearson
Keeper's Lodge
Firby
YORK, YO6 7LH
Tel: 01653-618900
Email: 100265.1564@compuserve.com

Authors wishing to use Windows Write or Word for Windows may contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Tel: 01439-788385
Compuserve: 100331,644.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Postcode / Country: _____
 Telephone Number: _____
 Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐
 UK private membership £12 ☐
 Overseas private membership £14 ☐
 Airmail supplement (not needed for Europe) £4 ☐
 UK Corporate membership £100 ☐
 Corporate membership overseas £135 ☐
 Sustaining membership £430 ☐
 Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1996/97 Committee

Chairman	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary	Sylvia Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	Duncan Pearson 01653-618900 100265.1564@compuserve.com	Keeper's Cottage, Firby, YORK, YO6 7LH
Activities	Jon Sandles 01904-612882 100257.1756@compuserve.com	138 Burton Stone Lane, York YO3 6DF
Education	Dr Ian Clark 01388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Projects and Publicity	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Duncan Pearson	Keeper's Cottage, Firby, YORK (01653-618900)
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (01302-539761), Anthony and Sylvia Camacho, Ray Cannon (01252-874697), John Searle (0181-858 6811), David Ziemann (0181-348 4039), Jon Sandles (01904-612882), Ajay Askoolum	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Mallings, Castlegate,
MALTON, North Yorks YO17 0DP
Tel: 01653-696760
Fax: 01653-697719
Email: 100265.1564@compuserve.com
Web: www.causeway.co.uk

Compass R&D Ltd
10 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 01483-302249
Fax: 01483-302279

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants. RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4A 0HA
Tel: 0171-353 8900
Fax: 0171-353 3325
Email: 100020.2632@compuserve.com

Insight Systems Aps
Nordre Strandvej 119A
DK-3150 Hellebæk
Denmark
Tel: +45 42 10 70 22
Fax: +45 42 10 75 74
Email: insight@inet.uni.c.dk

APL2000
2 Independence Way
Suite 100
Princeton NJ 08540 USA
Tel: +1 (609) 734-9692
Fax: +1 (609) 734-9644
Email: sales@apl2000.com
Web: members.aol.com/APL2000

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 0171-922 8866
Fax: 0171-928 1006
Email: microapl@microapl.demon.co.uk
Web: www.microapl.co.uk

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: 03474-2337